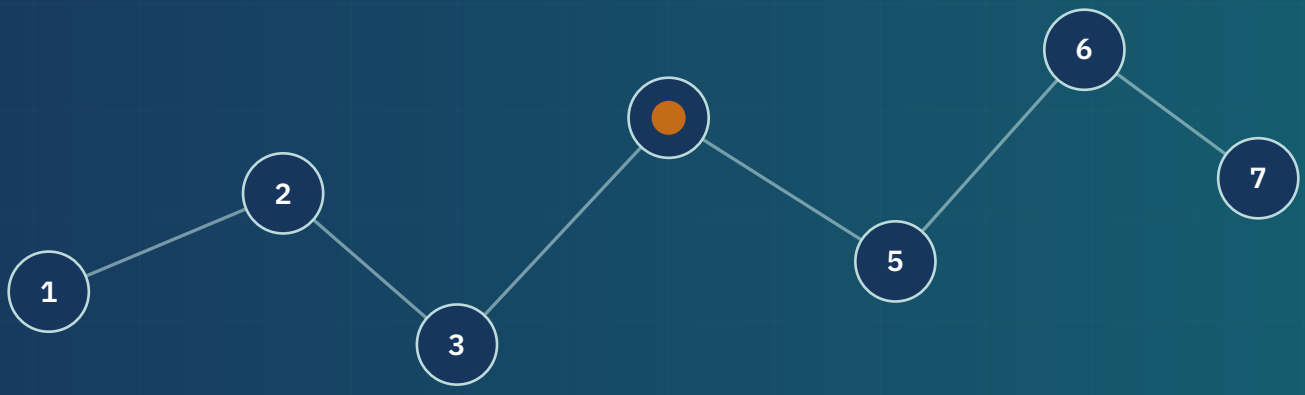
75 VISUAL C++ LESSONS / SOURCE AVAILABLE

CRACKING THE LEETCODE INTERVIEW

Data structures, algorithms, and the reusable ideas behind them.



TRACE THE STATE, THEN WRITE THE CODE

Om Tripathi

ARRAYS TREES GRAPHS HEAPS
GREEDY SEARCH DYNAMIC PROGRAMMING

Cracking The LeetCode Interview

75 Visual Lessons for Technical Interviews

Om Tripathi

Cover Design: Om Tripathi

First digital edition

CONTENTS

Introduction

This book explains 75 common coding interview problems in plain C++. The goal is not to memorize 75 answers. The goal is to see why each solution works and to learn how to rebuild it on your own.

We will work with arrays, strings, linked lists, trees, tries, graphs, heaps, greedy methods, binary search, bits, and dynamic programming. Each lesson starts with the simple idea, follows the changing values through an example, then turns that idea into code.

For every problem, we will focus on the following questions:

1. What is the input?
2. What data structure should we use?
3. What algorithmic technique should we use?
4. What should we do with the data?
5. What should the output be?

We first write the idea in **pseudocode**. Then we translate it into C++. This keeps the reasoning separate from language details and makes the final code easier to understand.

How the C++ code is used

The online judge creates the input, calls the required method, and checks the returned answer. That is why these solution classes usually do not print anything. The code uses ordinary modern C++. Tree, graph, and linked-list node types are the standard types supplied by the judge.

Judge-provided pointer types used throughout the book

Several lessons use structures supplied by the coding platform rather than defined inside every solution. A `ListNode` stores `val` and `next`; a `TreeNode` stores `val`, `left`, and `right`; and a graph `Node` stores a value plus a vector of neighboring node pointers. Complete reference definitions appear in the final C++ reference section.

STL Guide for This Book

The C++ Standard Library gives us ready-made containers and algorithms. The name **STL** is often used for this collection, especially for containers, iterators, and algorithms. This guide covers the standard-library tools used by the 75 solutions in this book.

Version used by the book

Every complete solution in this edition compiles as C++11, C++17, and C++23. The code deliberately avoids newer-only shortcuts when a clear C++11 form is available. This means the same solution can be pasted into an older interview environment without changing the algorithm.

The common setup

Each solution includes the headers it needs and then uses the standard namespace. For example:

```cpp
#include <algorithm>
#include <string>
#include <vector>

using namespace std;

vector<int> values{4, 1, 7};
sort(values.begin(), values.end());
```

After `using namespace std;`, write `vector`, `string`, and `sort` instead of `std::vector`, `std::string`, and `std::sort`. This is convenient in short coding-interview files. In a large project or a header file, many teams prefer the explicit `std::` form because it reduces the chance of two libraries using the same name.

C++11, C++17, and C++23

What changes for this book?

Version	What it means here	Command-line example
C++11	The minimum version required by every solution. It provides `vector`, hash tables, range-based loops, lambdas, `nullptr`, `move`, and the string-to-number functions used here.	`g++ -std=c++11 file.cpp`
C++17	All book code works unchanged. Features such as structured bindings, `optional`, and `string_view` exist, but the solutions do not depend on them.	`g++ -std=c++17 file.cpp`
C++23	All book code still works unchanged. Newer library additions may make other programs shorter, but they do not change the algorithms taught here.	`g++ -std=c++23 file.cpp`

The important rule is simple: choose the version accepted by the judge. If the judge does not say, C++17 is a common practical choice. The solutions in this book remain valid even when the judge selects C++11.

Header guide

Only include the headers needed by the current solution.

Headers used in the 75 solutions

Header	Main names used	Why it is used
`<algorithm>`	`sort`, `min`, `max`, `reverse`, `swap`	Ordering and small reusable algorithms
`<array>`	`array`	Fixed-size frequency tables
`<cctype>`	`isalnum`, `tolower`	Character checks and conversion
`<cstdint>`	`uint32_t`	An integer with an exact bit width
`<functional>`	`greater<int>`	Changes a priority queue into a min heap
`<limits>`	`numeric_limits<int>`	Safe smallest or largest starting values
`<queue>`	`queue`, `priority_queue`	BFS queues and heaps
`<sstream>`	`stringstream`	Reading values from a string
`<stack>`	`stack`	Last-in, first-out processing
`<string>`	`string`, `to_string`, `stoi`, `stoull`	Text storage and number conversion
`<unordered_map>`	`unordered_map`	Key-to-value lookup
`<unordered_set>`	`unordered_set`	Fast membership tests
`<utility>`	`pair`, `move`	Two-value records and moving containers
`<vector>`	`vector`	Resizable arrays and most returned lists

Vector and array functions

Resizable and fixed-size arrays

Operation	Meaning	Usual cost
`v[i]`	Read or change the value at index `i`.	$O(1)$
`v.size()`	Number of stored values.	$O(1)$
`v.empty()`	True when the container has no values.	$O(1)$
`v.push_back(x)`	Add `x` at the end.	$O(1)$ average
`v.back()`	Read the final value.	$O(1)$
`v.reserve(n)`	Reserve room before repeated appends.	$O(n)$ when storage moves
`v.clear()`	Remove all values.	Linear in stored objects
`v.begin()`, `v.end()`	Iterator range covering every value.	$O(1)$
`a.fill(x)`	Put `x` in every cell of a fixed `array`.	$O(n)$

```cpp
#include <array>
#include <vector>

using namespace std;

vector<int> values;
values.reserve(3);
values.push_back(4);
values.push_back(7);

array<int, 26> count{};
count.fill(0);
```

Use `vector` when the size changes. Use `array<T,N>` when the size is known at compile time, such as the 26 lowercase English letters.

String functions

Text operations used in the book

Operation	Meaning and important note
`s[i]`	Character at index `i`; check the index first.
`s.size()`	Number of characters; the return type is unsigned.
`s.empty()`	True when the string has no characters; clearer than `s.size()==0`.
`substr(pos,len)`	Copy part of a string; cost is proportional to the copied text.
`s.find(ch,pos)`	Find the next character or substring; returns `string::npos` if absent.
`s.push_back(ch)`	Add one character; constant time on average.
`to_string(x)`	Convert a number to a string; available in C++11.
`stoi(s)`	Convert text to `int`; may throw for invalid text.
`stoull(s)`	Convert text to `unsigned long long`; useful for stored lengths.

```cpp
#include <string>

using namespace std;

string encoded = "4#code";
size_t mark = encoded.find('#');
int length = stoi(encoded.substr(0, mark));
string word = encoded.substr(mark + 1, length);
```

Hash map and hash set functions

Both containers use hashing. Average lookup, insertion, and removal are $O(1)$, although the worst case can be $O(n)$.

Fast lookup operations

Operation	Meaning
`table[key]`	Read or create the value stored for `key`.
`table.find(key)`	Return an iterator to the key, or `table.end()`.
`table.count(key)`	Return 0 or 1 for the unique-key containers used here.
`table.insert(x)`	Add a key or a key-value pair.
`table.clear()`	Remove all entries.
`for (auto& entry : table)`	Visit every stored entry; the order is unspecified.

```cpp
#include <unordered_map>
#include <unordered_set>

using namespace std;

unordered_map<int, int> indexOf;
indexOf[7] = 3;
if (indexOf.count(7)) {
    int index = indexOf[7];
}

unordered_set<int> seen;
seen.insert(12);
bool alreadySeen = seen.count(12) != 0;
```

Do not use `operator[]` only to test whether a map key exists: it may create a new entry. Use `count` or `find` for a pure lookup.

Queue, stack, and priority queue

The accessible end of each structure

Type	Add, read, remove	Typical use
Queue	`push`, `front`, `pop`	BFS and first-in, first-out work
Stack	`push`, `top`, `pop`	Nested delimiters and last-in, first-out work
Max heap	`push`, `top`, `pop`	Repeatedly take the largest value
Min heap	`push`, `top`, `pop`	Repeatedly take the smallest value; the full type appears below.

```cpp
#include <functional>
#include <queue>
#include <vector>

using namespace std;

queue<int> bfs;
bfs.push(3);
int nextNode = bfs.front();
bfs.pop();

priority_queue<int> maxHeap;
priority_queue<int, vector<int>, greater<int>> minHeap;
```

Calling `pop()` does not return the removed value. Read `front()` or `top()` first, then call `pop()`.

Algorithms used in the book

Standard algorithms normally receive a half-open range: `[begin,end)`. The first iterator is included; the end iterator points one position after the final element.

Algorithm quick reference

Operation	Meaning	Cost
`sort(range)`	Sort in increasing order.	$O(n \log n)$
`sort(range,rule)`	Sort using a custom rule.	$O(n \log n)$
`min(a,b)`, `max(a,b)`	Return the smaller or larger value.	$O(1)$
`reverse(range)`	Reverse a range in place.	$O(n)$
`swap(a,b)`	Exchange two values.	Usually $O(1)$

The following comparison works in C++11 because its parameter types are written explicitly:

```
#include <algorithm>
#include <vector>

using namespace std;

vector<vector<int>> intervals{{1, 3}, {2, 4}, {3, 5}};
sort(intervals.begin(), intervals.end(),
     [](const vector<int>& a, const vector<int>& b) {
         return a[1] < b[1];
     });
```

Writing `const auto&` as a lambda parameter is shorter, but generic lambda parameters require C++14 or newer.

Numbers, characters, streams, and moves

Other library tools used by the solutions

Name	Purpose
Smallest integer	`numeric_limits<int>::min()` is useful when every valid answer may be negative.
Largest integer	`numeric_limits<int>::max()` is useful as an initial upper bound.
Exact 32-bit integer	`uint32_t` is unsigned and has exactly 32 bits when the implementation provides it.
Character check	`isalnum(ch)` tests whether a character is a letter or digit.
Lowercase conversion	`tolower(ch)` produces the lowercase form of a character.
String stream	`stringstream` reads formatted values from a string.
Move	`move(value)` allows a container's stored data to be transferred instead of copied when possible.
Pair	`pair<A,B>` stores two related values in `first` and `second`.

When calling `isalnum` or `tolower`, cast a possibly negative `char` to `unsigned char` first. This avoids undefined behavior for characters outside the basic ASCII range.

Newer shortcuts not required here

The following features are useful, but using them would raise the minimum language version. The right column shows the portable form used in this book.

Version-sensitive alternatives

Newer feature	First version	Form used in this book
Generic lambda	C++14	Replace an `auto` parameter with the full type.
Structured bindings	C++17	Use `pair.first`, `pair.second`, or an ordinary loop variable.
Views and optional values	C++17	Use `string`, indexes, pointers, or a separate Boolean value.
Membership check	C++20	Replace `contains(key)` with `count(key)` or `find(key)!=end()`.
Range sorting	C++20	Replace `ranges::sort(values)` with `sort(values.begin(),values.end())`.
Bit count	C++20	Replace `popcount(value)` by clearing the lowest set bit in a loop.
Safe midpoint	C++20	Use `left+(right-left)/2`.
Formatted print	C++23	Online-judge solutions return values; they do not need formatted printing.

Fast checklist before submitting

STL mistakes that often cause wrong answers

- Include the correct header instead of relying on another header to include it by accident.
- Check `empty()` before calling `front()`, `back()`, or `top()`.
- Remember that `pop()` removes an item but does not return it.
- Do not dereference `end()` after an unsuccessful `find()`.
- Remember that `unordered_map` iteration order is not sorted.
- Use `static_cast<int>(container.size())` when mixing a size with signed integer indexes.
- Count sorting and heap operations as $O(n \log n)$ when explaining the final complexity.

The rest of the book introduces these tools inside real problems. Return to this guide whenever a container operation or header name is unfamiliar.

Visual Data Structure Guide

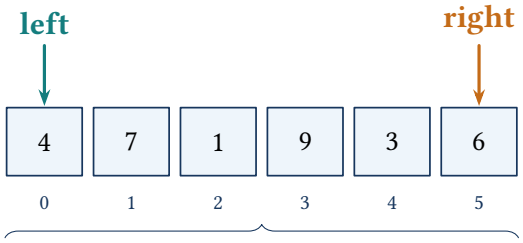
Algorithms are easier to follow when saved values have a clear shape. Orange shows what we are working on now, teal shows what is finished or kept, and navy shows saved information. The different borders also remain clear when the page is printed in grayscale.

Visual legend used throughout the book

Array and `vector`

An array is a row of values. Each box has a number called an index. A C++ `vector` can grow, but we still use indexes in the same way.

An array is a row of numbered boxes



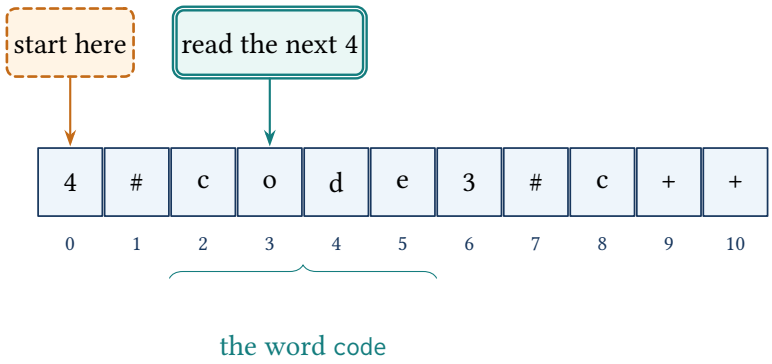
use index `i` to open box `values[i]`

Arrays appear in two-pointer scans, binary search, bit results, prices, house values, traversal orders, and one-dimensional dynamic programming.

String and reading position

A string is a row of characters. We keep one position to show where reading starts and use the length to know where it stops.

The first number tells us how many letters to read

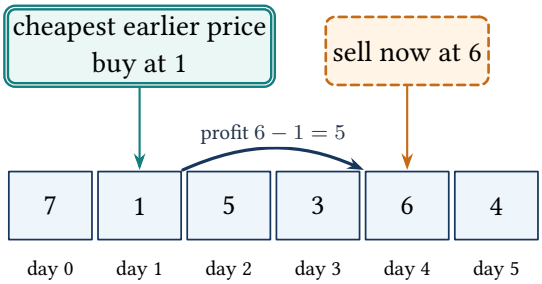


The first number is 4, so we read the next four letters.

Remember the best earlier value

For stock profit, we only need the cheapest price seen before today.

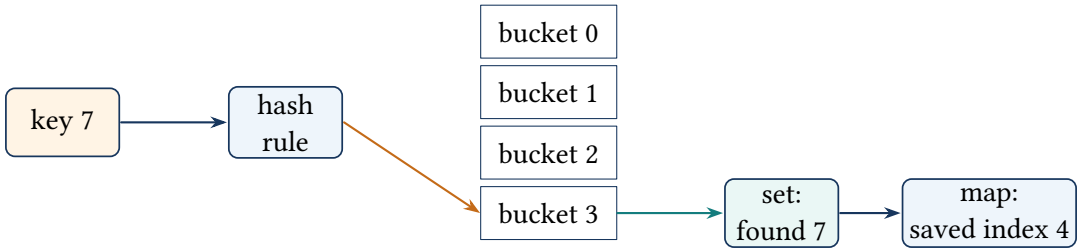
For each sell day, use the cheapest earlier price



Hash set and hash map

A hash set answers "Have I seen this key?" A hash map saves extra information beside the key, such as an index.

A key is changed into a small bucket number

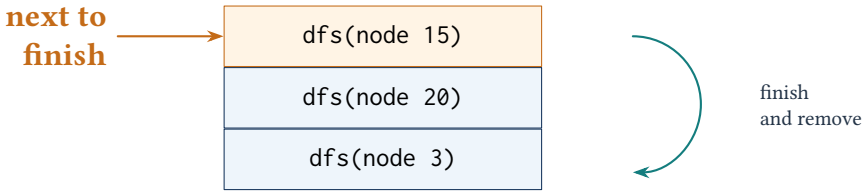


A hash table normally finds a saved key quickly. Spreading keys among the buckets keeps it fast.

Stack and function calls

A stack handles the newest item first. Function calls use a stack too: the newest call finishes before the older call below it.

Function calls stack on top of one another

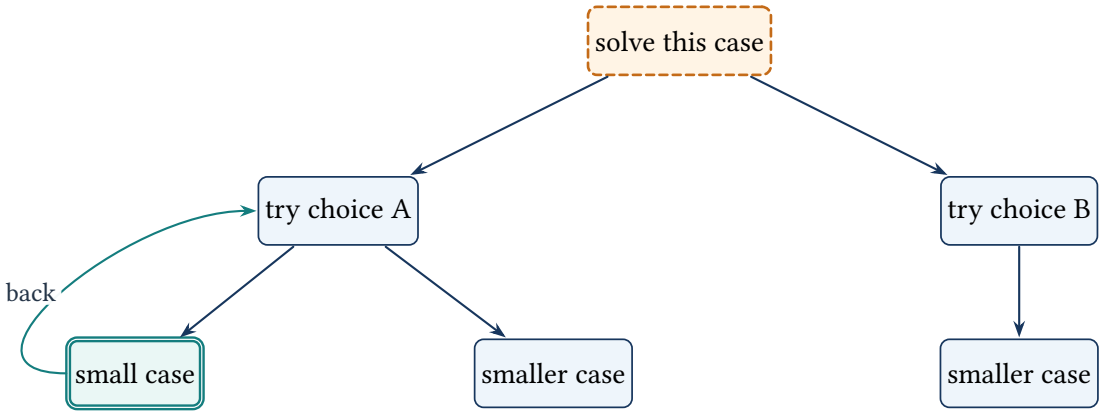


The stack grows by one box for each unfinished function call.

A problem can split into choices

This picture shows every choice a function may try. Each choice creates a smaller version of the same problem.

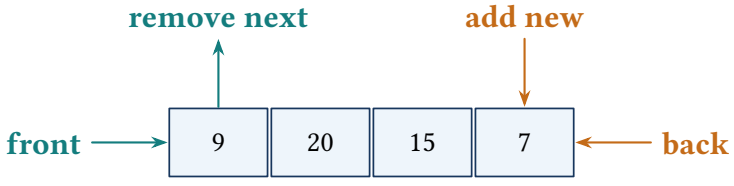
One problem splits into smaller choices



Queue

A queue removes the oldest item first. We remove from the front and add new items at the back.

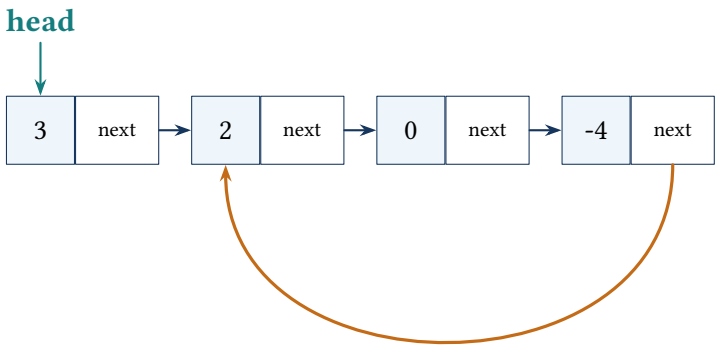
Queue: remove at the front, add at the back



Singly linked list

Each node stores a value and an arrow to the next node. To move through the list, follow the arrows.

Each node points to the next node

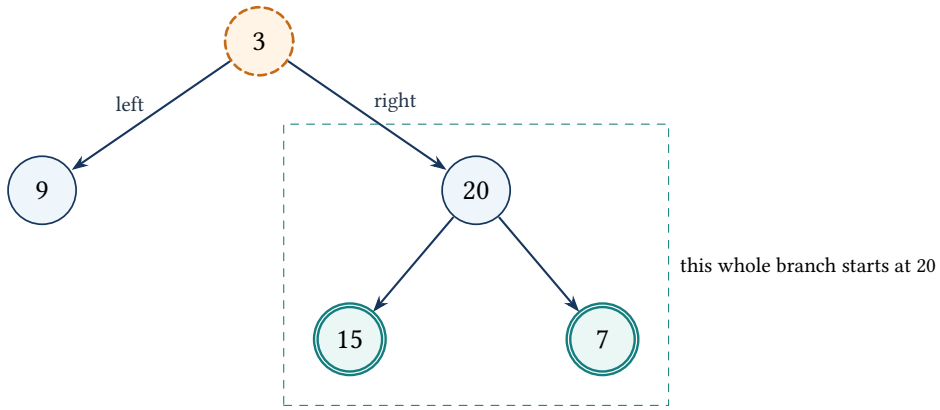


last node points back to 2

Binary tree

A binary-tree node can point to a left child and a right child. Every node below one node forms that node's branch.

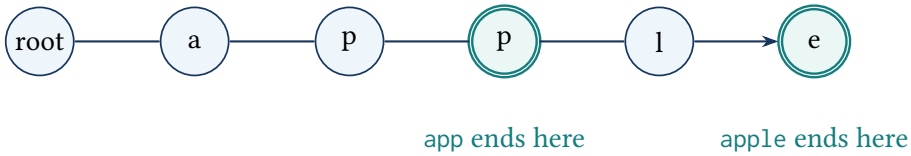
A node can have a left and a right child



Trie: words that share a beginning

A Trie stores one letter at a time. Words with the same beginning follow the same path until their letters become different.

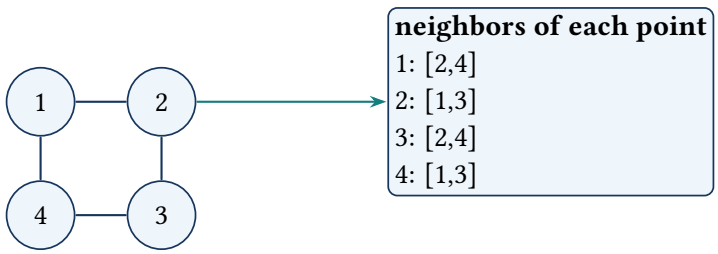
The words `app` and `apple` share one path



Graph and adjacency list

A graph has numbered points joined by lines. For each point, a neighbor list records the points directly connected to it.

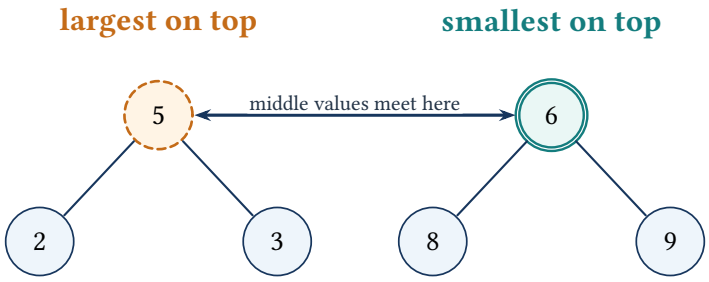
The same graph shown as neighbor lists



Heap: quickly see the largest or smallest value

A max heap keeps the largest value on top. A min heap keeps the smallest value on top, so that value can be taken next.

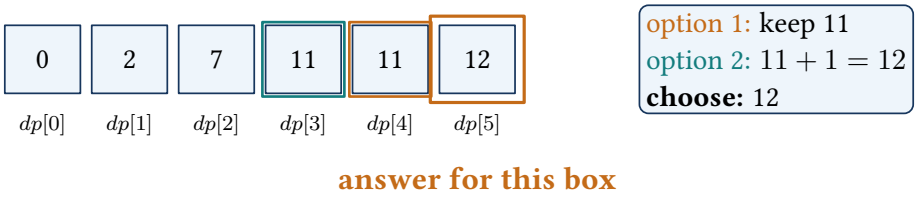
The top shows the next value we can take



A table that reuses earlier answers

A DP array saves answers already found. The colored boxes show which earlier answers are used for the next box.

Choose the better of two earlier answers

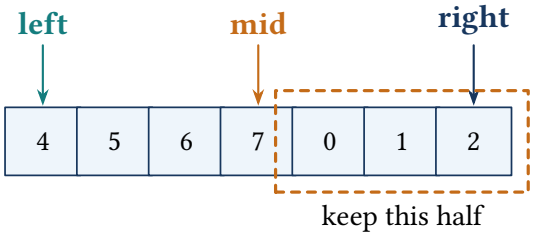


First decide what each box means. Then fill the boxes only after the earlier answers they need are ready.

Binary-search interval

Binary search keeps the part that may still contain the answer and throws away the half that cannot contain it.

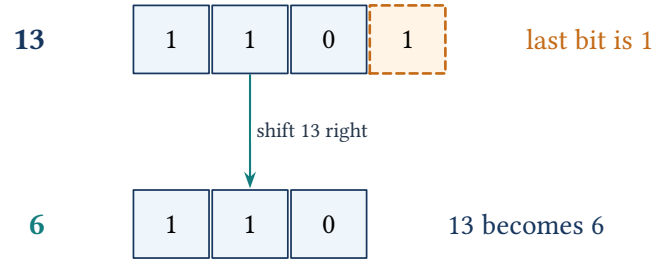
Keep only the half that may contain the answer



Binary digits and right shift

A number can be written with only 0s and 1s. Shifting right removes the last binary digit and leaves a smaller number.

Shifting right drops the last binary digit



ARRAY PATTERNS

Lesson 1: Two Sum

Problem at a glance

Difficulty	Approach	Main tool
Easy	Arrays and hashing	`unordered_map`

The Problem

You receive a sequence of integers and a required total. Find two different positions whose values combine to that total, then return those positions. Assume the input has one valid pair.

Examples and Rules

Read the output carefully

The problem returns *indices*, not the two values. Array positions use zero-based indexing, and the same position cannot be used twice.

Input	Matching values	Output
`[4,10,-2,7]`, target `8`	$10 + (-2) = 8$	`[1,2]`
`[5,1,9]`, target `14`	$5 + 9 = 14$	`[0,2]`
`[6,6]`, target `12`	$6 + 6 = 12$	`[0,1]`

For a compact dry run, use `[2,7,11,15]` with target 9. Its indexed array is

Index	0	1	2	3
Value	2	7	11	15

Because `nums[0]` + `nums[1]` $= 2 + 7 = 9$, the answer is `[0,1]`.

What We Need to Find

The input consists of:

- An array of integers called `nums`.
- An integer called `target`.

Our goal is to find two elements in the array whose sum is equal to the target and return their indices.

Data Structure: Hash Table

We can use a **hash table** to efficiently solve this problem.

In C++, an `unordered_map` can be used to implement a hash table.

The hash table will store values that we have already visited, along with their matching indices.

The main idea is to calculate the **complement** of the current element.

If the current element is x, then the required complement is: complement $=$ `target` $- x$

This comes from: $x +$ complement $=$ `target`

So: $\boxed{\text{complement} = \texttt{target} - x}$

For every element in the array, we check whether this complement already exists in our hash table.

How the Hash Table Works

While iterating through the array, we store every element that we have already visited in the hash table.

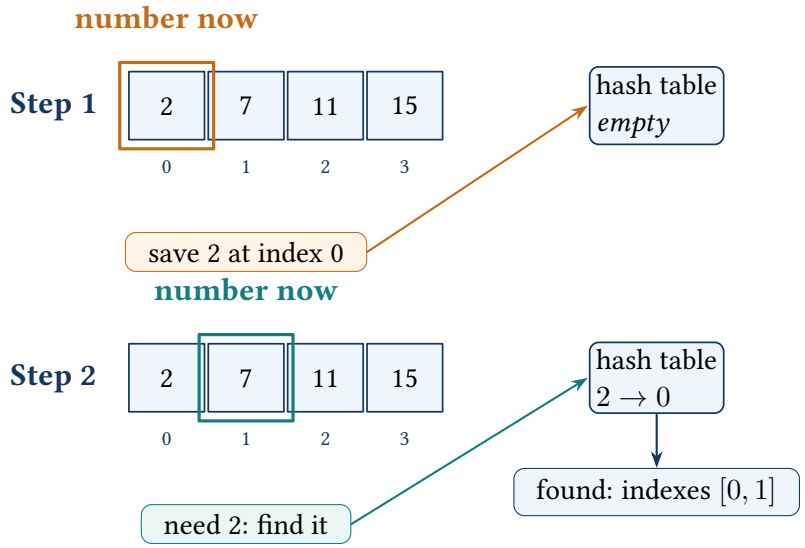
For the current element, we calculate: complement $=$ `target` $-$ current element

We then check whether the complement exists in the hash table.

- If the complement exists, we have found the required pair.
- If the complement does not exist, we store the current element and its index in the hash table.

This lets us find the required pair in a single traversal of the array.

Two Sum: save a number, then find the one needed



Before checking a number, the table contains only numbers seen earlier.

Key idea: the table contains only earlier numbers

Before index i is processed, every entry in the hash map came from an index smaller than i. A successful complement lookup uses two different positions and automatically keeps the scan order.

The two rows above are the full dry run. The first value is remembered. At the next index, the needed value is already in the table, so the answer is found without scanning the earlier part of the array again.

Pseudocode

The overall pseudocode is:

1. Start an empty hash table to store previously seen values and their indices.
2. Loop over every element in the array.
3. For the current element, calculate its complement: complement $=$ `target` $-$ current element
4. Check whether the complement exists in the hash table.
5. If the complement exists:
   - Return the index of the complement from the hash table.
   - Return the index of the current element.
6. If the complement does not exist:
   - Insert the current element and its index into the hash table.
7. If no pair is found, return an empty array as an edge case.

The hash table can be implemented using `unordered_map` in C++.

Code 1: Complete C++ solution: Two Sum

```cpp
#include <unordered_map>
#include <vector>

using namespace std;

class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> hashTable;

        for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
            const int complement = target - nums[i];

            if (hashTable.count(complement)) {
                return {hashTable[complement], i};
            }

```

```
            hashTable[nums[i]] = i;
        }

        return {};
    }
};
```

Code Walkthrough

Step 1: Start the Hash Table

We first create an empty hash table:

```
unordered_map<int, int> hashTable;
```

The key represents the value from the array, while the value stored in the hash table represents its index.

For example: `hashTable[2] = 0`

means that the value 2 was found at index 0.

Step 2: Traverse the Array

We iterate through the array:

```
for (int i = 0; i < nums.size(); i++)
```

Each element is processed exactly once.

Step 3: Calculate the Complement

For the current element `nums[i]`, calculate:

```
int complement = target - nums[i];
```

For example, if: `target` $= 9$

and: `nums[i]` $= 2$

then: complement $= 9 - 2 = 7$

So, we need to determine whether 7 has already appeared in the array.

Step 4: Check the Hash Table

We check whether the complement exists:

```
if (hashTable.count(complement))
```

If it exists, we have found the required pair.

We can then return:

```
return {hashTable[complement], i};
```

The hash table provides the index of the previously visited element, while i is the index of the current element.

Step 5: Store the Current Element

If the complement does not exist, we store the current element and its index:

```
hashTable[nums[i]] = i;
```

This allows future elements to check whether the current element is their required complement.

Edge Case

If no valid pair is found, we return an empty array:

```
return {};
```

This handles the case where there are no elements that satisfy the required condition.

Time and Space

Time Complexity

The algorithm iterates through the array only once.

So, the time complexity is: $\boxed{O(n)}$

where n is the number of elements in the input array.

The hash table allows us to check whether a complement exists in constant average time.

Space Complexity

In the worst case, we may need to store every element of the array in the hash table.

So, the space complexity is: $\boxed{O(n)}$

So, the overall complexity is: $\boxed{\text{Time} = O(n)}$ $\boxed{\text{Space} = O(n)}$

ARRAY PATTERNS

Lesson 2: 3Sum

Problem at a glance

Difficulty	Approach	Main tool
Medium	Sorting and two pointers	Sorted array

The Problem

Find every distinct group of three values whose sum is zero. Each group must use three different array positions, and the returned collection must not repeat the same value triplet.

Example

Consider: `nums` $= [-1, 0, 1, 2, -1, -4]$

The valid triplets are: $[-1, -1, 2]$

and $[-1, 0, 1]$

Both triplets have a sum of zero: $-1 + (-1) + 2 = 0$

and $-1 + 0 + 1 = 0$

So, the output is: $[[-1, -1, 2], [-1, 0, 1]]$

Another example is: `nums` $= [0, 1, 1]$

There is no combination of three distinct indices whose values sum to zero, so the result is an empty list.

What We Need to Find

The input is an array of integers called `nums`.

We need to return all triplets satisfying: `nums[i]` + `nums[j]` + `nums[k]` $= 0$

where all three indices are different.

We must also make sure that duplicate triplets are not included in the result.

Algorithmic Technique: Two Pointers

The technique used for this problem is the **two-pointer technique**.

The general idea is:

1. Sort the input array.
2. Traverse the array from the beginning.
3. For each element, use two pointers to search for the remaining two elements.

For a fixed element `nums[i]`, we need to find two values whose sum is: $-$`nums[i]`

The two pointers are:

- `j`: starts immediately after `i`.
- `k`: starts at the end of the array.

We then calculate: `sum` $=$ `nums[i]` $+$ `nums[j]` $+$ `nums[k]`

Because the array is sorted, we can move the pointers intelligently.

Pointer Movement

If: `sum` < 0

then the sum is too small, so we increment `j`: `j++`

This moves toward a larger value.

If: `sum` > 0

then the sum is too large, so we decrement `k`: `k--`

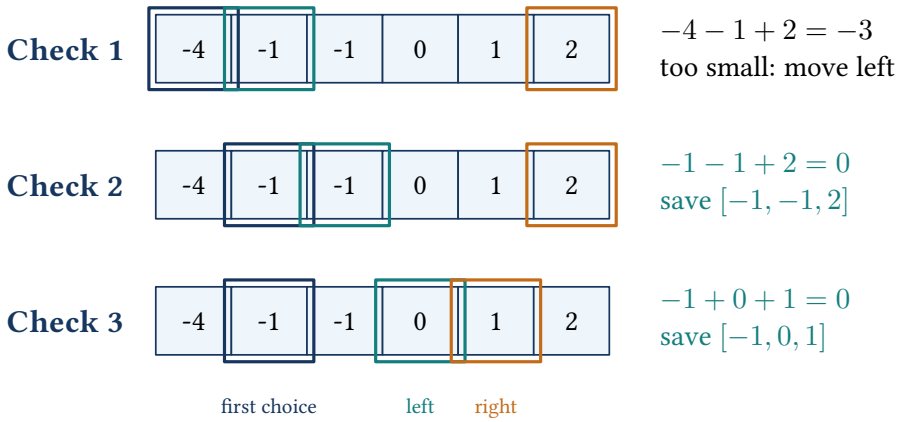
This moves toward a smaller value.

If: `sum` $= 0$

then we have found a valid triplet.

We add it to the result and move both pointers.

Pick one number, then move left and right



If the sum is too small, move left to a larger number. If it is too large, move right to a smaller number.

Key idea: every pair still worth checking lies between left and right

After choosing the first number, the remaining possible pairs lie between the left and right positions. A small sum moves left; a large sum moves right.

The three array rows show the important pointer changes. Blue marks the fixed value, teal marks the left pointer, and orange marks the right pointer.

Pseudocode

The overall process is:

1. Start an empty result vector.
2. If the size of `nums` is less than 3, return the empty result.
3. Sort the input array.
4. Traverse the array using index `i`.
5. Skip duplicate values for `i`.
6. Set: `j` $= i + 1$ and `k` $= n - 1$
7. While: `j` $<$ `k` calculate: `sum` $=$ `nums[i]` $+$ `nums[j]` $+$ `nums[k]`
8. If the sum is zero:
   - Add the triplet to the result.
   - Skip duplicate values for `j`.
   - Skip duplicate values for `k`.
   - Increment `j`.
   - Decrement `k`.
9. Else if the sum is less than zero: `j++`
10. Otherwise: `k--`
11. Return the result.

Code 2: Complete C++ solution: 3Sum

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
```

```
        vector<vector<int>> result;
        if (nums.size() < 3) return result;
        sort(nums.begin(), nums.end());
        const int n = static_cast<int>(nums.size());
        for (int i = 0; i < n - 2; ++i) {
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            int j = i + 1;
            int k = n - 1;
            while (j < k) {
                const long long sum =
                    static_cast<long long>(nums[i]) + nums[j] + nums[k];
                if (sum == 0) {
                    result.push_back({nums[i], nums[j], nums[k]});
                    while (j < k && nums[j] == nums[j + 1]) ++j;
                    while (j < k && nums[k] == nums[k - 1]) --k;
                    ++j;
                    --k;
                } else if (sum < 0) {
                    ++j;
                } else {
                    --k;
                }
            }
        }
        return result;
    }
};
```

Code Walkthrough

Step 1: Start the Result

We first create a vector to store all valid triplets:

```
vector<vector<int>> result;
```

If there are fewer than three elements, a triplet cannot be formed:

```
if (nums.size() < 3) {
    return result;
}
```

Step 2: Sort the Array

We sort the array:

```
sort(nums.begin(), nums.end());
```

Sorting is important because it allows the two-pointer technique to determine which pointer should move based on the current sum.

For example: $[0, -1, 2, -1, -4, 1]$

becomes: $[-4, -1, -1, 0, 1, 2]$

Step 3: Fix the First Element

We traverse the array:

```
for (int i = 0; i < nums.size() - 2; i++)
```

The index `i` represents the first element of the triplet.

For every fixed `i`, we search for two additional values.

Handling Duplicate Values for `i`

Since duplicate triplets are not allowed, we skip duplicate values:

```
if (i > 0 && nums[i] == nums[i - 1]) {
    continue;
}
```

This prevents us from processing the same first value multiple times.

Step 4: Start the Two Pointers

For every fixed `i`, we start:

```
int j = i + 1;
int k = nums.size() - 1;
```

So: $i < j < k$

The pointer `j` starts immediately after `i`, while `k` starts at the last element.

Step 5: Calculate the Sum

While: `j` $<$ `k`

we calculate:

```
int sum = nums[i] + nums[j] + nums[k];
```

There are three possibilities.

Case 1: Sum Equals Zero

If: `sum` $= 0$

we have found a valid triplet.

We add it to the result:

```
result.push_back({
    nums[i],
    nums[j],
    nums[k]
});
```

We then skip duplicate values for both pointers and move them toward each other.

Case 2: Sum Is Less Than Zero

If: `sum` < 0

we need to increase the sum.

Because the array is sorted, we move `j` to the right:

```
j++;
```

This gives us a potentially larger value.

Case 3: Sum Is Greater Than Zero

If: `sum` > 0

we need to decrease the sum.

Because the array is sorted, we move `k` to the left:

```
k--;
```

This gives us a potentially smaller value.

Why Sorting Helps

Sorting creates an important property: `nums[j]` $\leq$ `nums[k]`

So, if the current sum is too small, moving `j` forward increases the sum.

If the current sum is too large, moving `k` backward decreases the sum.

This lets us search for the required pair efficiently without checking every possible pair.

Handling Duplicates

Duplicate triplets are not allowed.

After finding a valid triplet, we skip duplicate values for `j`:

```
while (j < k && nums[j] == nums[j + 1]) {
    j++;
}
```

Similarly, we skip duplicate values for `k`:

```
while (j < k && nums[k] == nums[k - 1]) {
    k--;
}
```

After that, both pointers are moved:

```
j++;
k--;
```

Time and Space

The array is first sorted, which takes: $O(n \log n)$

After sorting, we use a loop over the array and a two-pointer scan for each fixed element.

The resulting dominant complexity is: $\boxed{O(n^2)}$

where n is the number of elements in the input array.

The sorting step is smaller than the $O(n^2)$ two-pointer processing for the overall asymptotic complexity.

The extra space depends on the sorting implementation. The two-pointer portion itself uses constant extra space, aside from the space required for the returned result. The overall time complexity is $O(n^2)$, and the two-pointer scan uses $O(1)$ extra space, excluding the output vector and implementation-dependent sorting stack space.

3Sum Summary

The key idea is:

1. Sort the array.
2. Fix one element.
3. Use two pointers to find the remaining two elements.
4. If the sum is too small, move the left pointer right.
5. If the sum is too large, move the right pointer left.
6. If the sum is zero, record the triplet.
7. Skip duplicates to ensure unique triplets.

So, the two-pointer technique reduces the problem to an efficient $O(n^2)$ solution.

ARRAY PATTERNS

Lesson 3: Best Time to Buy and Sell Stock

Problem at a glance

Difficulty	Approach	Main tool
Easy	Greedy running minimum	Array

The Problem

The array `prices` stores the stock price for each day. Choose one day to buy and a later day to sell. Return the greatest possible profit. If every possible transaction loses money, return zero.

The ordering rule

Buy before sell

For a buy price B and a later selling price S, profit $= S - B$. The algorithm may not use a low price that appears after the selling day.

Examples and Rules

Prices	Best transaction	Answer
`[7,1,5,3,6,4]`	buy at 1, sell at 6	5
`[7,6,4,3,1]`	no profitable sale	0

The first example earns $6 - 1 = 5$. In the decreasing example, every future price is smaller than its earlier prices, so the best legal answer is zero.

First Idea: Try Every Transaction

The direct method chooses every pair of days (i, j) with $i < j$, computes `prices[j]` $-$ `prices[i]`, and keeps the largest result. There are about $n(n-1)/2$ legal pairs, so this approach takes $O(n^2)$ time.

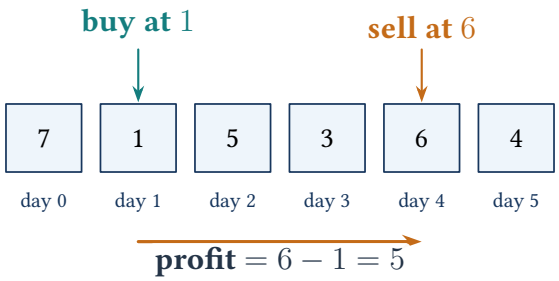
The repeated work is the clue: for every selling day, the brute-force method searches the earlier days again to find the cheapest possible purchase.

Greedy Observation

For a fixed selling day, the best buying day is always the cheapest earlier day. So, while scanning from left to right, we only need to remember the minimum price seen so far.

minimum previous price $\longrightarrow$ current selling price $\longrightarrow$ best profit

For each selling day, keep the cheapest earlier price



When day 4 is the selling day, 1 is the cheapest price in days 0 through 3. No other earlier buying price can produce a larger profit that day.

The Two Variables

State keeped during the scan

`minPrice`	Smallest price seen up to the current day. Start at positive infinity.
`maxProfit`	Greatest legal profit found so far. Start at zero so a loss is never returned.

Key idea: the two variables summarize the entire processed prefix

Before moving to the next day, `minPrice` is the cheapest price in the processed prefix, and `maxProfit` is the best legal transaction entirely inside that prefix. No older price needs to be stored separately.

Step-by-Step Algorithm

For every price, perform three operations in this order:

1. Update the cheapest price: `minPrice` $= \min($`minPrice`$,$ `price`$)$.
2. Calculate the profit obtained by selling today: `currentProfit` $=$ `price` $-$ `minPrice`.
3. Update the best result: `maxProfit` $= \max($`maxProfit`$,$ `currentProfit`$)$.

Updating `minPrice` first allows the current day to act as both the new minimum and a zero-profit sale. This never creates an invalid positive transaction, and future days may use that price as their buying day.

Detailed Visual Dry Run

Scanning `[7,1,5,3,6,4]`

Day	Price	`minPrice`	Sell-today profit	`maxProfit`
0	7	7	0	0
1	1	1	0	0
2	5	1	4	4
3	3	1	2	4
4	6	1	5	5
5	4	1	3	5

The minimum changes only when a cheaper buying opportunity appears. The maximum profit changes only when today's selling price improves the answer.

Why the Greedy Choice Is Correct

Suppose the current selling price is P. For any legal earlier buying price B, the profit is $P - B$. With P fixed, this expression is largest when B is as small as possible. So, the cheapest earlier price is the only buying choice needed for the current day.

Every day is considered once as a possible selling day. As a result, every best transaction is considered when its selling day is processed, and the global maximum is correct.

Pseudocode

```
maxProfit(prices):
    minPrice = positive infinity
    bestProfit = 0

    for each price in prices:
        minPrice = min(minPrice, price)
        profitToday = price - minPrice
        bestProfit = max(bestProfit, profitToday)

    return bestProfit
```

Code 3: Complete C++ solution: Best Time to Buy and Sell Stock

```cpp
#include <algorithm>
#include <limits>
#include <vector>

using namespace std;

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minPrice = numeric_limits<int>::max();
        int maxProfitSeen = 0;
```

```

        for (const int price : prices) {
            minPrice = min(minPrice, price);
            const int currentProfit = price - minPrice;
            maxProfitSeen = max(maxProfitSeen, currentProfit);
        }

        return maxProfitSeen;
    }
};
```

Output and local testing

This class does not print output by itself. The online judge creates the input, calls `maxProfit`, and checks the returned integer. A separate local test harness was used to compile and validate the method.

Compact Interview Version

The same update can be written directly with `std::min` and `std::max`. The part to remember is the key rule, not the number of lines:

```
for (const int price : prices) {
    minPrice = min(minPrice, price);
    maxProfitSeen = max(maxProfitSeen, price - minPrice);
}
```

Time and Space

Let n be the number of days. The loop visits every price exactly once, so the time complexity is $O(n)$. Only two running values are stored, so the extra space is $O(1)$.

Why the extra space is constant

The implementation stores only a constant number of integers regardless of the input length. So, its extra-space complexity is $O(1)$.

Edge Cases and Common Mistakes

- An empty array returns zero because the loop performs no update.
- One price cannot form a transaction, so the answer remains zero.
- A decreasing array must return zero, not the smallest negative loss.
- Update the running minimum using only the current and earlier days.
- Do not solve the multiple-transaction variation; this problem allows one transaction.
- Do not sort the prices. Sorting destroys the original day order.

How to Explain It in an Interview

How to explain it in an interview

I would first mention the $O(n^2)$ pair check. It repeatedly searches for a buying price before each sale. During one left-to-right pass, I instead keep the cheapest price seen so far. For each current price, I calculate the profit from selling today and update the global maximum. The key rule is that the minimum and best profit are correct for the processed prefix. Each element is visited once, so the algorithm takes $O(n)$ time and $O(1)$ extra space.

Useful clarifying questions include whether only one transaction is allowed, whether buying and selling must occur on different days, and whether returning zero is required when no profit exists.

Pattern-Recognition Clue

Consider this greedy pattern when each current value should be combined with the best earlier value:

remember the best earlier value $\rightarrow$ use today's value

$\Downarrow$

update the best answer

Revision Summary

Best Time to Buy and Sell Stock

Technique	Greedy running minimum
Keep	`minPrice` and `maxProfitSeen`
Update	Compare today's price with the minimum, then test selling today
Complexity	$O(n)$ time and $O(1)$ extra space

TRIE PATTERNS

Lesson 4: Design Add and Search Words Data Structure

Problem at a glance

Difficulty	Approach	Main tool
Medium	Trie and recursive DFS	Trie

Problem Overview

The goal of this problem is to design a data structure that supports adding words and searching for words.

The problem is closely related to the **Trie** (Prefix Tree) data structure that we have used previously.

The main operations are:

- Add a word to the data structure.
- Search whether a given word exists.

The important idea is that the words will be stored character by character inside a Trie.

Approach

We will use a **Trie** data structure.

The overall steps are:

1. Create a Trie node structure.
2. Each node contains an array of 26 children, one for every lowercase English letter.
3. Each node also contains a Boolean variable indicating whether the node represents the end of a complete word.
4. Create a root Trie node.
5. While adding a word, traverse its characters one by one.
6. Create missing child nodes when needed.
7. Mark the final node as the end of the word.
8. During searching, traverse the Trie according to the characters of the word.
9. Return whether the complete word exists.

Key idea: a DFS state represents one matched pattern prefix

A call `dfs(word,index,node)` means that the first `index` pattern characters match the Trie path ending at `node`. A normal letter follows one child; a dot explores every non-null child while preserving the same state meaning.

Trie Node

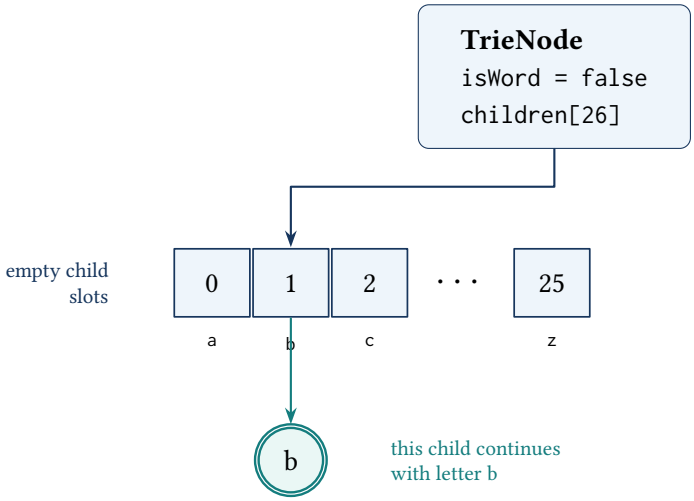
We create a structure called `TrieNode`.

Each node contains:

- An array of 26 child pointers.
- A Boolean variable `isWord`.

The Boolean variable tells us whether the current node represents the end of a complete word.

What one Trie node stores



Each numbered slot stands for one letter. A used slot points to the next letter. `isWord` tells us whether a complete word ends here.

Trie Node Structure

```
struct TrieNode {
    TrieNode* children[26];
    bool isWord;

    TrieNode() {
        isWord = false;

        for (int i = 0; i < 26; i++) {
            children[i] = nullptr;
        }
    }
```

```
};
```

A new node starts with every child pointer set to `nullptr` and `isWord = false`. It has no outgoing character path and does not yet mark the end of a stored word.

Creating the Root

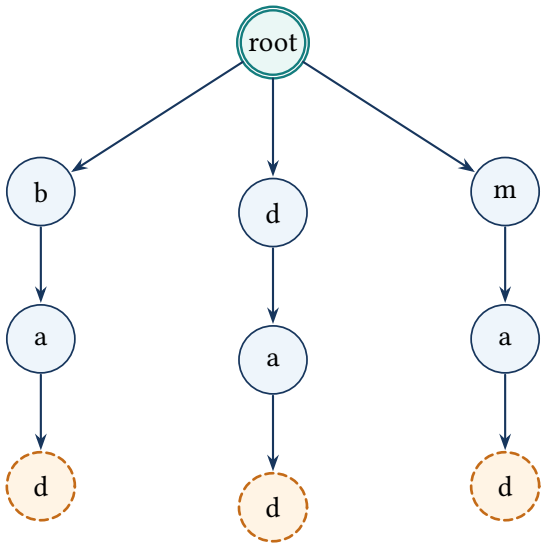
We create the root of the Trie using:

```
TrieNode* root = new TrieNode();
```

The root does not represent any particular character.

It serves as the starting point from which all words are stored.

Trie after adding bad, dad, and mad



orange node = complete word

The root holds no letter. Following the arrows spells each word. Orange nodes show where a complete word ends.

Adding a Word

To add a word, we start from the root node:

```
TrieNode* node = root;
```

We then iterate through every character in the word.

For each character, we convert it into an index between 0 and 25: `index` $= c -' a'$

This lets us access the matching child in the `children` array.

For example: $a \to 0$ $b \to 1$ $c \to 2$

and so on.

Adding Characters to the Trie

For every character c, we check whether its matching child node already exists.

If it does not exist, we create a new Trie node.

Conceptually:

```
if (node->children[c - 'a'] == nullptr) {
    node->children[c - 'a'] = new TrieNode();
}
```

We then move to that child:

```
node = node->children[c - 'a'];
```

This process continues until every character of the word has been processed.

Marking the End of a Word

After processing all characters, we mark the current node as the end of a word:

```
node->isWord = true;
```

This is important because one word can be a prefix of another word.

For example, suppose we insert: `app`

and: `apple`

The node matching to the final character of `app` must be marked as a complete word even though additional characters continue afterward for `apple`.

Search Operation

To search for a word, we again start at the root:

```
TrieNode* node = root;
```

We then process every character in the word.

For each character c, we calculate: `index` $= c -' a'$

and check whether the matching child exists.

If the child does not exist, the word cannot be present in the Trie.

So, we return:

```
false
```

If the child exists, we move to that node and continue searching.

After processing every character, we return:

```
node->isWord
```

This ensures that the complete word exists rather than merely being a prefix of another word.

Why `isWord` Is Needed

A path is not automatically a word

After inserting `apple`, the path $a \to p \to p \to l \to e$ exists. That proves `app` is a prefix, but it does not prove that `app` was inserted as a complete word. The second `p` node must have `isWord = true` before `search("app")` may return true.

The rule is simple: following child pointers proves that a path exists; `isWord` proves that the path ends at a stored word.

```
class WordDictionary {
public:

    struct TrieNode {
        TrieNode* children[26];
        bool isWord;

        TrieNode() {
            isWord = false;

            for (int i = 0; i < 26; i++) {
                children[i] = nullptr;
            }
        }
    };

    TrieNode* root;

    WordDictionary() {
        root = new TrieNode();
    }

    void addWord(string word) {
        TrieNode* node = root;

        for (char c : word) {
            int index = c - 'a';

            if (node->children[index] == nullptr) {
```

```
                node->children[index] = new TrieNode();
            }

            node = node->children[index];
        }

        node->isWord = true;
    }
};
```

Search with a Recursive Helper

The search operation becomes more interesting when the problem allows a special character such as: '.'

The dot can represent any lowercase English character.

So, a normal character can be handled by following one specific child, while a dot requires us to potentially explore all 26 children.

We can handle this using recursion.

The recursive search function receives:

- The current word.
- The current character index.
- The current Trie node.

Conceptually:

```
bool search(string word) {
    return dfs(word, 0, root);
}
```

The recursive helper is:

```
bool dfs(string word, int index, TrieNode* node)
```

Base Case

If we have reached the end of the word: `index == word.size()`

then we return whether the current node represents the end of a complete word:

```
return node->isWord;
```

Normal Character

If the current character is not a dot, we simply follow its matching child.

If that child does not exist, we return: `false`

Otherwise, we recursively continue with the next character.

Dot Character

If the current character is: `'.'`

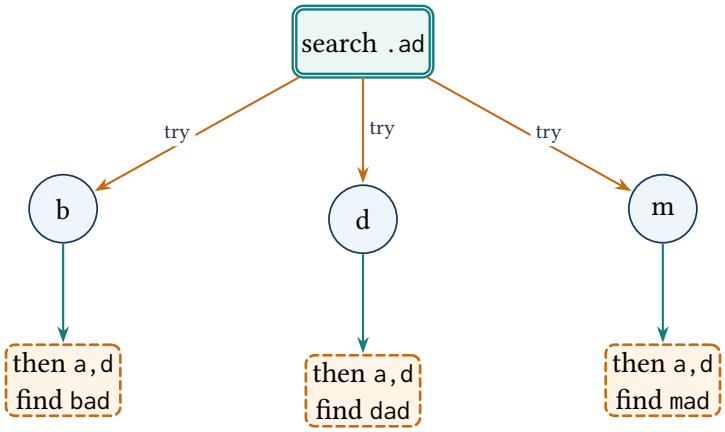
then it can represent any lowercase English letter.

So, we must try every possible child: $0, 1, 2, \ldots, 25$

If any of these recursive searches returns `true`, then the word exists.

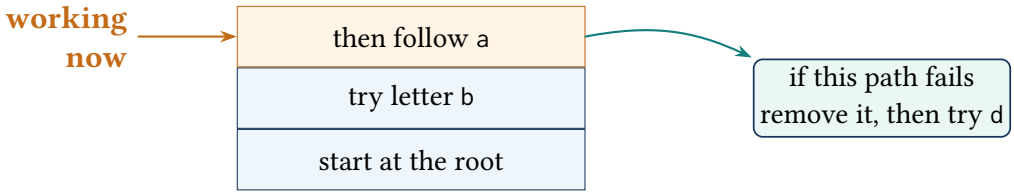
Otherwise, we return `false`.

A dot tries every possible first letter



The dot may try any first letter. The letters a and d then follow one path. The search stops when one complete word is found.

Only one dot choice is tried at a time



Only the path being checked is kept on the stack. If it fails, remove that path and try the next first letter.

Complete Search Code

```
using namespace std;

bool dfs(string word, int index, TrieNode* node) {

    if (index == word.size()) {
        return node->isWord;
    }

    char c = word[index];

    if (c == '.') {

        for (int i = 0; i < 26; i++) {

            if (node->children[i] != nullptr &&
                dfs(word, index + 1, node->children[i])) {
                return true;
            }
        }

        return false;

    } else {

        int childIndex = c - 'a';

        if (node->children[childIndex] == nullptr) {
            return false;
        }

        return dfs(
            word,
            index + 1,
            node->children[childIndex]
        );
    }
}
```

Here is the complete version:

Code 4: Complete C++ data structure: Word Dictionary

```
#include <string>

using namespace std;

class WordDictionary {
public:
    struct TrieNode {
        TrieNode* children[26];
```

```
        bool isWord;
        TrieNode() : isWord(false) {
            for (int i = 0; i < 26; ++i) children[i] = nullptr;
        }
    };
    TrieNode* root;
    WordDictionary() : root(new TrieNode()) {}
    void addWord(const string& word) {
        TrieNode* node = root;
        for (char c : word) {
            const int index = c - 'a';
            if (node->children[index] == nullptr) {
                node->children[index] = new TrieNode();
            }
            node = node->children[index];
        }
        node->isWord = true;
    }
    bool search(const string& word) {
        return dfs(word, 0, root);
    }
private:
    bool dfs(const string& word, int index, TrieNode* node) {
        if (index == static_cast<int>(word.size())) {
            return node->isWord;
        }
        const char c = word[index];
        if (c == '.') {
            for (int i = 0; i < 26; ++i) {
                if (node->children[i] != nullptr &&
                    dfs(word, index + 1, node->children[i])) {
                    return true;
                }
            }
            return false;
        }
        const int childIndex = c - 'a';
        if (node->children[childIndex] == nullptr) return false;
        return dfs(word, index + 1, node->children[childIndex]);
    }
};
```

Time and Space

Let L be the pattern length and d the number of wildcard positions. Adding a word takes $O(L)$ time. A search without wildcards also takes $O(L)$ time. With wildcards, the most useful exact description is $O(V_L)$, where V_L is the number of Trie states actually visited up to depth L. A loose worst-case upper bound is $O(26^d L)$ for a dense Trie. The recursive call stack uses $O(L)$ space, while the Trie itself uses space based on the total number of stored characters.

Key Idea

Here is the whole idea:

1. Use a Trie to store all words.
2. Each node has 26 possible children.
3. Use `isWord` to identify the end of a complete word.
4. For normal characters, follow one specific child.
5. For `'.'`, recursively try all possible children.

The Trie allows words to be stored and searched character by character, while recursion allows the search to branch whenever a wildcard character is seen.

Detailed Visual Dry Run

Add `bad`, `dad`, and `mad`; then search `.ad`

Step	Operation	What happens
1	`addWord("bad")`	Create the path root $\to b \to a \to d$, then mark the final node as a word.
2	`addWord("dad")`	Create root $\to d \to a \to d$. The earlier word stays unchanged.
3	`addWord("mad")`	Create root $\to m \to a \to d$.
4	`search(".ad")`	The dot tries the root's children. The b branch follows a, d and reaches a word end, so the search stops with `true`.

TRIE PATTERNS

Lesson 5: Implement Trie (Prefix Tree)

Problem at a glance

Difficulty	Approach	Main tool
Medium	Prefix tree	Trie

The Problem

Build a prefix-tree class with four operations:

1. `Trie()` – Create an empty Trie.
2. `insert(word)` – Store a word in the Trie.
3. `search(word)` – Return `true` if the complete word exists in the Trie.
4. `startsWith(prefix)` – Return `true` if at least one stored word starts with the given prefix.

The words contain only lowercase English letters: `'a'` to `'z'`

Important Observation

A prefix is not necessarily a complete word.

For example, suppose we insert: `"apple"`

Then: `search("app")` = `false`

because `"app"` was not inserted as a complete word.

However, `startsWith("app")` = `true`

because `"apple"` starts with the prefix `"app"`.

This distinction is important when designing the Trie.

What is a Trie?

A **Trie**, also called a **Prefix Tree**, is a tree-based data structure used to store strings.

Instead of storing an entire word inside a single node, a Trie stores the word character by character.

For example, consider the words: `"app"`

and `"apple"`

Both words reuse the same nodes for the shared prefix `app`. The longer word continues from that node through `l` and `e`.

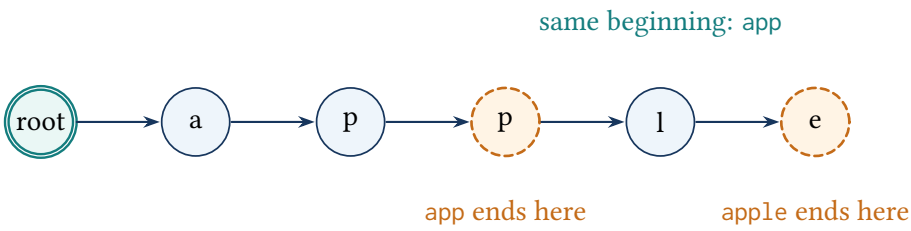
The path: $a \to p \to p$

represents the word `"app"`.

The path: $a \to p \to p \to l \to e$

represents the word `"apple"`.

The words `app` and `apple` share the same beginning



`search("app")` checks that a word ends at the second `p`. `startsWith("app")` only checks that the letters can be followed.

Key idea: the current node represents exactly the consumed prefix

After consuming the first k characters, the current pointer is the Trie node for that length-k prefix. Reaching a node proves the prefix exists; only `isWord=true` proves it was inserted as a complete word.

Why Do We Need an End Marker?

Consider inserting: `"apple"`

The nodes for: $a \to p \to p$

already exist as part of the word.

However, this does not mean that: `"app"`

is a complete word.

So, every Trie node needs a Boolean variable: `isWord`

which tells us whether the current node represents the end of a complete word.

If both `"app"` and `"apple"` were inserted, the second `p` node and the final `e` node each have `isWord = true`. The vector diagram above shows these two endings without mixing tree structure with ASCII art.

Trie Node

Each Trie node contains:

- An array of 26 child pointers.
- A Boolean variable `isWord`.

The 26 children correspond to: $a, b, c, \ldots, z$

So, each node can have at most 26 children.

Trie Node Implementation

```
struct TrieNode {

    TrieNode* children[26];

    bool isWord;

    TrieNode() {

        isWord = false;

        for (int i = 0; i < 26; i++) {
            children[i] = nullptr;
        }
    }
};
```

A newly constructed node has 26 null child pointers and `isWord = false`. Connections and word endings are added only during insertion.

Root Node

We create a root node for the Trie.

```
TrieNode* root = new TrieNode();
```

The root node does not represent any character.

It simply acts as the starting point from which all words are stored.

The root is a sentinel node: it stores no character. Its child pointers lead to the first letters of the stored words, and every later edge represents the next character in a path.

Insert Operation

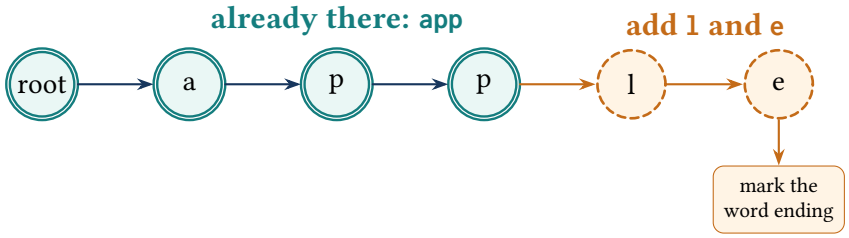
The purpose of the `insert()` operation is to store a word in the Trie.

Suppose we want to insert: `"apple"`

We start from the root and process every character one by one.

The path becomes: $root \to a \to p \to p \to l \to e$

Adding `apple` reuses `app` and adds `l,e`



Follow letters that are already present. Add a new circle only for a missing letter, then mark where the complete word ends.

Character to Index Conversion

Since every node contains an array of 26 children, we convert a character into an index using: $\texttt{index} = c - \texttt{'a'}$

For example: $\texttt{'a'} - \texttt{'a'} = 0$ $\texttt{'b'} - \texttt{'a'} = 1$ $\texttt{'c'} - \texttt{'a'} = 2$

and so on.

So: $\texttt{'z'} - \texttt{'a'} = 25$

Creating Missing Nodes

For every character, we check whether the matching child already exists.

If it does not exist, we create a new node:

```
if (node->children[index] == nullptr) {
    node->children[index] = new TrieNode();
}
```

Then we move to that child:

```
node = node->children[index];
```

This process continues until every character of the word has been processed.

Marking the End of the Word

After processing the final character, we mark the current node as the end of a complete word:

```
node->isWord = true;
```

This is what allows the Trie to distinguish between a complete word and merely a prefix.

Insert Implementation

```
void insert(string word) {

    TrieNode* node = root;

    for (char c : word) {

        int index = c - 'a';

        if (node->children[index] == nullptr) {
            node->children[index] = new TrieNode();
        }

        node = node->children[index];
    }

    node->isWord = true;
}
```

Search Operation

The purpose of `search()` is to determine whether the **complete word** exists in the Trie.

Suppose we search for: `"apple"`

We start from the root and follow: $a \to p \to p \to l \to e$

If at any point the required child does not exist, the word does not exist.

So, we immediately return: `false`

If we successfully process every character, we still need to check: `isWord`

This is because the searched string may only be a prefix of a stored word.

Search Implementation

```
bool search(string word) {

    TrieNode* node = root;

    for (char c : word) {

        int index = c - 'a';

        if (node->children[index] == nullptr) {
            return false;
        }

        node = node->children[index];
    }

```

```
    return node->isWord;
}
```

`startsWith()` Operation

The third operation is:

```
startsWith(prefix)
```

Its purpose is to determine whether at least one word stored in the Trie begins with the given prefix.

For example, if the Trie contains: `"apple"`

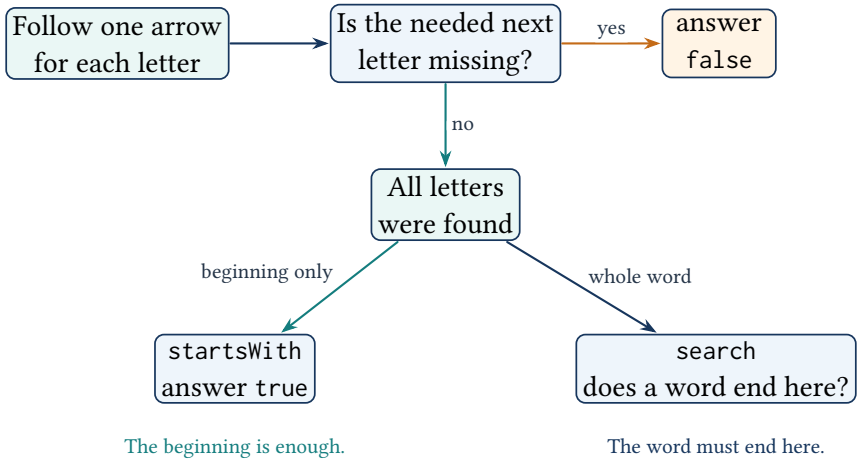
then: `startsWith("app") = true`

because `"apple"` begins with `"app"`.

However, unlike `search()`, we do **not** need to check `isWord` at the end.

We only need to determine whether the complete prefix path exists.

`search` and `startsWith` stop differently



`startsWith()` Implementation

```
bool startsWith(string prefix) {

    TrieNode* node = root;

    for (char c : prefix) {

        int index = c - 'a';

        if (node->children[index] == nullptr) {
            return false;
        }

```

```
        node = node->children[index];
    }

    return true;
}
```

Code 5: Complete C++ data structure: Trie

```
#include <string>

using namespace std;

class Trie {
public:
    struct TrieNode {
        TrieNode* children[26];
        bool isWord;
        TrieNode() : isWord(false) {
            for (int i = 0; i < 26; ++i) children[i] = nullptr;
        }
    };
    TrieNode* root;
    Trie() : root(new TrieNode()) {}
    void insert(const string& word) {
        TrieNode* node = root;
        for (char c : word) {
            const int index = c - 'a';
            if (node->children[index] == nullptr) {
                node->children[index] = new TrieNode();
            }
            node = node->children[index];
        }
        node->isWord = true;
    }
    bool search(const string& word) {
        TrieNode* node = root;
        for (char c : word) {
            const int index = c - 'a';
            if (node->children[index] == nullptr) return false;
            node = node->children[index];
        }
        return node->isWord;
    }
    bool startsWith(const string& prefix) {
        TrieNode* node = root;
        for (char c : prefix) {
            const int index = c - 'a';
            if (node->children[index] == nullptr) return false;
            node = node->children[index];
```

```
        }
        return true;
    }
};
```

Example Walkthrough

Suppose we perform:

```
Trie trie;

trie.insert("apple");
```

The Trie contains the path: $a \to p \to p \to l \to e$

and the final node has: `isWord` $=$ `true`

Now consider:

```
trie.search("apple");
```

The complete path exists and the final node has `isWord = true`, so: `true`

Now:

```
trie.search("app");
```

The path exists, but if `"app"` was not inserted separately, the node has: `isWord` $=$ `false`

So: `false`

However:

```
trie.startsWith("app");
```

returns: `true`

because the prefix path exists.

`search()` vs `startsWith()`

The main difference is:

Operation	What it checks	Uses `isWord`?
`search()`	Complete word exists	Yes
`startsWith()`	Prefix path exists	No

This distinction is one of the most important concepts in the problem.

Time and Space

Let L be the length of the word or prefix.

For `insert()`, we process every character exactly once: $\boxed{O(L)}$

For `search()`, we also process every character at most once: $\boxed{O(L)}$

For `startsWith()`, the same applies: $\boxed{O(L)}$

So:

$$\boxed{\begin{aligned} \text{Insert} &= O(L) \\ \text{Search} &= O(L) \\ \text{StartsWith} &= O(L) \end{aligned}}$$

The space required by the Trie depends on the total number of characters inserted.

If the total number of inserted characters is N, the Trie can use: $\boxed{O(N)}$

nodes in the worst case.

Key Takeaways

- A Trie is also called a Prefix Tree.
- A Trie stores words character by character.
- Each node contains 26 child pointers for lowercase English letters.
- `isWord` identifies whether a node represents the end of a complete word.
- `search()` checks both the path and `isWord`.
- `startsWith()` only checks whether the prefix path exists.
- Character-to-index conversion is performed using: $c - \texttt{'a'}$
- Insert, search, and prefix operations take: $O(L)$ time, where L is the length of the input string.

Detailed Visual Dry Run

Insert `apple`, then compare `search` with `startsWith`

Step	Operation	What happens
1	`insert("apple")`	Walk through a, p, p, l, e, creating missing nodes. Mark only the e node as a complete word.
2	`search("apple")`	The whole path exists and the final node is marked, so return `true`.
3	`search("app")`	The path exists, but the second p node is not a word end, so return `false`.
4	`startsWith("app")`	The same path exists. A prefix does not need a word-end mark, so return `true`.

GRAPH PATTERNS

Lesson 6: Alien Dictionary

Problem at a glance

Difficulty	Approach	Main tool
Hard	Topological sorting	Directed graph and queue

The Problem

A list of words has already been sorted using an unknown alphabet order. Recover one character order consistent with that list. Return an empty string if the comparisons imply a contradiction; when several orders are possible, any valid one is acceptable.

Input and Output

The input is: `vector<string> words`

where the words are already sorted according to the unknown alien language.

The output is: `string`

containing the unique characters in their valid alien-language ordering.

If no valid ordering exists: `""`

If multiple valid orderings exist, any valid ordering is acceptable.

Understanding Dictionary Order

Dictionary order compares two strings from left to right. The first position where they differ decides which word comes first. For example, `ape` comes before `apple`: the shared prefix is `ap`, then `e` comes before `p`.

Only the first difference matters

```
a   p   e
=   =   <
a   p   p   l e
```

So, ape $<$ apple.

The same concept is used in the alien dictionary, except that the ordering of the characters is unknown.

Main Idea

The ordering of adjacent words reveals ordering between characters. Compare abc and axc. Their first characters match. The first difference is b versus x, so the alien alphabet must place b before x.

abc $<$ axc $\implies$ $b < x$ $\implies$ $b \to x$

This gives us a directed relationship: $b \longrightarrow x$

So, the problem can be converted into a **directed graph**.

Graph Representation

Each unique character is represented as a node in the graph.

If we discover that: $a < b$

we create a directed edge: $a \longrightarrow b$

The meaning is: a must appear before b

After constructing all such relationships, we need to find an ordering of the graph in which every directed edge goes from an earlier character to a later character.

This is exactly the purpose of: Topological Sorting

Kahn's Algorithm

We use **Kahn's algorithm** to perform topological sorting.

The algorithm is based on the concept of: Indegree

The indegree of a node is the number of edges coming into that node.

For example, if: $a \to c$

and: $b \to c$

then: $\text{indegree}(c) = 2$

because two characters must appear before c.

Characters with: $\text{indegree} = 0$

can be placed at the beginning of the ordering.

Key idea: the queue contains exactly the available characters

Every queued character has zero remaining indegree and has not yet been placed in the result. Removing one character deletes its outgoing rules; a neighbor is enqueued exactly when its final unmet rule disappears.

Algorithm Steps

The solution can be divided into four major steps.

Step 1: Start the Graph

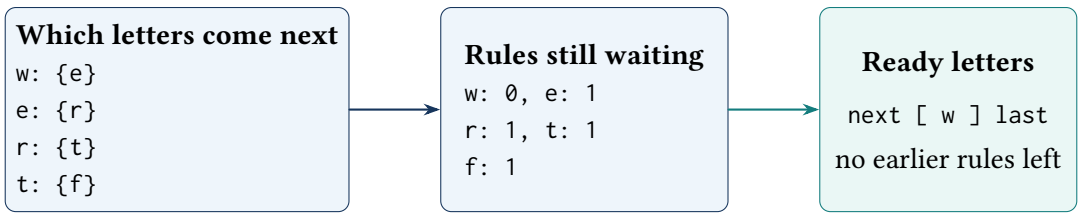
Create:

- An adjacency list for the graph.
- An indegree array/map for all characters.

Since the alien language uses lowercase English letters, there can be at most 26 possible characters.

We also need to make sure that every character appearing in the input is represented in the graph, even if its indegree is zero.

What we save while building the letter order



The first box shows which letter must follow another. The second counts how many earlier-letter rules remain. The last box holds letters ready to use.

Step 2: Build the Directed Graph

We compare every pair of adjacent words. For $i = 0, 1, \ldots, n-2$, let `word1=words[i]`

and: `word2=words[i+1]`

We compare their characters from left to right.

Let j represent the character position.

As long as: `word1[j]` $=$ `word2[j]`

there is no ordering information, so we continue.

When we find the first different pair: `word1[j]` $\neq$ `word2[j]`

we have discovered an ordering relationship.

If: `word1[j]` $= a$

and: `word2[j]` $= b$

then: $\boxed{a \to b}$

and: $\text{indegree}(b)++$

After finding the first different character, we stop comparing the two words because later characters provide no additional information about their dictionary ordering.

Important Edge Case: Invalid Prefix

There is an important invalid case.

Consider: `["abc","ab"]`

The first word is longer than the second word, but the second word is a complete prefix of the first.

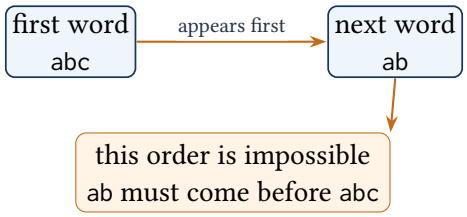
This ordering is impossible in dictionary order.

So, if: $|\texttt{word1}| > |\texttt{word2}|$

and all characters of `word2` match the beginning of `word1`, then the input is invalid.

We return: `""`

A longer word cannot come before its own beginning



Step 3: Start the Queue

After constructing the graph, we look for all characters whose indegree is zero.

These characters have no dependencies.

So, they can appear at the beginning of the ordering.

We add them to a queue:

```
queue<char> q;
```

For every character c:

```
if (indegree[c] == 0) {
    q.push(c);
}
```

Step 4: Perform Topological Sort

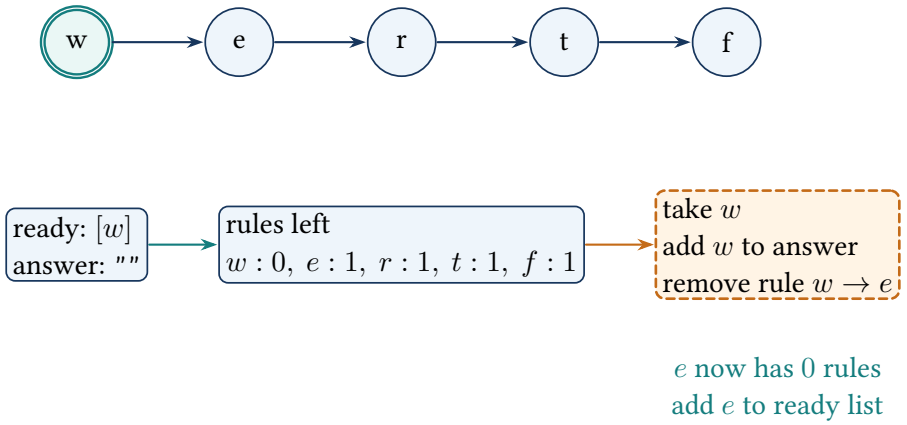
While the queue is not empty:

1. Remove one character from the queue.

2. Add it to the result.
3. Visit all of its neighboring characters.
4. Decrease their indegree by one.
5. If a neighbor's indegree becomes zero, add it to the queue.

Conceptually, take a node, append it to the result, remove its outgoing dependencies, and enqueue any nodes that have become available.

Take a ready letter, then unlock the next one



A letter becomes ready only after every letter that must come before it has been used. In a cycle, some letters never become ready.

Example

Consider `["wrt", "wrf", "er", "ett", "rftt"]`. Compare only adjacent words and stop at each pair's first different character.

Building the character rules

Adjacent pair	First difference	Rule
`wrt`, `wrf`	`t` versus `f`	$t \to f$
`wrf`, `er`	`w` versus `e`	$w \to e$
`er`, `ett`	`r` versus `t`	$r \to t$
`ett`, `rftt`	`e` versus `r`	$e \to r$

These edges form the chain $w \to e \to r \to t \to f$, so one valid answer is `"wertf"`.

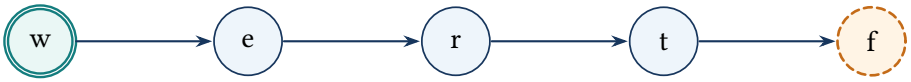
Graph Structure

The relationships can be visualized as: $w \longrightarrow e \longrightarrow r \longrightarrow t \longrightarrow f$

The topological ordering of this graph gives: $\boxed{w, e, r, t, f}$

which corresponds to: `"wertf"`

The letter rules form one clear order



Starting with `w`, the only letter with no earlier rule, gives $w \to e \to r \to t \to f$.

Pseudocode

```
alienOrder(words):
    create graph
    create indegree
    for every word:
        add all characters to graph
        start their indegree
    for every pair of adjacent words:
        compare characters from left to right
        if all characters match and
           first word is longer:
            return ""
        when first different characters are found:
            create directed edge
            increase indegree
            stop comparing this pair
    create queue
    for every character:
        if indegree == 0:
            add character to queue
    result = ""
    while queue is not empty:
        character = queue.front()
        queue.pop()
        add character to result
        for every neighbor:
            decrease indegree
            if indegree becomes 0:
                add neighbor to queue
    if result contains every unique character:
        return result
    return ""
```

Code 6: Complete C++ solution: Alien Dictionary

```cpp
#include <algorithm>
#include <queue>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>

using namespace std;

class Solution {
public:
    string alienOrder(vector<string>& words) {
        unordered_map<char, unordered_set<char>> graph;
        unordered_map<char, int> indegree;
        for (const string& word : words) {
            for (char c : word) {
                graph[c];
                indegree[c] = 0;
            }
        }
        for (int i = 0; i + 1 < static_cast<int>(words.size()); ++i) {
            const string& word1 = words[i];
            const string& word2 = words[i + 1];
            const int len = static_cast<int>(min(
                word1.size(), word2.size()));
            bool foundDifference = false;
            for (int j = 0; j < len; j++) {
                if (word1[j] != word2[j]) {
                    const char from = word1[j];
                    const char to = word2[j];
                    if (graph[from].find(to) == graph[from].end()) {
                        graph[from].insert(to);
                        indegree[to]++;
                    }
                    foundDifference = true;
                    break;
                }
            }
            if (!foundDifference && word1.size() > word2.size()) {
                return "";
            }
        }
        queue<char> q;
        for (const auto& entry : indegree) {
            if (entry.second == 0) q.push(entry.first);
        }
        string result;
        while (!q.empty()) {
```

```
            const char current = q.front();
            q.pop();
            result += current;
            for (char neighbor : graph[current]) {
                indegree[neighbor]--;
                if (indegree[neighbor] == 0) q.push(neighbor);
            }
        }
        return result.size() == indegree.size() ? result : "";
    }
};
```

Why Duplicate Edges Must Be Avoided

Suppose we discover the same relationship multiple times: $a \to b$

If we increment the indegree every time without checking whether the edge already exists, we may incorrectly calculate: $\text{indegree}(b)$

So, the implementation uses an `unordered_set` for the neighbors.

Before adding: $a \to b$

we check whether the edge already exists.

Only a new edge increases the indegree.

Detecting a Cycle

A valid alien alphabet ordering must form a directed acyclic graph.

Suppose the graph contains: $a \to b$

and: $b \to a$

Then there is a cycle: $a \to b \to a$

No valid ordering exists.

Kahn's algorithm detects this naturally.

If a cycle exists, some nodes will always have: $\text{indegree} > 0$

and so they will never enter the queue.

As a result: $|\texttt{result}| < |\texttt{indegree}|$

So, we return: `""`

Complete Process

The entire solution can be summarized as:

$$\boxed{\begin{array}{c}\text{Words} \rightarrow \text{Compare Adjacent Words} \rightarrow \text{Build Directed Graph} \\ \Downarrow \\ \text{Calculate Indegrees} \rightarrow \text{Kahn's Algorithm} \rightarrow \text{Topological Order}\end{array}}$$

The important transformation is: $\boxed{\text{Unknown Alphabet Ordering} \rightarrow \text{Graph Dependencies}}$

Once the character dependencies are represented as a graph, the problem becomes a standard topological sorting problem.

Time and Space

Let: C

be the total number of characters across all words.

That is: $C = \sum_{i=0}^{n-1} |\texttt{words[i]}|$

Building the graph scans the words, and the topological sort processes every stored vertex and edge once. So, the full time complexity is: $\boxed{O(C + U + E)}$

Because the distinct edges arise from adjacent-word comparisons, this is commonly simplified to $O(C)$ for this problem.

The graph contains the unique characters and the precedence edges discovered between them.

Let: $U =$ number of unique characters

and let E be the number of distinct directed edges. Then the additional space is: $\boxed{O(U + E)}$

Since the language uses only the lowercase English alphabet: $U \leq 26$

So, in this particular problem, U is bounded by a constant, but writing $O(U + E)$ describes the graph storage exactly and generalizes to a larger alphabet.

Key Takeaways

- The words are already sorted according to an unknown alphabet.
- Compare adjacent words to discover character ordering relationships.
- The first different characters determine an ordering: $a \rightarrow b$ means a comes before b.
- Represent the relationships using a directed graph.
- Use indegree to determine which characters can be placed next.
- Kahn's algorithm performs the required topological sort.
- An invalid prefix such as: `"abc"`, `"ab"` makes the ordering impossible.

- If the result does not contain every unique character, a cycle exists and the answer is invalid.
- Time complexity: $O(C + U + E) = O(C)$
- Space complexity: $O(U + E)$ where U is the number of unique characters and E is the number of distinct precedence edges.

TREE TRAVERSAL AND RECURSION

Lesson 7: Binary Tree Level Order Traversal

Problem at a glance

Difficulty	Approach	Main tool
Medium	Breadth-first search	Queue

The Problem

Given the root of a binary tree, return its values one level at a time from top to bottom. Nodes on the same depth belong to the same output row.

Example

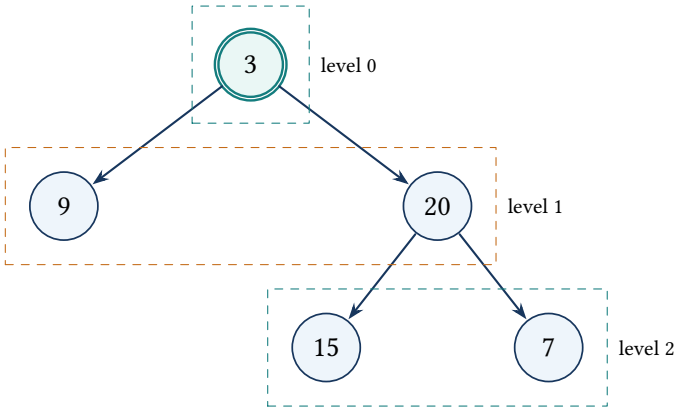
For the tree with root 3, children 9 and 20, and children 15 and 7 under 20, return `[[3],[9,20],[15,7]]`. An empty tree returns an empty vector.

From the Direct Idea to BFS

A depth-first traversal can record each node's depth, but it needs extra logic to place values into the correct row. The cleaner approach processes the tree in the same order as the required output: breadth first. A queue stores the next nodes to visit.

At the beginning of each outer-loop iteration, the queue contains exactly one complete level. Save the current queue size before adding any children. That size tells us how many nodes belong to the current row.

The queue handles one tree level at a time



Queue after each level: `[3]` → `[9,20]` → `[15,7]`.

Algorithm and Key Idea

Start by pushing the root. While the queue is not empty, save `levelSize`, remove exactly that many nodes, append their values to one row, and push each non-null child. The key rule is that every queued node is unprocessed and the first `levelSize` nodes share the same depth.

Key idea: one saved queue prefix equals one tree level

Immediately before an outer-loop round, the queue contains all unprocessed nodes in breadth-first order. The saved size identifies exactly the current depth; children appended during the round belong to the next depth and are so not consumed until the following round.

Pseudocode

```
levelOrder(root):
    if root is null: return []
    queue = [root]
    result = []
    while queue is not empty:
        levelSize = queue.size()
        level = []
        repeat levelSize times:
            node = pop front
            append node.value to level
            push non-null children
        append level to result
    return result
```

Code 7: Complete C++ solution: Binary Tree Level Order Traversal

```cpp
#include <queue>
#include <utility>
#include <vector>

using namespace std;

class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        if (root == nullptr) return {};

        vector<vector<int>> result;
        queue<TreeNode*> pending;
        pending.push(root);

        while (!pending.empty()) {
            const int levelSize = static_cast<int>(pending.size());
            vector<int> level;
            level.reserve(levelSize);

            for (int i = 0; i < levelSize; ++i) {
                TreeNode* node = pending.front();
                pending.pop();
                level.push_back(node->val);
                if (node->left != nullptr) pending.push(node->left);
                if (node->right != nullptr) pending.push(node->right);
            }
            result.push_back(move(level));
        }
        return result;
    }
};
```

Time, Space, and Common Mistakes

Every node enters and leaves the queue once, so time is $O(n)$. The queue uses $O(w)$ space, where w is the maximum tree width; the returned rows use $O(n)$ output space. Save the level size before pushing children, and handle a null root before accessing it.

How to explain it in an interview

I use BFS because the output is grouped by depth. The queue contains the next level. I snapshot its size, process exactly those nodes, and enqueue their children for the following level. Each node is processed once: $O(n)$ time and $O(w)$ extra queue space.

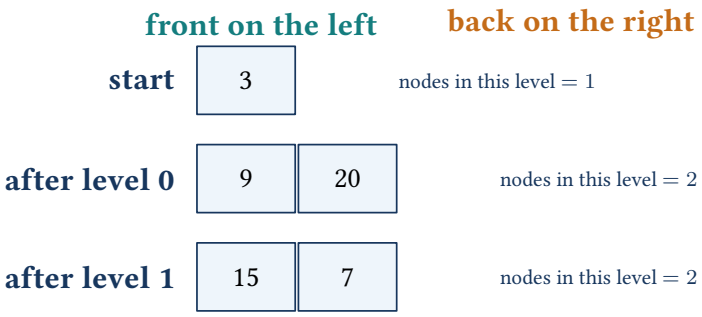
Detailed Visual Dry Run

For the worked example, the queue changes by complete levels. Children are added only after the current node is removed.

Queue trace for `[3,9,20,null,null,15,7]`

Round	Saved size	Nodes processed and children appended	Output
1	1	Remove 3; append 9 and 20	`[[3]]`
2	2	Remove 9; remove 20 and append 15 and 7	`[[3],[9,20]]`
3	2	Remove 15 and 7; neither has children	`[[3],[9,20],[15,7]]`

What the queue holds after each level



The saved size is essential. If the loop instead continued until the queue became empty, the children would be consumed in the same row as their parents and the level grouping would be lost.

Code Walkthrough

1. Reject a null root and return an empty result.
2. Push the root into `pending`. The queue now represents the first unprocessed level.
3. Copy `pending.size()` into `levelSize`. This value does not change when children are pushed later in the round.
4. Pop exactly `levelSize` nodes, record their values, and enqueue their non-null children from left to right.
5. Move the completed row into the result and repeat for the next depth.

Although the code contains a loop inside another loop, it is still linear. The inner loops across all rounds process each node exactly once; they do not restart a full scan for every level.

Edge Cases, Follow-ups, and Practice

Test an empty tree, one node, a completely skewed tree, and a wide final level. A common mistake is to enqueue null children and then dereference them. Useful follow-ups include zigzag level order, returning only the rightmost node at each depth, and computing the average value of every level.

Practice checkpoint

Without looking at the code, explain what the queue contains immediately before each outer-loop iteration. Then modify the algorithm so alternating rows are reversed while the traversal remains breadth first.

TREE TRAVERSAL AND RECURSION

Lesson 8: Binary Tree Maximum Path Sum

Problem at a glance

Difficulty	Approach	Main tool
Hard	Postorder DFS	Binary tree

The Problem

A path may start and end at any nodes, but it cannot reuse a node. Return the largest sum of any non-empty path. The path does not have to pass through the root.

Example

For `[-10,9,20,null,null,15,7]`, the best path is $15 \to 20 \to 7$, so the answer is 42.

Building the Recursive State

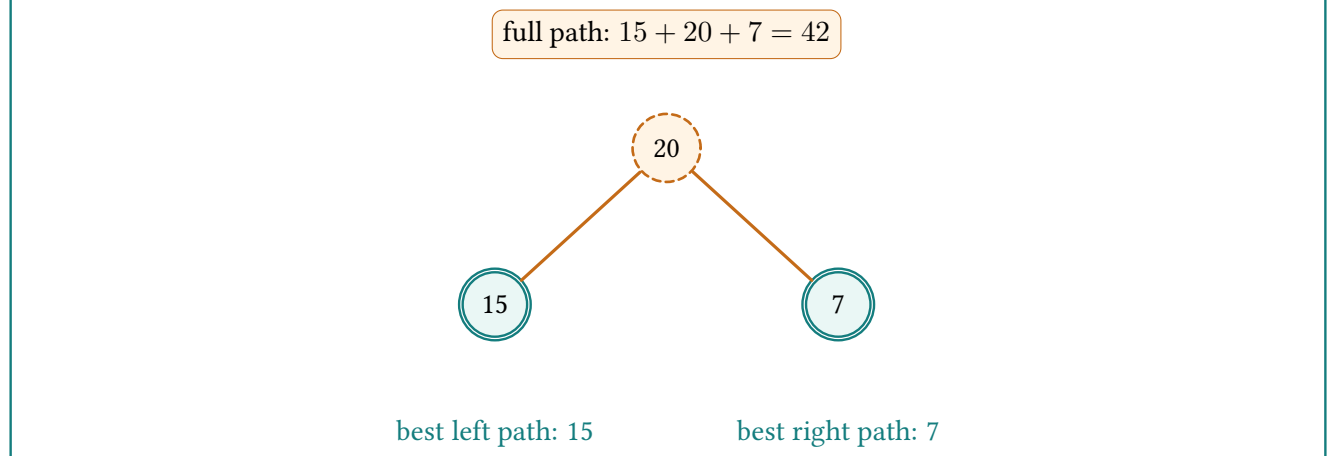
A brute-force search could start a path at every node and explore many of the same subtrees repeatedly. Postorder DFS removes that repetition. A parent can continue through at most one child, so each recursive call returns the best one-sided gain beginning at its node.

For a node with value x: $L = \max(0, \text{left gain}), \qquad R = \max(0, \text{right gain})$. The complete path turning at this node is $L + x + R$. The value returned to the parent is $x + \max(L, R)$.

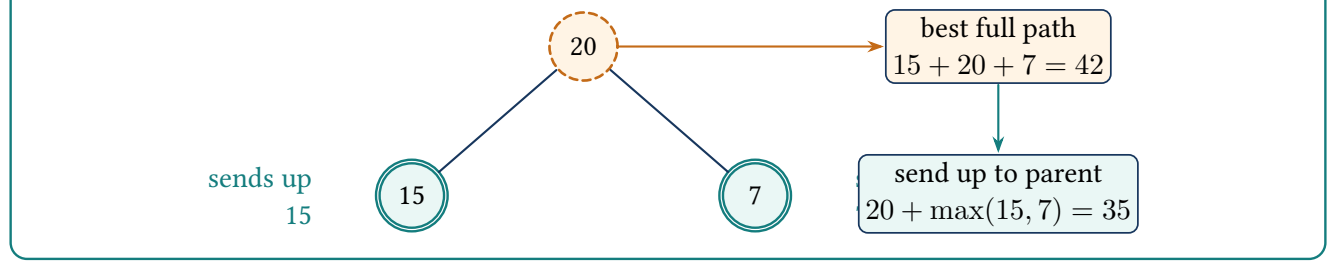
Key idea: a returned gain is always a single extendable chain

After `gain(node)` finishes, its return value is the maximum sum of a path that starts at `node` and travels downward through at most one child. The global answer separately stores the best complete path seen so far, which may use both children at exactly one turning node.

A path can join both children at one middle node



A node sends only one branch up to its parent



Pseudocode

```
dfs(node):
    if node is null: return 0
    leftGain = max(0, dfs(node.left))
    rightGain = max(0, dfs(node.right))
    answer = max(answer, leftGain + node.value + rightGain)
    return node.value + max(leftGain, rightGain)
```

Code 8: Complete C++ solution: Binary Tree Maximum Path Sum

```cpp
#include <algorithm>
#include <limits>

using namespace std;

class Solution {
    int bestPath = numeric_limits<int>::min();

    int gain(TreeNode* node) {
        if (node == nullptr) return 0;
        const int left = max(0, gain(node->left));
        const int right = max(0, gain(node->right));
        bestPath = max(bestPath, node->val + left + right);
```

```
        return node->val + max(left, right);
    }

public:
    int maxPathSum(TreeNode* root) {
        bestPath = numeric_limits<int>::min();
        gain(root);
        return bestPath;
    }
};
```

Why It Works and Complexity

Postorder traversal makes both child gains available before processing the parent. Every possible path has a highest turning node; the algorithm tests that path when it processes that node. Negative gains are discarded because they can only reduce the sum. Time is $O(n)$, and recursion uses $O(h)$ stack space for tree height h.

Common mistakes

Do not start the answer to zero because all node values may be negative. Do not return the two-child path to the parent; that would branch and stop being a valid path. Return only the better one-sided gain.

All-negative tree

For a tree such as `[-3,-2,-5]`, every child gain is clipped to zero, but each node still tests its own value as a non-empty path. Initializing the global answer to negative infinity returns -2, while initializing it to zero would incorrectly allow an empty path.

From Brute Force to One Postorder Traversal

A direct method could treat every node as a possible path start and explore every possible continuation. Long chains and overlapping subtrees would then be traversed repeatedly, leading to $O(n^2)$ work on a skewed tree. The optimized method asks each subtree one reusable question: "What is the best one-sided gain that can be extended through your parent?"

This returned value and the global answer are deliberately different:

- the value returned upward may use at most one child;
- the complete choice recorded locally may join both child gains.

Detailed Visual Dry Run

Postorder values for `[-10,9,20,null,null,15,7]`

Node	Left gain	Right gain	Complete choice	Gain returned
9	0	0	9	9
15	0	0	15	15
7	0	0	7	7
20	15	7	42	35
-10	9	35	34	25

The best complete path is recorded at node 20. The value 42 is not returned to node -10, because returning both branches would create an invalid fork.

Code Walkthrough

The helper first recurses left and right, so it is a postorder traversal. `max(0, gain)` discards a negative branch. The expression `node->val + left + right` tests the path whose highest turning point is the current node. The helper then returns the current value plus only the larger child gain. Resetting `bestPath` inside the public method makes the same `Solution` object safe for another call.

Edge Cases and Interview Follow-ups

The tree may contain one negative node, all negative nodes, or a best path that never touches the root. Recursion uses $O(h)$ call-stack space and can reach $O(n)$ for a skewed tree. A useful follow-up is to return the actual nodes in the maximum path; that requires storing or reconstructing the chosen child direction as well as the sum.

How to explain it simply

Each recursive call returns the best downward path the parent can extend. At each node I also test the complete path that joins the best non-negative left and right gains. Every path has exactly one highest turning node, so one of these local choices represents the global optimum.

GRAPH TRAVERSAL

Lesson 9: Clone Graph

Problem at a glance

Difficulty	Approach	Main tool
Medium	DFS with memoization	Graph and hash map

The Problem

Given one node in a connected undirected graph, return a deep copy. Every original node must correspond to one new node, and neighbor relationships must be keepd without sharing original pointers.

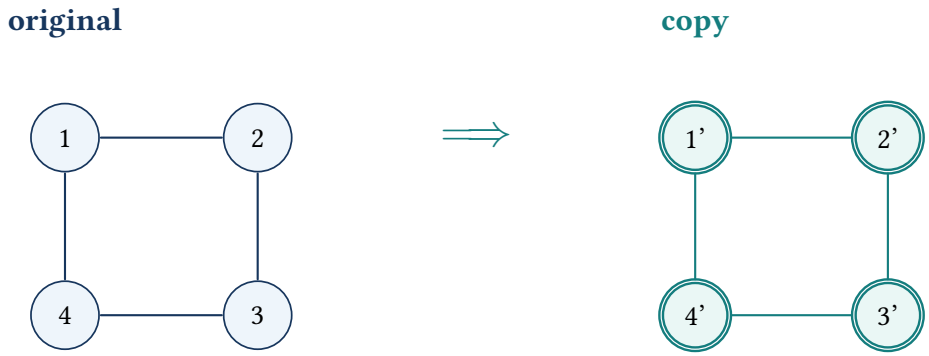
Why a Hash Map Is Needed

A plain recursive copy loops forever on cycles and may clone the same node more than once. Store original pointer → clone pointer before recursing into neighbors. Seeing an existing key means the clone is already available.

Key idea: every discovered original has exactly one registered clone

Before DFS follows an outgoing edge, the current original node already has a map entry. As a result, a back edge, self-loop, or shared neighbor always returns the same clone pointer instead of allocating another object.

Copy every node and every connection



Pseudocode

```
clone(node):
    if node is null: return null
    if node is in cloneMap: return cloneMap[node]
    copy = new Node(node.value)
    cloneMap[node] = copy
    for neighbor in node.neighbors:
        copy.neighbors.push_back(clone(neighbor))
    return copy
```

Code 9: Complete C++ solution: Clone Graph

```cpp
#include <unordered_map>

using namespace std;

class Solution {
    unordered_map<Node*, Node*> clones;

    Node* clone(Node* node) {
        if (node == nullptr) return nullptr;
        const auto found = clones.find(node);
        if (found != clones.end()) return found->second;

        Node* copy = new Node(node->val);
        clones[node] = copy;
        for (Node* neighbor : node->neighbors) {
            copy->neighbors.push_back(clone(neighbor));
        }
        return copy;
    }

public:
    Node* cloneGraph(Node* node) {
        clones.clear();
        return clone(node);
    }
};
```

Why It Works, Time, and Space

The map creates exactly one clone for every reachable original node. Each edge is copied when its adjacency entry is visited. Time is $O(V + E)$, map and recursion space are $O(V)$. Insert the clone into the map before visiting neighbors; doing it afterward fails on cycles and self-loops.

How to explain it in an interview

I run DFS and memoize original-to-copy pointers. If a node was cloned already, I reuse it. Otherwise I create and register the clone first, then recursively copy every neighbor. This handles cycles and keeps shared neighbors.

Why a Plain Recursive Copy Fails

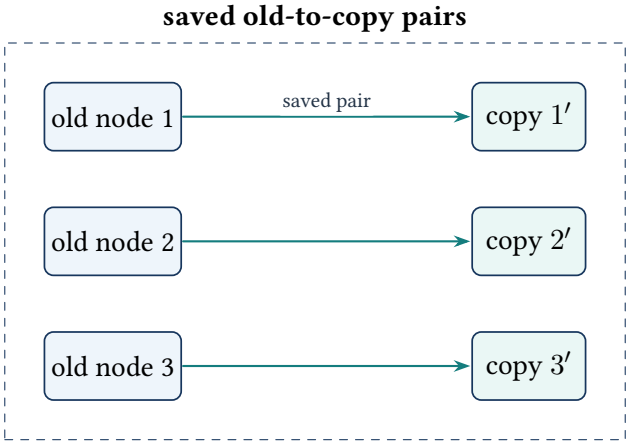
On a tree, recursively copying children eventually reaches null. A graph may return to a node already on the active recursion path. Without the map, the copy of node 1 asks for node 2, node 2 asks for node 1 again, and recursion never ends. The map is both a cycle guard and the record that keeps one-to-one node identity.

Detailed Map and DFS Trace

Cloning the four-node cycle

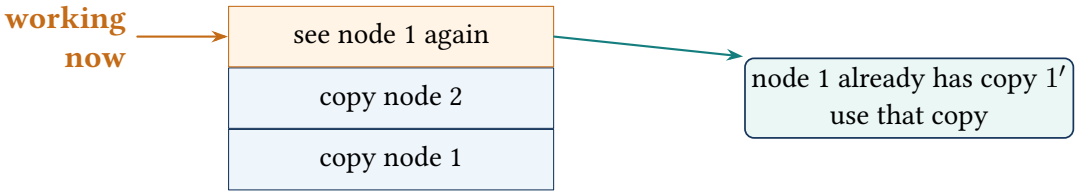
Visit	Map before the recursive calls	Action
1	empty	create $1'$, store $1 \to 1'$, then visit its neighbors
2	$1 \to 1'$	create $2'$, store it, then visit neighbor 1
1 again	$1 \to 1', 2 \to 2'$	reuse $1'$; do not recurse again
3 and 4	earlier clones remain stored	create each once and copy every edge

Each original node points to its one copy



If an old node appears again, the saved pair returns the same copy. This keeps all shared connections and cycles correct.

If a node was copied already, stop going deeper



The repeated node creates nothing new. Use its saved copy and finish this function call.

Code Walkthrough

1. A null input represents an empty graph and returns null.
2. Search `clones` before allocating. An existing entry means the node and all references to its identity already have a copy.
3. Allocate a new node and immediately place it in the map.
4. Recursively clone every neighbor and append the returned pointer to the copy's adjacency vector in the same order.

The immediate insertion in step 3 is the key rule-preserving step. It makes the partial clone visible before an edge can lead back to it.

Edge Cases, Alternative, and Practice

Test a single isolated node, a self-loop, two nodes connected to each other, and several nodes sharing the same neighbor. Verify that cloned pointers are different from all original pointers. An iterative BFS alternative uses a queue plus the same original-to-clone map and has the same $O(V + E)$ cost.

Practice checkpoint

Draw a graph containing a self-loop and explain exactly why registering the copy after cloning its neighbors would recurse forever.

DYNAMIC PROGRAMMING

Lesson 10: Coin Change

Problem at a glance

Difficulty	Approach	Main tool
Medium	Bottom-up dynamic programming	One-dimensional DP

The Problem

Given coin denominations and a target amount, return the fewest coins needed to form that amount. Coins may be reused. Return -1 when the amount is unreachable.

Example

With `coins=[1,2,5]` and `amount=11`, the best construction is $5 + 5 + 1$, so the answer is 3.

How We Get to the Solution

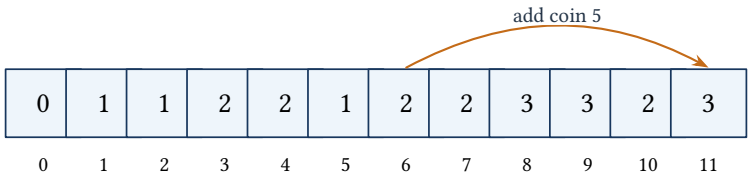
Trying every sequence of coins creates an exponential recursion tree. The same remaining amounts occur repeatedly. Let $dp[x]$ be the minimum number of coins needed to form x. Start $dp[0] = 0$ and every other entry to an unreachable sentinel `amount` $+ 1$.

For each amount x and each coin $c \leq x$: $dp[x] = \min(dp[x], 1 + dp[x - c])$.

Key idea: every smaller amount is already best

When the outer loop reaches amount x, each referenced state $dp[x - c]$ has already been finalized. Trying every possible final coin tests every construction of x, and the minimum choice becomes best.

To make 11, add coin 5 to the best answer for 6



Pseudocode

```
dp = array(amount + 1, amount + 1)
dp[0] = 0
for current from 1 through amount:
    for coin in coins:
        if coin <= current:
            dp[current] = min(dp[current], 1 + dp[current - coin])
return -1 if dp[amount] is unreachable, otherwise dp[amount]
```

Code 10: Complete C++ solution: Coin Change

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, amount + 1);
        dp[0] = 0;

        for (int current = 1; current <= amount; ++current) {
            for (const int coin : coins) {
                if (coin <= current) {
                    dp[current] = min(dp[current], 1 + dp[current - coin]);
                }
            }
        }
        return dp[amount] == amount + 1 ? -1 : dp[amount];
    }
};
```

Time and Space

For $A = \texttt{amount}$ and m coin types, time is $O(Am)$ and space is $O(A)$. The key rule is that before computing $dp[x]$, every smaller amount is already best. Avoid using zero as the default for unreachable amounts; it would look like a free solution.

Pattern clue

When a target can be built from reusable choices and the question asks for a minimum, maximum, or count, define the answer for every smaller target and extend those solved states.

Brute Force and Repeated Work

A recursive brute-force solution tries every coin, subtracts it from the remaining amount, and repeats. For amount 11, branches such as $11 \to 10 \to 8$ and $11 \to 9 \to 8$ both solve the same remaining amount 8. Without memoization, the recursion tree grows exponentially. The DP table stores the answer to each remaining amount once.

Detailed Table Walkthrough

Building dp for coins `[1,2,5]`

Amount	Best count	Reason
0	0	base case: no coins are needed
1	1	$1 + dp[0]$ using coin 1
2	1	coin 2 is better than two coin-1 choices
5	1	one coin of value 5
6	2	coin 1 plus the best answer for 5
10	2	$5 + 5$
11	3	$1 + dp[10]$, producing $1 + 5 + 5$

The sentinel `amount + 1` is safe because no valid answer can require more than `amount` positive-valued coins when coin 1 exists. More generally, it simply represents a value worse than every possible valid answer under the problem rules.

Code Walkthrough

The outer loop grows the solved prefix from 1 through `amount`. For each coin not larger than the current amount, the code considers taking that coin last. The remaining state `current - coin` is smaller and has already been solved. After all coins are tested, `dp[current]` is the minimum among every possible final coin.

Edge Cases, Alternatives, and Interview Notes

Amount zero returns zero. An empty coin list or unreachable amount returns -1. Duplicate denominations do not change the minimum but perform unneeded work. A top-down memoized solution uses the same update rule and is valuable when only a small subset of amounts is reached.

For example, `coins=[2]` and `amount=3` leaves $dp[3]$ at the sentinel because neither amount 1 nor amount 3 can be formed. This is different from a valid answer of zero, which occurs only when the requested amount is zero.

How to explain it simply

I define $dp[x]$ as the fewest coins needed for amount x. For every amount, I try each denomination as the final coin and extend the already best state $dp[x - c]$. The table prevents repeated recursion and gives $O(Am)$ time with $O(A)$ space.

DYNAMIC PROGRAMMING

Lesson 11: Combination Sum IV

Problem at a glance

Difficulty	Approach	Main tool
Medium	Ordered-count dynamic programming	One-dimensional DP

The Problem

Given distinct positive integers and a target, count the ordered sequences that sum to the target. With `nums=[1,2,3]` and target 4, the answer is 7. For example, $[1, 3]$ and $[3, 1]$ are different combinations.

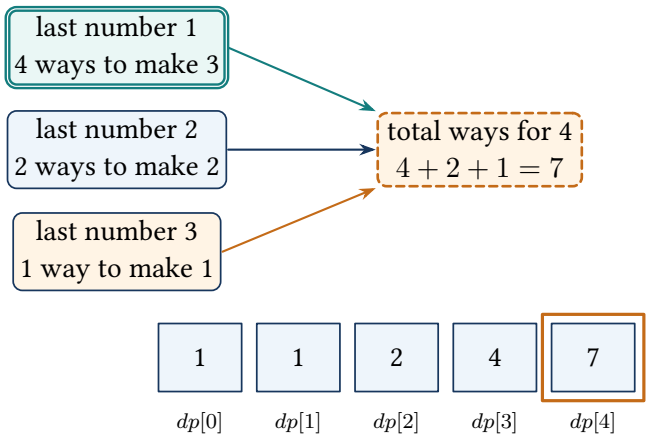
Reasoning and Loop Order

Let $dp[x]$ be the number of ordered sequences totaling x. The empty sequence is one way to create zero, so $dp[0] = 1$. For every target value from 1 upward, try every number as the final element: $dp[x] = \sum_{v \le x} dp[x - v]$. The outer loop must be the current total and the inner loop the choice number. Reversing them counts unordered coin combinations instead.

Key idea: the table counts every ordered sequence by its final value

Before computing $dp[x]$, all counts for smaller totals are complete. The sequences ending with different final values do not overlap, so adding $dp[x - v]$ over every valid v counts each ordered sequence exactly once.

Three last-number choices build the answer for 4



For total 4, group the sequences by their last number. The three groups contain 4, 2, and 1 sequence, giving $4 + 2 + 1 = 7$.

DP for [1,2,3] and target 4

Target x	0	1	2	3	4
$dp[x]$	1	1	2	4	7

Pseudocode and C++ Implementation

```
dp[0] = 1
for total from 1 through target:
    for value in nums:
        if value <= total:
            dp[total] += dp[total - value]
return dp[target]
```

Code 11: Complete C++ solution: Combination Sum IV

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    int combinationSum4(vector<int>& nums, int target) {
        vector<unsigned long long> dp(target + 1, 0);
        dp[0] = 1;
        for (int total = 1; total <= target; ++total) {
            for (const int value : nums) {
                if (value <= total) dp[total] += dp[total - value];
            }
        }
        return static_cast<int>(dp[target]);
    }
};
```

Why It Works, Complexity, and Common Mistakes

Every valid sequence totaling x has one final value v; removing it leaves a sequence counted by $dp[x-v]$. These groups do not overlap, so their counts add exactly. Time is $O(Tm)$, space is $O(T)$, where T is the target and m is the number of values. Do not set $dp[0] = 0$, and remember that order matters in this problem.

How to explain it in an interview

I count ways for every smaller total. The base $dp[0] = 1$ represents choosing nothing. For each total, I try each number as the last choice and add the ways to form the remaining total. This produces ordered sequences in $O(Tm)$.

Why the Loop Order Counts Ordered Sequences

The worked example contains seven sequences for target 4. The important detail is that $[1, 3]$ and $[3, 1]$ are different. The outer loop chooses the total being built, while the inner loop tries every possible final value. For each final value, all prefixes that form the remaining total are appended with that value.

If the loops were reversed—numbers outside and totals inside—the table would count unordered coin combinations instead. Loop order is part of the meaning of the state, not merely a coding preference.

Complete Dry Run

Seven sequences for target 4

Total	Ways	Sequences added by their last value
0	1	empty prefix
1	1	$[1]$
2	2	$[1, 1]$, $[2]$
3	4	$[1, 1, 1]$, $[2, 1]$, $[1, 2]$, $[3]$
4	7	$[1, 1, 1, 1]$, $[2, 1, 1]$, $[1, 2, 1]$, $[3, 1]$, $[1, 1, 2]$, $[2, 2]$, $[1, 3]$

Code Walkthrough and Safety

The unsigned table provides extra intermediate range, while the problem guarantees the final answer fits a signed 32-bit integer. The base value 1 at index zero represents the one valid empty prefix. Every access uses `total - value` only after confirming that the result is non-negative.

Edge Cases and Follow-ups

Target zero has one empty sequence. A number greater than the target is simply ignored. Inputs must be positive; allowing zero or negative values would break the increasing-state dependency and could create infinitely many sequences. A common follow-up asks for unordered combinations, which requires reversing the loop order.

Practice checkpoint

Explain why $dp[0]$ is one even though the answer contains no number. Then rewrite the update rule for combinations where order does not matter.

TREE CONSTRUCTION

Lesson 12: Construct Binary Tree from Preorder and Inorder Traversal

Problem at a glance

Difficulty	Approach	Main tool
Medium	Recursive partitioning	Tree and hash map

The Problem

Preorder lists each root before its subtrees. Inorder lists the left subtree, then the root, then the right subtree. Reconstruct and return the unique tree when all values are distinct.

Example

`preorder=[3,9,20,15,7]` and `inorder=[9,3,15,20,7]` reconstruct the tree rooted at 3, with left child 9 and right subtree rooted at 20.

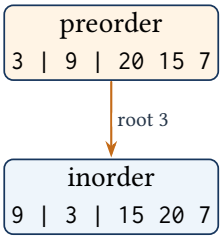
Recursive Split

The next unused preorder value is the current root. Its position in inorder splits the current interval into left and right subtrees. A hash map gives that position in average $O(1)$ time, avoiding a repeated linear search.

Key idea: the next preorder value roots the current inorder interval

Each recursive call owns exactly one consecutive inorder interval. The shared preorder index points to that interval's root; after the left call consumes all left-subtree nodes, the right call begins at exactly the next unused preorder value.

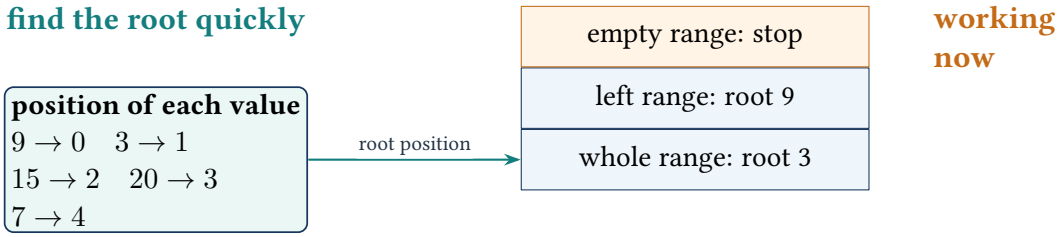
Preorder gives the root; inorder shows its left and right sides



left side: `[9]`

right side: `[15,20,7]`

The map finds each root; the stack remembers unfinished ranges



The map tells us where the root sits. The stack remembers which left and right parts still need to be built.

Pseudocode

```
build(left, right):
    if left > right: return null
    rootValue = preorder[preorderIndex++]
    root = new TreeNode(rootValue)
    middle = inorderPosition[rootValue]
    root.left = build(left, middle - 1)
    root.right = build(middle + 1, right)
    return root
```

Code 12: Complete C++ solution: Construct Binary Tree

```
#include <unordered_map>
#include <vector>

using namespace std;

class Solution {
    const vector<int>* preorderValues = nullptr;
    unordered_map<int, int> inorderIndex;
    int preorderIndex = 0;

    TreeNode* build(int left, int right) {
        if (left > right) return nullptr;
        const int rootValue = (*preorderValues)[preorderIndex++];
        TreeNode* root = new TreeNode(rootValue);
        const int middle = inorderIndex[rootValue];
        root->left = build(left, middle - 1);
        root->right = build(middle + 1, right);
        return root;
    }

public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        preorderValues = &preorder;
        preorderIndex = 0;
```

```
        inorderIndex.clear();
        for (int i = 0; i < static_cast<int>(inorder.size()); ++i) {
            inorderIndex[inorder[i]] = i;
        }
        return build(0, static_cast<int>(inorder.size()) - 1);
    }
};
```

Why It Works, Time, and Space

Each call consumes exactly one preorder root and assigns it the inorder range that contains exactly its descendants. Building the left range before the right range matches preorder order. Every node is created once, so time and stored map space are $O(n)$; recursion uses $O(h)$ stack space.

Common mistakes

Build the left subtree before the right subtree because preorder lists the entire left subtree first. Use interval boundaries instead of copying array slices. The distinct-value assumption is what makes the inorder index unique.

Direct Method and Its Limitation

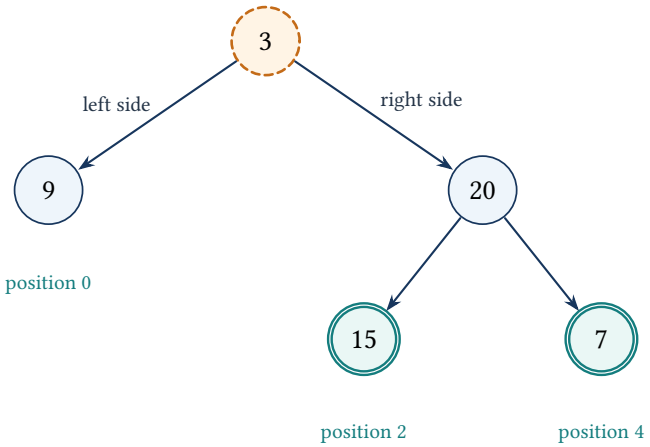
A simple recursive version can search the full inorder array for every new root. It constructs the correct tree, but the repeated linear searches cost $O(n^2)$ for a skewed tree. Building a value-to-index map once changes every root lookup to average $O(1)$, producing an $O(n)$ construction.

Detailed Recursive Walkthrough

Preorder `[3,9,20,15,7]`, inorder `[9,3,15,20,7]`

Call	Next preorder root	Inorder interval	Result
1	3	$[0, 4]$	split at index 1; left $[0, 0]$, right $[2, 4]$
2	9	$[0, 0]$	leaf; both child intervals are empty
3	20	$[2, 4]$	split at index 3
4	15	$[2, 2]$	left leaf of 20
5	7	$[4, 4]$	right leaf of 20

The tree after all left and right parts are built



The preorder pointer advances exactly once per created node. The inorder boundaries determine when a subtree is empty; no array slices need to be allocated.

Code Walkthrough

The public method stores a pointer to the preorder vector, resets the shared index, and creates the inorder lookup map. The helper reads the next root, allocates it, finds its inorder position, and recursively builds the left interval before the right interval. The base case `left > right` returns a null child.

Assumptions, Edge Cases, and Follow-ups

The traversals must contain the same distinct values and represent a valid tree. Empty arrays produce a null root. A skewed tree uses $O(n)$ recursion depth. Duplicate values make the reconstruction ambiguous unless extra occurrence information is supplied. A useful variation reconstructs from inorder and postorder; in that case roots are consumed from the end and the right subtree must be built first.

How to explain it simply

Preorder tells me the next root. Its position in inorder divides the exact left and right descendant ranges. A hash map removes repeated searches, and a single preorder index ensures each node is consumed once, for $O(n)$ time.

TWO POINTERS

Lesson 13: Container With Most Water

Problem at a glance

Difficulty	Approach	Main tool
Medium	Two pointers	Array

The Problem

Each array value is the height of a vertical line. Choose two lines that form the container with the greatest area. For indexes $l < r$: area $= \min(h_l, h_r)(r - l)$.

Example

For `[1,8,6,2,5,4,8,3,7]`, lines at indexes 1 and 8 create area $\min(8, 7) \cdot 7 = 49$.

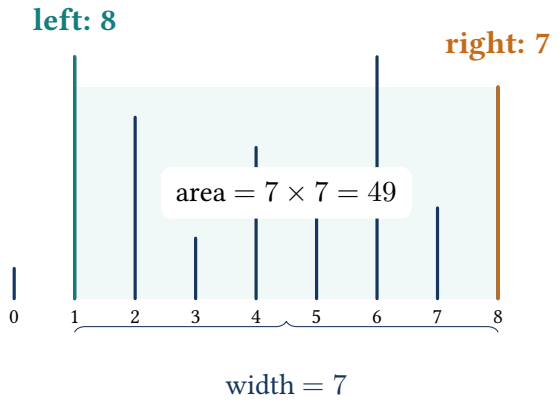
From Brute Force to Two Pointers

Checking every pair takes $O(n^2)$. Start with the widest possible pair. The shorter line limits the area. Moving the taller line inward only reduces width while keeping the same limiting height or worse, so it cannot improve that pair. Move the shorter line and search for a taller boundary.

Key idea: every removed wall is safely dominated

Before each iteration, every pair outside the current interval has already been measured or proved unable to beat a measured pair. Once the current area is recorded, any narrower pair that keeps the shorter wall has no greater limiting height, so that wall may be discarded.

The shorter wall limits how much water fits



Pseudocode and C++ Implementation

```
left = 0, right = n - 1, best = 0
while left < right:
    best = max(best, min(height[left], height[right]) * (right-left))
    if height[left] <= height[right]: left++
    else: right--
return best
```

Code 13: Complete C++ solution: Container With Most Water

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    int maxArea(vector<int>& height) {
        int left = 0;
        int right = static_cast<int>(height.size()) - 1;
        long long best = 0;

        while (left < right) {
            const long long width = right - left;
            const long long wall = min(height[left], height[right]);
            best = max(best, width * wall);
            if (height[left] <= height[right]) ++left;
            else --right;
        }
        return static_cast<int>(best);
    }
};
```

Why It Works, Time, and Space

When a pair is processed, every pair that keeps its shorter wall and uses a smaller width is no better. Discarding that wall cannot remove the best unseen pair. Each pointer moves at most $n - 1$ times, so time is $O(n)$ and space is $O(1)$.

Common mistake

Area uses the smaller height, not the larger one. This is not binary search: the pointers move according to which boundary limits the current container.

Detailed Pointer Dry Run

Worked example `[1,8,6,2,5,4,8,3,7]`

Left, right	Heights	Width	Area	Pointer removed
0, 8	1, 7	8	8	left: height 1 limits the pair
1, 8	8, 7	7	49	right: height 7 is shorter
1, 7	8, 3	6	18	right
1, 6	8, 8	5	40	either equal wall may move
2, 6	6, 8	4	24	left

No later pair exceeds 49, so the answer is the container between indexes 1 and 8.

Why Moving the Shorter Wall Is Safe

Suppose the left wall is shorter. Keeping it while moving the right pointer inward reduces the width and cannot increase the limiting height above that same left wall. Every such pair is no better than the current pair. The only chance to improve is to discard the shorter wall and search for a taller one. This reason is the key rule behind the two-pointer method.

Code Walkthrough

The code begins with the widest possible pair. It calculates width and the smaller boundary with `long long` intermediates, updates the best area, and advances exactly one pointer. Since every iteration shrinks the interval, the loop terminates after at most $n - 1$ moves.

Edge Cases and Follow-ups

Two lines form the smallest valid input. Zero-height lines are allowed and simply create zero area. Equal boundary heights may move either side because the current limiting height cannot improve while the width shrinks. A common follow-up asks why sorting is impossible: the original index distance is part of the area and must be keepd.

How to explain it simply

I start with maximum width. The shorter wall limits the current area, and any narrower pair that keeps that wall cannot do better. I move the shorter side, test each remaining choice once, and obtain $O(n)$ time and $O(1)$ space.

ARRAYS, HASHING, AND BITS

Lesson 14: Contains Duplicate

Problem at a glance

Difficulty	Approach	Main tool
Easy	Membership hashing	Hash set

The Problem

Return true when any integer appears at least twice. Return false when every value is distinct.

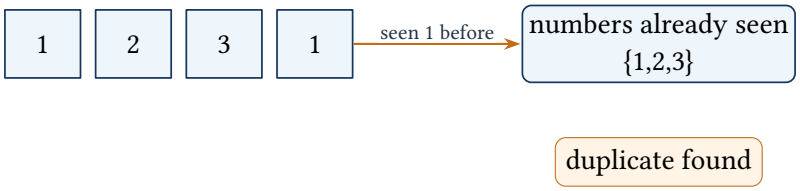
How We Get to the Solution

Comparing every pair costs $O(n^2)$. Sorting costs $O(n \log n)$ and changes the input order. A hash set answers the exact question we need: have we seen this value before?

Key idea: the set equals the distinct processed prefix

Immediately before reading `nums[i]`, the set contains every distinct value at an earlier index and no future value. A membership hit is a real duplicate; otherwise insertion keeps the statement for the next iteration.

Stop when a number appears again



Pseudocode

```
seen = empty set
for value in nums:
    if value is already in seen: return true
    insert value into seen
return false
```

Code 14: Complete C++ solution: Contains Duplicate

```cpp
#include <unordered_set>
#include <vector>

using namespace std;

class Solution {
public:
```

```
    bool containsDuplicate(vector<int>& nums) {
        unordered_set<int> seen;
        seen.reserve(nums.size());
        for (const int value : nums) {
            if (!seen.insert(value).second) return true;
        }
        return false;
    }
};
```

Complexity and Revision

Hash insertion is average-case $O(1)$, giving average $O(n)$ time and $O(n)$ space. The worst-case hash bound is $O(n^2)$, although interview analysis normally states the average case. Empty and one-element arrays return false.

Pattern clue

Questions asking whether an item has appeared before usually suggest a set. Questions needing information attached to that item suggest a map.

Example Walkthrough

For `[1,2,3,1]`, the set evolves as follows:

Insertion result reveals the duplicate

Current value	Set before insertion	Result
1	$\{\}$	insert succeeds
2	$\{1\}$	insert succeeds
3	$\{1, 2\}$	insert succeeds
1	$\{1, 2, 3\}$	insert fails; return true

The set is enough because only membership matters. A map would store an unneeded second value.

Code Walkthrough and Alternatives

`seen.reserve(nums.size())` reduces avoidable rehashing but is not required for why the solution works. `insert` returns a pair whose Boolean field is false when the key already exists, allowing lookup and insertion in one operation. Returning immediately avoids scanning the rest of the array.

Sorting is an alternative: sort, then compare adjacent values in $O(n \log n)$ time and constant or implementation-dependent extra space. The nested-loop baseline uses $O(1)$ space but $O(n^2)$ time.

Edge Cases and Interview Notes

Empty and one-element inputs are distinct by definition. Negative values and zero require no special handling. Repeated values may appear more than twice; the second occurrence is enough. Hash operations are average-case constant time, not a deterministic worst-case guarantee.

Practice checkpoint

Change the question to "return the first duplicated value" and explain why the same scan works. Then change it to "return each value's frequency" and identify why a map is now required.

ARRAYS, HASHING, AND BITS

Lesson 15: Counting Bits

Problem at a glance

Difficulty	Approach	Main tool
Easy	Bit dynamic programming	Array

The Problem

For every integer from 0 through n, return the number of set bits in its binary representation. For $n = 5$, return `[0,1,1,2,1,2]`.

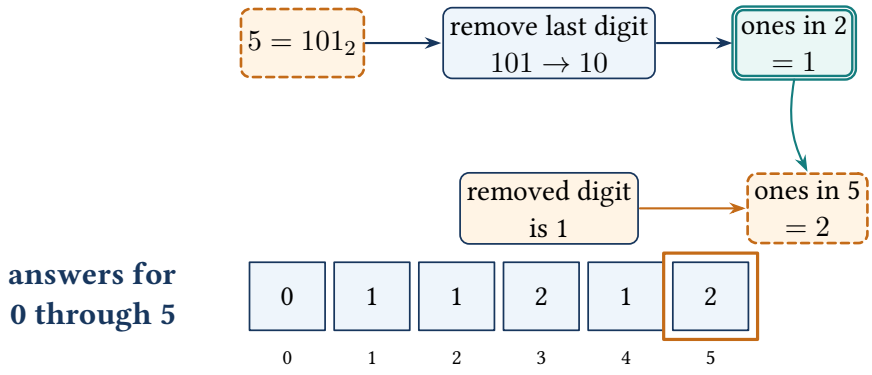
Update Rule

Counting every number bit by bit takes $O(n \log n)$. Dividing i by two removes its last binary digit, and $i\&1$ tells us whether that removed digit was one: $bits[i] = bits[i >> 1] + (i\&1)$.

Key idea: the shifted dependency is already solved

When the loop reaches i, the index $i >> 1$ is smaller than i, so its bit count is already correct. Adding the removed least-significant bit gives the exact count for i and extends the solved prefix by one entry.

Use the answer for 2 to find the answer for 5



Binary update rule

i	0	1	2	3	4	5
binary	0	1	10	11	100	101
set bits	0	1	1	2	1	2

Pseudocode

```
bits[0] = 0
for value from 1 through n:
    bits[value] = bits[value / 2] + (value mod 2)
return bits
```

Code 15: Complete C++ solution: Counting Bits

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    vector<int> countBits(int n) {
        vector<int> bits(n + 1, 0);
        for (int value = 1; value <= n; ++value) {
            bits[value] = bits[value >> 1] + (value & 1);
        }
        return bits;
    }
};
```

Why It Works and Complexity

The update rule reuses the already-computed answer for $\lfloor i/2 \rfloor$ and adds the final bit. Time is $O(n)$; extra working space is $O(1)$ excluding the required $O(n)$ output. The base $bits[0] = 0$ is essential.

Building the Bit Update Rule

Every non-negative integer i can be written as $i = 2\lfloor i/2 \rfloor + (i \bmod 2)$. Shifting right removes the final bit. The expression `i & 1` is one for an odd number and zero for an even number. So the number of set bits is the answer for the shifted value plus the removed bit.

Binary states from 0 through 5

i	Binary	$i >> 1$	Final bit	$bits[i]$
0	000	–	–	0
1	001	0	1	$bits[0] + 1 = 1$
2	010	1	0	$bits[1] + 0 = 1$
3	011	1	1	$bits[1] + 1 = 2$
4	100	2	0	$bits[2] + 0 = 1$
5	101	2	1	$bits[2] + 1 = 2$

Code Explanation, Alternatives, and Edge Cases

The output vector is filled with zeros, so the base case is already present. The loop starts at one; `value » 1` is always smaller and has already been computed. A direct method could count bits separately for every integer in $O(n \log n)$. Another useful update rule is $bits[i] = bits[i\&(i-1)] + 1$, which removes the lowest set bit.

For $n = 0$, the output is `[0]`. The vector is required output space, so it should not be described as avoidable extra memory.

How to explain it in an interview

I reuse the count for the number with its final binary digit removed. Right shift gives that smaller number, and `i & 1` tells whether the removed digit contributes one. Each answer is built in constant time, so the entire range takes $O(n)$.

DIRECTED GRAPHS

Lesson 16: Course Schedule

Problem at a glance

Difficulty	Approach	Main tool
Medium	DFS cycle detection	Directed graph

The Problem

Courses are vertices. A prerequisite pair $[a, b]$ means $b \to a$: course b must be completed before a. Return whether every course can be finished.

Key Observation

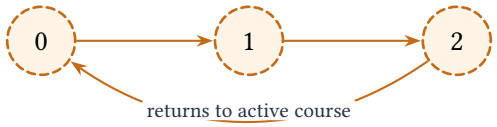
A valid schedule exists exactly when the prerequisite graph has no directed cycle. The three-state method tracks both fully visited nodes and nodes in the current DFS path. A three-state array expresses the same idea:

- 0: unvisited,
- 1: visiting in the current recursion path,
- 2: completely processed and safe.

Key idea: state 1 is exactly the active DFS path

A course changes from unvisited to visiting when its DFS frame begins and changes to done only after all outgoing edges are proved safe. So an edge to state 1 is a back edge and a cycle, whereas an edge to state 2 can be reused without more recursion.

Following the arrows returns to course 0



Pseudocode

```
hasCycle(course):
    if state[course] == visiting: return true
    if state[course] == done: return false
    state[course] = visiting
    for next in graph[course]:
        if hasCycle(next): return true
    state[course] = done
    return false
```

Code 16: Complete C++ solution: Course Schedule

```cpp
#include <vector>

using namespace std;

class Solution {
    bool hasCycle(int course,
                  const vector<vector<int>>& graph,
                  vector<int>& state) {
        if (state[course] == 1) return true;
        if (state[course] == 2) return false;
        state[course] = 1;
        for (const int next : graph[course]) {
            if (hasCycle(next, graph, state)) return true;
        }
        state[course] = 2;
        return false;
    }

public:
    bool canFinish(int numCourses,
                   vector<vector<int>>& prerequisites) {
        vector<vector<int>> graph(numCourses);
        for (const auto& edge : prerequisites) graph[edge[1]].push_back(edge[0]);
        vector<int> state(numCourses, 0);
        for (int course = 0; course < numCourses; ++course) {
            if (hasCycle(course, graph, state)) return false;
        }
        return true;
    }
};
```

Why It Works, Time, and Space

A visiting neighbor is an ancestor in the current DFS path, so that edge closes a directed cycle. Done nodes have already been proved cycle-free and need not be explored again. Time and storage are $O(V + E)$; recursion depth is $O(V)$ in the worst case.

How to explain it in an interview

I build the prerequisite graph and run DFS from every course. A three-state array distinguishes an active recursion path from a completed node. Reaching an active node proves a cycle, so the courses cannot all be finished.

Why One Visited Boolean Is Not Enough

A node seen in a different completed DFS branch is safe; a node seen again in the current recursion path proves a cycle. A single Boolean cannot distinguish those situations. The three states mean unvisited, visiting, and completely processed. Only an edge to a visiting node is a back edge.

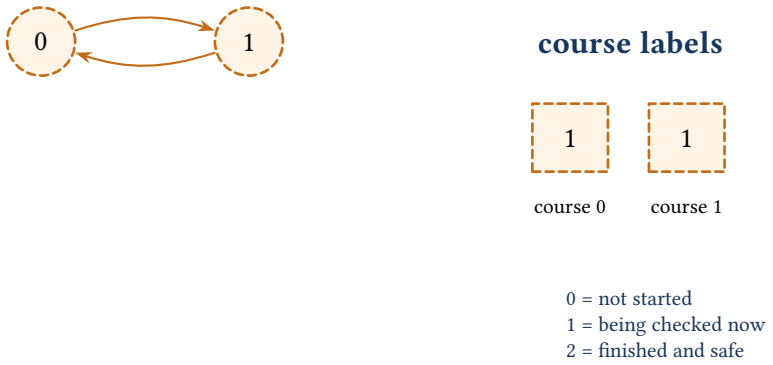
Detailed DFS Trace

Cycle in prerequisites `[[1,0],[0,1]]`

Action	States $(0, 1)$	Meaning
Enter course 0	$(1, 0)$	0 is on the active path
Follow $0 \to 1$	$(1, 1)$	1 joins the active path
Follow $1 \to 0$	$(1, 1)$	0 is already visiting; cycle found

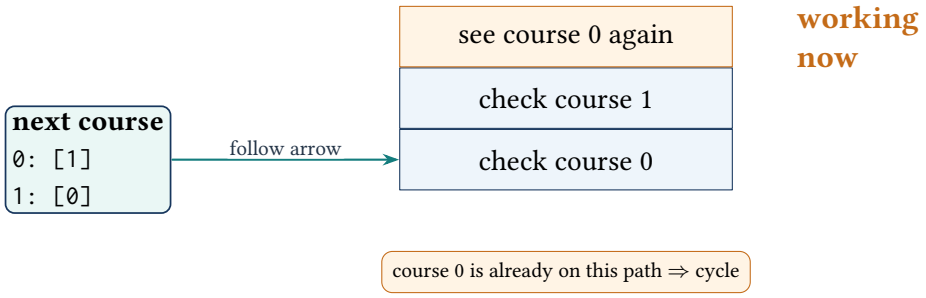
For `[[1,0]]`, course 1 finishes, changes to state 2, and no back edge is found.

Three labels separate new, active, and finished courses



An arrow to a course labeled 1 returns to the path being checked, so there is a cycle.

Course arrows and the path being checked



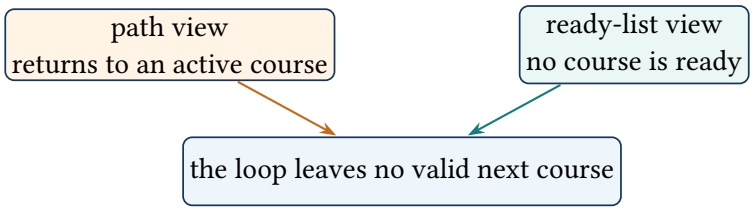
Code Walkthrough

The adjacency list stores edges from each prerequisite to the course unlocked by it. The helper returns immediately for visiting and done states. Before exploring neighbors it marks the course visiting; after every neighbor is proved safe, it marks the course done. The outer loop is needed because the graph may contain several disconnected components.

Alternative, Edge Cases, and Follow-ups

Kahn's topological-sort algorithm is an equally strong alternative: repeatedly take zero-indegree courses and check whether all courses are removed. An empty prerequisite list is immediately possible. A self-dependency such as $[0, 0]$ is a cycle. Duplicate edges do not change the DFS result, though they add repeated adjacency work.

Two methods notice the same impossible loop



Practice checkpoint

Modify the solution to return one valid course order. Decide whether DFS postorder or Kahn's queue makes that extension easier to explain.

STRING DP

Lesson 17: Decode Ways

Problem at a glance

Difficulty	Approach	Main tool
Medium	Prefix dynamic programming	String and DP

The Problem

Digits map from 1 through 26 to letters. Count the ways to split a digit string into valid codes. A zero cannot be decoded by itself.

Examples

`"12"` has two decodings: `1|2` and `12`. `"226"` has three. `"06"` has zero because a code cannot start with zero.

DP State

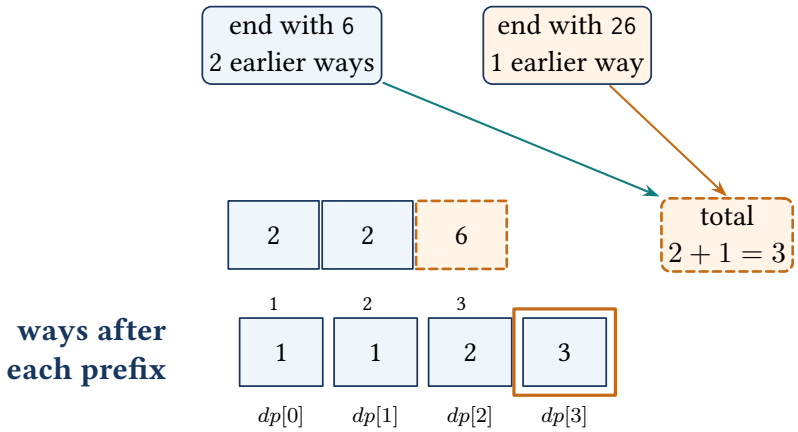
Let $dp[i]$ be the number of decodings for the first i characters. Start $dp[0] = 1$ for the empty prefix. At position i:

- if the one-digit code is 1 through 9, add $dp[i-1]$;
- if the two-digit code is 10 through 26, add $dp[i-2]$.

Key idea: each prefix count contains every valid ending once

After processing length i, $dp[i]$ counts exactly the decodings of the first i characters. Every decoding ends with either one nonzero digit or one valid value from 10 through 26, so the previous groups are complete and do not overlap.

For 226, count endings 6 and 26



Every decoding ends with either 6 or 26. These are separate groups, so their counts are added.

Decoding "226"

Prefix length	0	1	2	3
Prefix	empty	2	22	226
Ways	1	1	2	3

Pseudocode

```
if text is empty or starts with 0: return 0
dp[0] = 1
dp[1] = 1
for prefixLength from 2 through text.length:
    if last digit is 1 through 9:
        dp[prefixLength] += dp[prefixLength - 1]
    if last two digits form 10 through 26:
        dp[prefixLength] += dp[prefixLength - 2]
return dp[text.length]
```

Code 17: Complete C++ solution: Decode Ways

```cpp
#include <string>
#include <vector>

using namespace std;

class Solution {
public:
    int numDecodings(string s) {
        if (s.empty() || s[0] == '0') return 0;
        const int n = static_cast<int>(s.size());
        vector<int> dp(n + 1, 0);
        dp[0] = 1;
```

```
        dp[1] = 1;

        for (int i = 2; i <= n; ++i) {
            if (s[i - 1] != '0') dp[i] += dp[i - 1];
            const int twoDigits = (s[i - 2] - '0') * 10 + (s[i - 1] - '0');
            if (twoDigits >= 10 && twoDigits <= 26) dp[i] += dp[i - 2];
        }
        return dp[n];
    }
};
```

Why It Works, Complexity, and Common Mistakes

Every valid decoding ends in exactly one valid one-digit or two-digit code, so the update rule splits all possibilities without overlap. Time and space are $O(n)$. Carefully reject standalone zero, values above 26, and prefixes such as `"06"`. The DP can be reduced to $O(1)$ space because only the previous two entries are needed.

Brute Force and Repeated Prefixes

A recursive solution may decode one digit, decode two digits when valid, and recurse on the remaining suffix. For a string of many ones, both branches are available repeatedly and the recursion grows exponentially. The same suffix or prefix length is solved many times. Dynamic programming stores the number of decodings for each prefix exactly once.

Detailed Prefix Dry Run

Decoding "226"

Prefix	One-digit choice	Two-digit choice	Ways
empty	–	–	$dp[0] = 1$
2	2 is valid	–	$dp[1] = 1$
22	append 2: $dp[1]$	append 22: $dp[0]$	2
226	append 6: $dp[2]$	append 26: $dp[1]$	3

The three decodings are 2|2|6, 22|6, and 2|26.

Code Walkthrough

The first-character guard rejects an empty input or leading zero. The table uses prefix lengths, so `dp[i]` describes the first i characters. If the last character is nonzero, every decoding of the previous prefix can append it. If the last two characters form 10 through 26, every decoding two positions back can append that pair.

Important Zero Cases and Follow-ups

`"10"` and `"20"` each have one decoding. `"30"`, `"06"`, and `"100"` are invalid. The pair test must require at least 10, not merely at most 26. Since only two earlier states are read, the table can be compressed to two integers for $O(1)$ extra space.

Zero-heavy strings that expose invalid transitions

String	Ways	Reason
`0`	0	zero cannot stand alone
`10`	1	only the pair 10 is valid
`101`	1	`10|1`; zero blocks the one-digit transition
`100`	0	the final zero cannot pair with zero
`226`	3	`2|2|6`, `22|6`, and `2|26`

How to explain it simply

Every decoding ends with either one valid digit or one valid two-digit code. Those two groups do not overlap, so I add the matching prefix counts. The DP scans once in $O(n)$ time and explicitly handles zeros.

LINKED LISTS

Lesson 18: Linked List Cycle

Problem at a glance

Difficulty	Approach	Main tool
Easy	Floyd's two-pointer cycle detection	Linked list

The Problem

Return whether repeatedly following `next` can revisit a node. The platform's cycle position is not passed to the function.

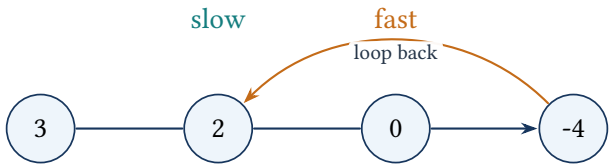
Why Slow and Fast Pointers Meet

A hash set can store visited pointers in $O(n)$ space. Floyd's algorithm uses two pointers: slow advances one edge and fast advances two. If there is no cycle, fast reaches null. Inside a cycle, fast gains one node per iteration on slow and must eventually meet it.

Key idea: null proves an end; pointer equality proves repetition

After k rounds, slow has moved k edges and fast has moved $2k$. If fast reaches null, the list is finite. If both pointers reference the same node, different traversal distances have reached one object, which can happen only after entering a cycle.

Fast catches slow when the list loops



Pseudocode

```
slow = head
fast = head
while fast exists and fast.next exists:
    slow = slow.next
    fast = fast.next.next
    if slow equals fast: return true
return false
```

Code 18: Complete C++ solution: Linked List Cycle

```cpp
using namespace std;

class Solution {
public:
    bool hasCycle(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;
        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
            if (slow == fast) return true;
        }
        return false;
    }
};
```

Time and Space

Both pointers make $O(n)$ total progress before fast reaches null or the pointers meet. Extra space is $O(1)$. Check both `fast` and `fast->next` before advancing two steps.

Pattern clue

Use slow and fast pointers when repeated next-state transitions may enter a cycle and the problem asks for constant extra space.

From a Hash Set to Constant Space

The simplest solution stores every visited node pointer in a hash set. Before advancing, it checks whether the current pointer was seen before. This is linear time and linear space. Floyd's method removes the set by using two walkers with different speeds.

If a cycle exists, slow and fast eventually enter it. After that, fast gains one node on slow per iteration, measured around the cycle. The distance modulo the cycle length must eventually become zero, so the pointers meet.

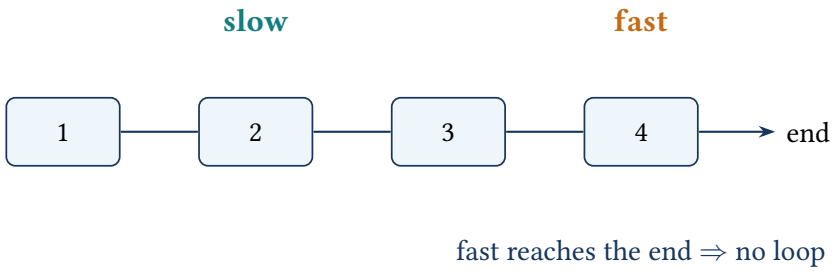
Pointer Dry Run

List $3 \to 2 \to 0 \to -4$, with $-4 \to 2$

Iteration	Slow	Fast
start	3	3
1	2	0
2	0	2
3	-4	-4

The equal pointers refer to the same node object, not merely equal stored values. So a repeated value in an acyclic list does not create a false cycle.

Without a loop, fast reaches the end



Code Explanation and Safety

Both pointers begin at the head. The loop verifies that fast and `fast->next` exist before moving fast two edges. The comparison occurs after movement; if a cycle exists, the pointers eventually become identical. If fast reaches null, the list has a finite end.

Edge Cases and Interview Follow-ups

Test an empty list, one node without a loop, one node pointing to itself, a two-node cycle, and a long non-cyclic prefix before a short cycle. A common follow-up asks for the cycle's entry node: after the first meeting, reset one pointer to head and advance both one step at a time; their next meeting is the entry.

Practice checkpoint

Explain why comparing `slow->val == fast->val` is incorrect. Then derive how resetting one pointer after the meeting locates the cycle entry.

STRING DESIGN

Lesson 19: Encode and Decode Strings

Problem at a glance

Difficulty	Approach	Main tool
Medium	Length-prefixed encoding	Strings

The Problem

Design two operations: encode a vector of arbitrary strings into one string, then decode it back exactly. Strings may be empty and may contain delimiter characters.

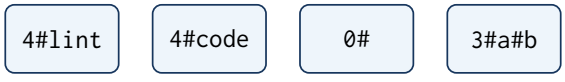
Why a Delimiter Alone Fails

Joining with # is ambiguous when a payload itself contains #. Prefix every string with its decimal length and a delimiter: `length#payload`. The decoder reads the length first, so it knows exactly how many following characters belong to the payload.

Key idea: the cursor always begins at the next frame length

After decoding a frame, the cursor has advanced past its decimal length, the separator, and exactly the announced payload characters. Separators inside a payload are never interpreted, so the next cursor position is clear.

Each stored string says its own length



the number says how many characters come next

Pseudocode

```
encode(strings): append length + '#' + string for every string
decode(data):
    i = 0
    while i < data.length:
        find next '#'
        parse length before it
        copy exactly length characters after it
        advance i to the next frame
```

Code 19: Complete C++ solution: Encode and Decode Strings

```cpp
#include <string>
#include <vector>

using namespace std;

class Codec {
public:
    string encode(const vector<string>& strings) {
        string encoded;
        for (const string& value : strings) {
            encoded += to_string(value.size());
            encoded.push_back('#');
            encoded += value;
        }
        return encoded;
    }

    vector<string> decode(const string& encoded) {
        vector<string> result;
        size_t index = 0;
        while (index < encoded.size()) {
            const size_t delimiter = encoded.find('#', index);
            const size_t length =
                static_cast<size_t>(stoull(encoded.substr(index, delimiter-index)));
            const size_t start = delimiter + 1;
            result.push_back(encoded.substr(start, length));
            index = start + length;
        }
        return result;
    }
};
```

Why It Works, Complexity, and Edge Cases

Every frame has one clear numeric prefix. The decoder consumes exactly the announced payload length, even when the payload contains # or is empty. Encoding and decoding both take $O(N)$ time for total character count N, with $O(N)$ returned storage. Test empty vectors, empty strings, delimiter characters, and multi-digit lengths.

How to explain it in an interview

I avoid delimiter ambiguity by storing the payload length before each string. The decoder finds the separator, parses the length, copies exactly that many characters, and continues at the next frame.

Why Simple Joining Is Not Reversible

Suppose the separator is #. Encoding `["ab","c#d"]` as `ab#c#d` loses the information that the second separator belongs inside a payload. Escaping separators is possible, but then the escape character itself must also be escaped. A length prefix is simpler because payload contents no longer affect parsing.

Frame-by-Frame Dry Run

Encoding `["lint","code","","a#b"]`

String	Frame	Decoder action
`lint`	`4#lint`	read 4, then copy four characters
`code`	`4#code`	read 4, then copy four characters
empty	`0#`	read 0 and append an empty string
`a#b`	`3#a#b`	copy three characters, including #

Code Walkthrough

Encoding appends the decimal length, one separator, and the untouched payload. Decoding searches only for the separator that terminates the numeric length. It parses that prefix, calculates the payload start, copies exactly the stated number of characters, and advances directly to the next frame. The platform supplies strings produced by the matching encoder, so invalid input handling is outside this problem's contract.

Edge Cases, Complexity Details, and Variations

Distinguish an empty vector from a vector containing one empty string: their encodings are respectively empty and `0#`. Multi-digit lengths must be parsed completely. Total work is linear in the encoded data size, although repeated string growth may allocate; reserving capacity is a practical optimization when total size is known.

Practice checkpoint

Decode `5#hello0#3#a#b` by hand. At each step record the separator position, parsed length, payload interval, and next frame index.

HEAPS AND STREAMING DATA

Lesson 20: Find Median from Data Stream

Problem at a glance

Difficulty	Approach	Main tool
Hard	Two balanced heaps	Priority queues

The Problem

Design a class that accepts integers one at a time and returns the median of all values seen so far. An odd-sized stream has one middle value; an even-sized stream uses the average of the two middle values.

Two-Heap Model

A sorted vector makes median lookup easy but insertion costs $O(n)$. Instead, split the stream into two halves:

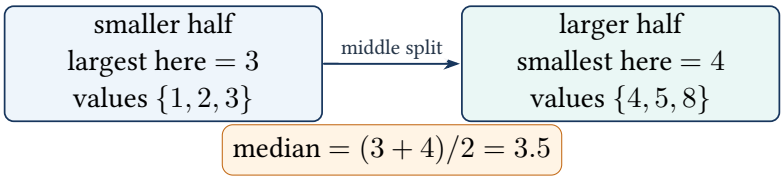
- `lower`: a max heap containing the smaller half;
- `upper`: a min heap containing the larger half.

Keep $|\texttt{lower}| = |\texttt{upper}|$ or one extra item in `lower`, and keep every lower value no greater than every upper value.

Key idea: the heap tops are always the middle boundary

After every insertion, all lower-heap values are no greater than all upper-heap values, and the lower heap has either the same size or one extra value. These two facts place the median at `lower.top()` or between the two heap tops.

The two top values give the median



Adding a Number

Push the new value into `lower`. Move the largest lower value to `upper`; this restores ordering. If `upper` becomes larger, move its smallest value back to `lower`. This restores the size rule.

Pseudocode

```
addNum(value):
    push value into lower max heap
    move lower.maximum into upper min heap
    if upper has more values than lower:
        move upper.minimum back into lower

findMedian():
    if lower has one extra value: return lower.maximum
    return average(lower.maximum, upper.minimum)
```

Code 20: Complete C++ data structure: MedianFinder

```cpp
#include <functional>
#include <queue>
#include <vector>

using namespace std;

class MedianFinder {
    priority_queue<int> lower;
    priority_queue<int, vector<int>, greater<int>> upper;

public:
    void addNum(int num) {
        lower.push(num);
        upper.push(lower.top());
        lower.pop();

        if (upper.size() > lower.size()) {
            lower.push(upper.top());
            upper.pop();
        }
    }

    double findMedian() const {
        if (lower.size() > upper.size()) return lower.top();
        return (static_cast<double>(lower.top()) + upper.top()) / 2.0;
    }
};
```

Why It Works and Complexity

Moving the maximum lower value into `upper` ensures all lower values remain no greater than all upper values. Rebalancing changes sizes by one, so the middle value or values remain at the heap tops. Insertion costs $O(\log n)$, median lookup costs $O(1)$, and stored space is $O(n)$.

Common mistakes

Use a max heap for the lower half and a min heap for the upper half. Convert to `double` before averaging to avoid integer truncation and overflow.

From a Sorted Vector to Two Heaps

Keeping every value in a sorted vector makes the median easy to read, but an insertion may shift $O(n)$ values. Sorting from scratch after every insertion is even more expensive. Two heaps keep only the ordering information needed for the middle: the maximum of the lower half and the minimum of the upper half.

The keeped invariants are:

1. every value in `lower` is at most every value in `upper`;
2. the heaps have equal sizes or `lower` has one extra value.

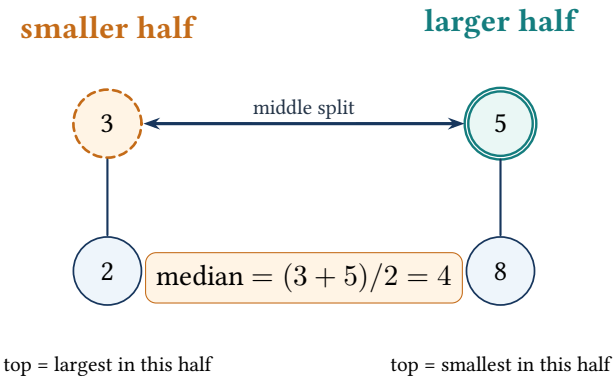
Insertion Dry Run

Adding $5, 2, 8, 3$

Inserted	Lower max heap	Upper min heap	Median
5	$\{5\}$	$\{\}$	5
2	$\{2\}$	$\{5\}$	3.5
8	$\{5, 2\}$	$\{8\}$	5
3	$\{3, 2\}$	$\{5, 8\}$	4

The braces show heap contents conceptually; only each heap's top is ordered for direct access.

The top of each heap is one middle value



Code Walkthrough

Every new value first enters `lower`. Moving `lower.top()` to `upper` guarantees the ordering boundary. If upper now has more values, moving its minimum back restores the size rule. The median is then one lower top for an odd count or the average of both tops for an even count.

Edge Cases, Overflow, and Follow-ups

The problem calls `findMedian` only after at least one insertion. Duplicate and negative values require no special handling. Cast before adding the two middle values; otherwise two large integers can overflow before being converted. A harder follow-up supports deletions or a sliding window, usually with lazy deletion maps because priority queues cannot erase arbitrary values efficiently.

How to explain it simply

I split the stream around the median. A max heap exposes the largest lower value and a min heap exposes the smallest upper value. After every insertion I restore ordering and a one-element size difference, giving $O(\log n)$ updates and $O(1)$ median queries.

BINARY SEARCH

Lesson 21: Find Minimum in Rotated Sorted Array

Problem at a glance

Difficulty	Approach	Main tool
Medium	Binary search	Array

The Problem

A strictly increasing array has been rotated. Return its minimum value in $O(\log n)$ time. All values are distinct.

Example

`[4,5,6,7,0,1,2]` has minimum 0. An unrotated array such as `[1,2,3]` has minimum 1.

Binary-Search Decision

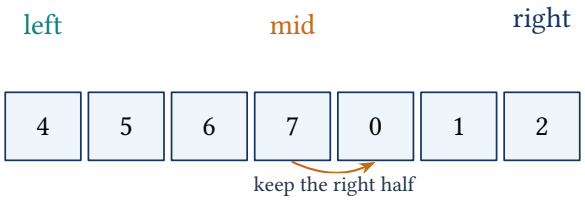
Compare the midpoint with the right endpoint:

- If $nums[mid] > nums[right]$, the rotation point is strictly right of mid, so set $left = mid + 1$.
- Otherwise, mid may itself be the minimum, so set $right = mid$.

Key idea: the closed interval always contains the rotation minimum

The comparison with the right endpoint identifies a half that cannot contain the minimum. The algorithm removes only that half, retaining `mid` when it may itself be the answer, until both boundaries meet at the minimum.

Because $7 > 2$, the minimum must be to the right



Pseudocode

```
left = 0
right = nums.length - 1
while left < right:
    middle = left + (right - left) / 2
    if nums[middle] > nums[right]:
        left = middle + 1
    else:
        right = middle
return nums[left]
```

Code 21: Complete C++ solution: Find Minimum in Rotated Sorted Array

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    int findMin(vector<int>& nums) {
        int left = 0;
        int right = static_cast<int>(nums.size()) - 1;
        while (left < right) {
            const int middle = left + (right - left) / 2;
            if (nums[middle] > nums[right]) left = middle + 1;
            else right = middle;
        }
        return nums[left];
    }
};
```

Why It Works, Complexity, and Common Mistakes

The search interval always contains the minimum. Each comparison safely discards one half while retaining mid when it may be the answer. Interval size halves each iteration: $O(\log n)$ time and $O(1)$ space. Use $right = mid$, not $mid - 1$, in the second branch.

From Linear Scan to Binary Search

A linear scan can return the smallest value in $O(n)$, but the rotated sorted structure allows a logarithmic decision. Comparing the midpoint with the right endpoint reveals whether the midpoint lies in the high rotated portion or in the low portion containing the minimum.

Boundary Dry Run

Searching `[4,5,6,7,0,1,2]`

Left	Mid	Right	Comparison	New interval
0 (4)	3 (7)	6 (2)	$7 > 2$	$[4, 6]$
4 (0)	5 (1)	6 (2)	$1 \leq 2$	$[4, 5]$
4 (0)	4 (0)	5 (1)	$0 \leq 1$	$[4, 4]$

The loop stops when both boundaries identify the minimum value 0.

Why Compare with the Right Endpoint?

If $nums[mid] > nums[right]$, the sorted order breaks somewhere strictly to the right of mid, so mid cannot be the minimum. Otherwise mid belongs to the low sorted portion and may itself be the minimum, which is why the assignment is `right = middle` rather than `middle - 1`.

Edge Cases and Variations

A one-element or unrotated array works without a special branch. The proof uses distinct values. If duplicates are allowed, equality may not identify a safe half; a common adaptation decrements the right boundary on equality, which can degrade to $O(n)$. Compute the midpoint as `left + (right-left)/2` to avoid integer overflow.

Practice checkpoint

Trace the algorithm on `[2,3,4,5,1]` and `[1,2,3,4]`. At every step state why the discarded half cannot contain the minimum.

UNDIRECTED GRAPHS

Lesson 22: Graph Valid Tree

Problem at a glance

Difficulty	Approach	Main tool
Medium	DFS cycle and connectivity check	Adjacency list

The Problem

Given n labeled vertices and undirected edges, return whether the graph is a tree. A tree must be connected and contain no cycle.

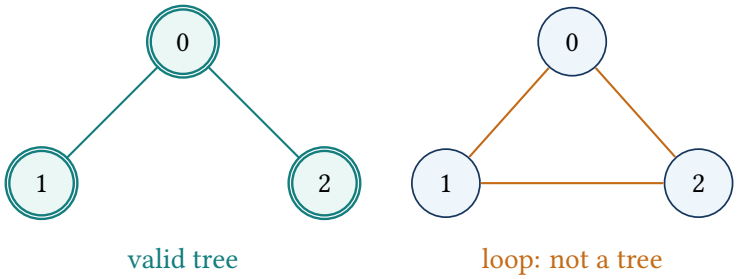
Two Conditions

An undirected tree with n vertices has exactly $n - 1$ edges. That count is a quick rejection. Then DFS from vertex zero, ignoring the edge back to the parent. Reaching any other visited neighbor proves a cycle. Finally, every vertex must have been reached.

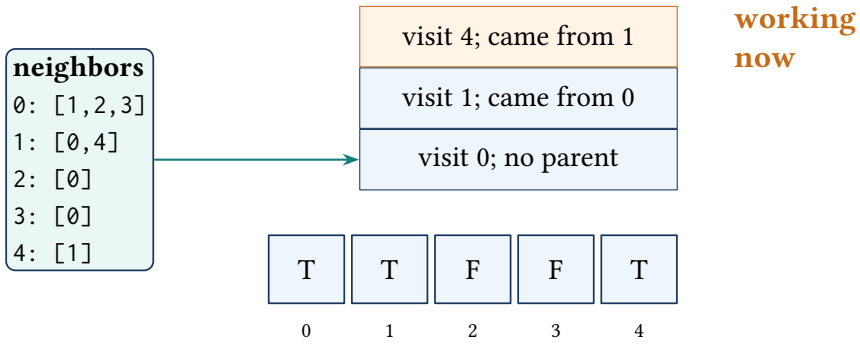
Key idea: visited nodes form the explored part of one rooted traversal

Every recursive frame records the edge used to enter its node as the parent edge. That one reverse edge is expected in an undirected graph and is skipped; any other visited neighbor closes a cycle. After DFS, full visitation proves that no disconnected component was omitted.

A tree is connected and has no loop



Neighbor lists, visited marks, and the path being checked



The list shows connected nodes, the stack remembers how we arrived, and the marks show which nodes were already visited.

Pseudocode

```
if edge count is not n - 1: return false
build an undirected adjacency list
dfs(node, parent):
    if node is already visited: return false
    mark node visited
    for neighbor in graph[node]:
        skip parent
        if dfs(neighbor, node) is false: return false
    return true
run dfs from node 0
return dfs succeeded and every node was visited
```

Code 22: Complete C++ solution: Graph Valid Tree

```cpp
#include <vector>

using namespace std;

class Solution {
    bool dfs(int node, int parent,
             const vector<vector<int>>& graph,
             vector<bool>& visited) {
        if (visited[node]) return false;
        visited[node] = true;
        for (const int neighbor : graph[node]) {
            if (neighbor == parent) continue;
            if (!dfs(neighbor, node, graph, visited)) return false;
        }
        return true;
    }
```

```cpp

public:
    bool validTree(int n, vector<vector<int>>& edges) {
        if (static_cast<int>(edges.size()) != n - 1) return false;
        vector<vector<int>> graph(n);
        for (const auto& edge : edges) {
            graph[edge[0]].push_back(edge[1]);
            graph[edge[1]].push_back(edge[0]);
        }
        vector<bool> visited(n, false);
        if (!dfs(0, -1, graph, visited)) return false;
        for (const bool reached : visited) if (!reached) return false;
        return true;
    }
};
```

Why It Works and Complexity

The parent check prevents treating the same undirected edge as a cycle. Any other repeated vertex closes a cycle. The final visited check proves connectivity. DFS and adjacency construction take $O(V + E)$ time and space.

How to explain it in an interview

A tree is connected and acyclic. I first verify the needed $n - 1$ edge count, then DFS while passing the parent. A visited non-parent neighbor is a cycle; any unvisited vertex afterward means the graph is disconnected.

Why Both Conditions Matter

A connected graph may still contain a cycle, and an acyclic graph may be a disconnected forest. So checking only one property is inenough. The $n - 1$ edge count is a fast needed condition, while DFS verifies the actual structure.

DFS Dry Run

Edges `[[0,1],[0,2],[0,3],[1,4]]`

Current	Parent	Action
0	-1	mark 0; recurse to 1, 2, and 3
1	0	ignore edge back to 0; recurse to 4
4	1	ignore edge back to 1; return true
2, 3	0	each is new and has only its parent edge
finish	–	all five vertices are visited; return true

An extra edge $[1, 2]$ would lead from one DFS branch to an already visited non-parent vertex and prove a cycle.

Code Walkthrough

The edge-count guard rejects obvious non-trees before allocating the graph. Each undirected edge is stored in both adjacency lists. DFS marks a node, skips exactly the edge to its parent, and rejects any other repeated node. The final loop confirms that vertex zero's traversal reached every component.

Alternative and Edge Cases

Union find is a high-value alternative: union every edge and reject an edge whose endpoints already share a representative, then confirm $n - 1$ edges. For $n = 1$, an empty edge list is a valid tree. Self-loops and parallel edges create cycles. The recursive DFS may use $O(n)$ stack space on a chain.

Practice checkpoint

Explain why the parent edge is skipped but any other visited neighbor is a cycle. Then outline the union-find version and compare its state with DFS.

ARRAYS AND HASHING

Lesson 23: Group Anagrams

Problem at a glance

Difficulty	Approach	Main tool
Medium	Standard-key hashing	Hash map

The Problem

Group strings that contain the same letters with the same frequencies. The groups and strings within them may be returned in any order.

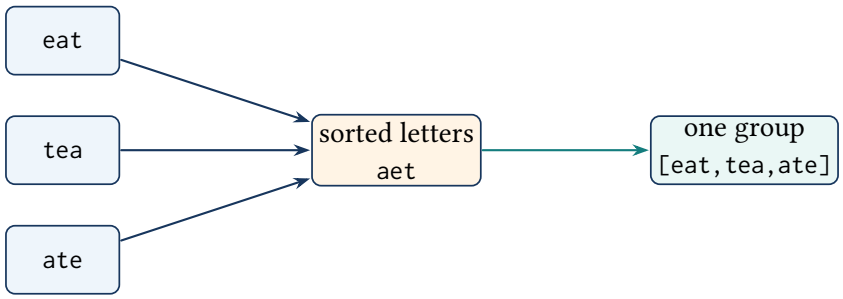
Sorted Standard Key

Anagrams become identical after sorting their characters. Use the sorted word as a hash-map key and append the original word to that bucket.

Key idea: each processed word is stored in its unique standard bucket

After processing a prefix of the input, every word from that prefix appears in the bucket indexed by its sorted characters. Two words share a bucket exactly when their character multisets match, so no later comparison between words is needed.

Sort each word to find its group



Pseudocode

```
groups = empty map from sorted key to words
for word in strings:
    key = characters of word sorted
    append word to groups[key]
return all map buckets
```

Code 23: Complete C++ solution: Group Anagrams

```cpp
#include <algorithm>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>

using namespace std;

class Solution {
public:
    vector<vector<string>>
    groupAnagrams(vector<string>& strings) {
        unordered_map<string, vector<string>> groups;
        for (const string& word : strings) {
            string key = word;
            sort(key.begin(), key.end());
            groups[key].push_back(word);
        }

        vector<vector<string>> result;
        result.reserve(groups.size());
        for (auto& entry : groups) result.push_back(move(entry.second));
        return result;
    }
};
```

Complexity and Alternatives

For n strings of maximum length k, sorting keys costs $O(nk \log k)$; stored keys and output use $O(nk)$ space. A 26-count frequency signature avoids sorting and gives $O(nk)$ time, but the sorted key is simpler and easy to explain.

Common mistakes

Append the original word, not the sorted key. Empty strings correctly share an empty key. Do not assume the order of buckets from an unordered map.

From Pairwise Comparison to a Standard Form

A direct method can compare every pair of strings by character frequencies, which costs roughly $O(n^2 k)$. The repeated work is deciding the same anagram identity again and again. A standard key gives every anagram in a group one identical representation, so one hash lookup finds its bucket.

Detailed Grouping Walkthrough

Worked example

Word	Sorted key	Bucket after insertion
`eat`	`aet`	`[eat]`
`tea`	`aet`	`[eat,tea]`
`tan`	`ant`	`[tan]`
`ate`	`aet`	`[eat,tea,ate]`
`nat`	`ant`	`[tan,nat]`
`bat`	`abt`	`[bat]`

Code Walkthrough

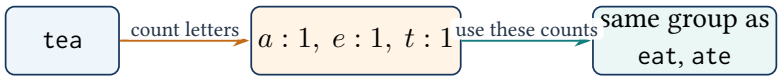
For each original word, the code makes one copy named `key`, sorts that copy, and appends the untouched word to `groups[key]`. After all words are classified, each map value is moved into the returned vector. Moving avoids copying every string bucket again.

Complexity Details, Edge Cases, and Alternative

If string lengths differ, a more precise sorting bound is $O(\sum_i k_i \log k_i)$. The map stores standard keys in addition to the returned strings. Empty strings all use the empty key, and repeated identical words remain repeated in the same group. Output order is unspecified because `unordered_map` iteration order is unspecified.

For lowercase English letters, a 26-element frequency signature avoids sorting. It runs in $O(\sum_i k_i)$ time but needs a collision-free way to serialize counts, such as separators between decimal values.

Count the letters instead of sorting



How to explain it simply

Anagrams become equal after sorting. I use that sorted form as a hash key and append the original word to its bucket. This converts repeated pairwise comparisons into one key construction and average constant-time lookup per word.

HOUSE ROBBER DP

Lesson 24: House Robber

Problem at a glance

Difficulty	Approach	Main tool
Medium	Rolling dynamic programming	Two running values

The Problem

Houses lie on a line. Return the maximum amount that can be robbed without choosing two adjacent houses.

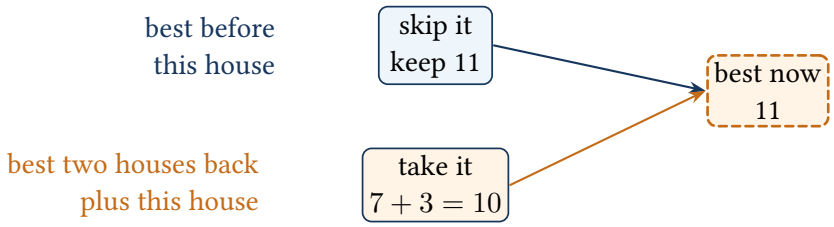
From DP Array to Two Variables

At each house, choose between skipping it and taking it with the best result from two houses earlier: $current = \max(previous, twoBack + money)$. Only the previous two states are needed, which is the space-optimized approach used in the displayed implementation.

Key idea: the two variables are the last two best prefixes

Before processing a house, `previous` is the optimum for all houses already seen and `twoBack` is the optimum before the immediately previous house. Computing the new value before shifting keeps both choices required by the update rule.

Compare skipping this house with taking it



After the decision, `previous` becomes the new answer and the old `previous` shifts into `twoBack` for the next house.

Rolling states for `[2,7,9,3,1]`

House value	2	7	9	3	1
Best so far	2	7	11	11	12

Pseudocode

```
twoBack = 0
previous = 0
for money in houses:
    current = max(previous, twoBack + money)
    twoBack = previous
    previous = current
return previous
```

Code 24: Complete C++ solution: House Robber

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    int rob(vector<int>& nums) {
        int twoBack = 0;
        int previous = 0;
        for (const int money : nums) {
            const int current = max(previous, twoBack + money);
            twoBack = previous;
            previous = current;
        }
        return previous;
    }
};
```

Why the Solution Works, Complexity, and Revision

The update rule considers both possibilities for every best prefix: the last house is skipped or included. Updating `twoBack` only after computing `current` keeps the old states. Time is $O(n)$, extra space is $O(1)$, and empty input naturally returns zero.

House Robber pattern

State: best answer for a prefix. **Choice:** skip current or take current plus the answer two positions back. **Signal:** adjacent choices cannot both be selected.

From Subsets to Dynamic Programming

Brute force chooses or skips every house, producing up to 2^n subsets and then rejecting those with adjacent selections. The useful fact is that the best decision for the first i houses depends only on the best answers for the first $i - 1$ and $i - 2$ houses.

For the current house, every best solution is in exactly one group:

- skip it and keep the best previous result;
- take it, which forces the previous house to be skipped, and add its money to the best result two positions back.

Detailed Rolling-State Trace

Houses `[2,7,9,3,1]`

Money	Two back	Previous	Take current	New best
2	0	0	2	2
7	0	2	7	7
9	2	7	11	11
3	7	11	10	11
1	11	11	12	12

Code Walkthrough

Before each iteration, `previous` is the best answer for the houses already processed and `twoBack` is the answer one prefix earlier. The code calculates `current` before overwriting either old value. It then shifts the states forward. This keeps the update rule without allocating a full DP array.

Edge Cases and Interview Variations

Empty input returns zero, one house returns its value, and two houses return the larger value. The standard problem uses non-negative money. If negative values were allowed and robbing no house remained valid, the zero initialization would correctly ignore them. Variations include circular houses, reconstructing which houses were chosen, or a minimum gap larger than one between selected positions.

How to explain it simply

At each house I either skip it and keep the previous optimum, or take it and add its money to the optimum from two houses back. Only those two earlier states are needed, so the solution is $O(n)$ time and $O(1)$ extra space.

HOUSE ROBBER DP

Lesson 25: House Robber II

Problem at a glance

Difficulty	Approach	Main tool
Medium	Two linear DP scans	Dynamic programming

The Problem

Houses form a circle, so the first and last houses are adjacent. Maximize the money robbed without choosing adjacent houses.

From a line to a circle

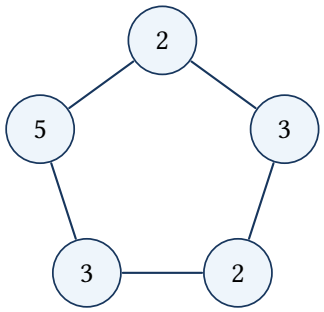
The circular problem uses the same take-or-skip update rule as a linear row of houses. Its only additional restriction is that the first and last houses cannot both be selected.

Break the Circle

No valid solution can contain both the first and last houses. So, solve two ordinary linear cases and take the larger result:

1. houses 0 through $n-2$, excluding the last;
2. houses 1 through $n-1$, excluding the first.

Break the circle into two straight rows

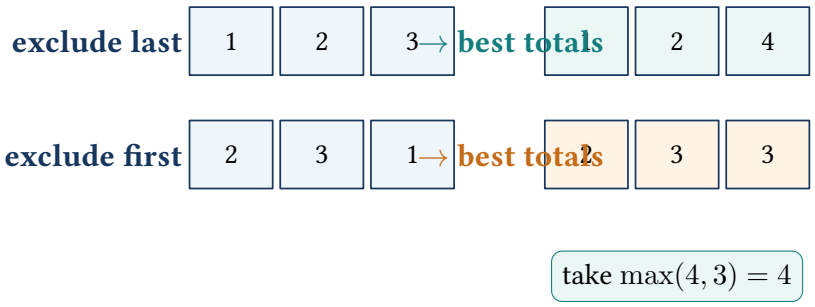


row A: skip the first house
row B: skip the last house

Linear DP Update Rule

For a range beginning at s, let $dp[i]$ be the best value using its first i houses. Then $dp[i] = \max(dp[i-1], dp[i-2] + \text{current house})$.

Solve each straight row separately



Pseudocode

```
robRange(start, end):
    dp[0] = 0
    dp[1] = nums[start]
    for each later house in the range:
        dp[i] = max(dp[i - 1], dp[i - 2] + houseValue)
    return final dp value

if there is only one house: return its money
return max(robRange(0, n - 2), robRange(1, n - 1))
```

Code 25: Complete C++ solution: House Robber II

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
    int robRange(const vector<int>& nums, int start, int end) {
        const int length = end - start + 1;
        vector<int> dp(length + 1, 0);
        dp[1] = nums[start];
        for (int i = 2; i <= length; ++i) {
            const int value = nums[start + i - 1];
            dp[i] = max(dp[i - 1], dp[i - 2] + value);
        }
        return dp[length];
    }

public:
    int rob(vector<int>& nums) {
        const int n = static_cast<int>(nums.size());
```

```
        if (n == 0) return 0;
        if (n == 1) return nums[0];
        return max(robRange(nums, 0, n - 2),
                   robRange(nums, 1, n - 1));
    }
};
```

Complexity and Space-Optimized Follow-up

Both scans are linear, so time is $O(n)$. The DP arrays use $O(n)$ space. Each scan can instead keep only the previous two states, reducing extra space to $O(1)$.

How to explain it in an interview

Because the first and last houses conflict, I solve two linear robber problems: exclude the first or exclude the last. The linear update rule chooses between skipping the current house and taking it with the best value two positions back.

Why Two Linear Cases Are Complete

No legal robbery can contain both endpoint houses because they are adjacent in the circle. Every legal solution belongs to at least one of two sets: solutions that exclude the first house or solutions that exclude the last. Taking the better optimum from those two sets covers every possibility.

Key idea: each helper call solves one complete legal linear case

Within a chosen range, the DP value after each house is the best legal robbery of that processed prefix. The two ranges together cover every circular solution because no legal solution can contain both endpoint houses.

Detailed Visual Dry Run

Houses `[1,2,3,1]`

Linear case	Range	DP values	Best choice
Exclude last	`[1,2,3]`	$1, 2, 4$	rob houses with 1 and 3
Exclude first	`[2,3,1]`	$2, 3, 3$	rob the middle value 3
Final	–	$\max(4, 3)$	return 4

For `[2,3,2]`, both endpoint values 2 cannot be taken together, so the middle house gives the answer 3.

Code Walkthrough

`robRange` creates a DP table for one inclusive interval. The first entry is the money in the first allowed house. Every later entry chooses between the previous optimum and the current house plus the optimum two positions earlier. The public method handles zero and one house before calling the helper on the two non-empty ranges.

Edge Cases, Space Optimization, and Follow-ups

The special case for one house prevents both ranges from becoming empty. Two houses return the larger value. Zero-valued houses work naturally. Each range can use the two-variable rolling state from House Robber, reducing extra space from $O(n)$ to $O(1)$ while preserving $O(n)$ time.

Practice checkpoint

Write down the two ranges for five houses. Prove that a legal solution that chooses neither endpoint is still considered by both ranges and so is not lost.

ARRAYS, STRINGS, AND GREEDY

Lesson 26: Maximum Subarray

Problem at a glance

Difficulty	Approach	Main tool
Medium	Kadane's algorithm	Array and running sum

The Problem

Given an integer array, return the largest sum of a non-empty consecutive subarray. The chosen values must occupy consecutive positions.

Example

For `[-2,1,-3,4,-1,2,1,-5,4]`, the best subarray is `[4,-1,2,1]`, whose sum is 6.

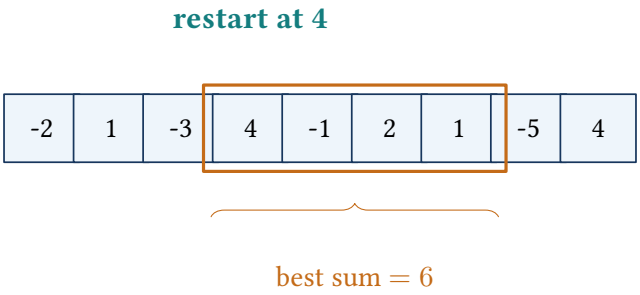
From Repeated Sums to One Scan

Trying every pair of boundaries repeats almost the same additions. At each position, only one question matters: is the previous running subarray useful, or is it better to start again at the current value? A negative prefix can only reduce every future extension, so it is discarded.

Key idea: the running sum is the best subarray ending here

After processing index i, `endingHere` is the maximum sum of a non-empty subarray whose right endpoint is exactly i. The global answer is the best value of `endingHere` seen at any endpoint.

Keep a useful prefix or restart



Pseudocode

```
endingHere = answer = nums[0]
for each value after the first:
    endingHere = max(value, endingHere + value)
    answer = max(answer, endingHere)
return answer
```

Code 26: Complete solution: Maximum Subarray

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int endingHere = nums[0];
        int answer = nums[0];
        for (int i = 1; i < static_cast<int>(nums.size()); ++i) {
            endingHere = max(nums[i], endingHere + nums[i]);
            answer = max(answer, endingHere);
        }
        return answer;
    }
};
```

Dry Run, Why the Solution Works, and Complexity

The running sums are $-2, 1, -2, 4, 3, 5, 6, 1, 5$. The maximum is 6. The update rule considers both possible forms of every best subarray ending at a position: start there or extend the best one ending immediately before. Time is $O(n)$, and extra space is $O(1)$. Start from the first value rather than zero so arrays containing only negative values work.

Walk Through the Example

Trace the example one update at a time

For `[-2,1,-3,4,-1,2,1,-5,4]`, the ending sums are $-2, 1, -2, 4, 3, 5, 6, 1, 5$. The restart at 4 discards the harmful negative prefix, and the global best becomes 6 after processing the following 1.

Why It Works

Every non-empty subarray ending at index i either begins at i or extends a subarray ending at $i-1$. Choosing the larger of exactly these two possibilities proves the update rule; taking the maximum over all endpoints proves the final answer.

Tests to try

Test one negative value, all-negative input, all-positive input, zeros, and an optimum that begins at index zero or ends at the final index.

How to explain it in an interview

I keep the best sum ending at the current position. A negative accumulated prefix is abandoned by comparing extension with a fresh start. The best ending value seen anywhere is the answer, in linear time and constant space.

A Closer Look

A useful way to begin is to ask: what does the current sum mean at one exact index? It is not the best sum anywhere in the array. It is the best sum of a subarray that must end at the current value. This difference is what makes one pass possible. If the old running sum is negative, carrying it forward only makes the next choice worse, so the scan starts again. If it is positive, adding the new value may continue a useful block.

For `[-2,1,-3,4,-1,2,1,-5,4]`, the running value is first -2. At 1, starting again is better than extending -2, so the running value becomes 1. The same decision happens again at 4 because the sum before it is negative. From 4 onward, the values $-1, 2, 1$ still leave a positive running sum, so that block is kept. A separate variable stores the best running value ever seen. Initializing both variables from the first element, rather than zero, is important when every array value is negative.

ARRAYS, STRINGS, AND GREEDY

Lesson 27: Maximum Product Subarray

Problem at a glance

Difficulty	Approach	Main tool
Medium	Dynamic running extremes	Array

The Problem

Return the largest product of a non-empty consecutive subarray. Values may be positive, negative, or zero.

Why the minimum matters

For `[-2,3,-4]`, the negative product -6 later becomes the best positive product 24 when multiplied by -4.

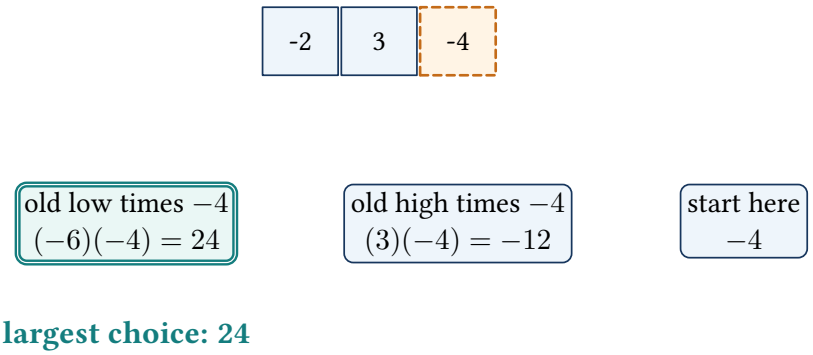
Building the State

For sums, one running maximum is enough. Products are different because a negative multiplier swaps large and small values. So each position keeps both the maximum and minimum product ending there.

Key idea: both product extremes end at the current position

After index i, `high` and `low` are respectively the largest and smallest products of subarrays ending exactly at i. Their three choices are the current value alone, the old high times the value, and the old low times the value.

The old low becomes the new high



Pseudocode and C++

```
low = high = answer = nums[0]
for value in nums[1...]:
    oldHigh = high; oldLow = low
    high = max(value, oldHigh*value, oldLow*value)
    low  = min(value, oldHigh*value, oldLow*value)
    answer = max(answer, high)
```

Code 27: Complete solution: Maximum Product Subarray

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int high = nums[0], low = nums[0], answer = nums[0];
        for (int i = 1; i < static_cast<int>(nums.size()); ++i) {
            const int oldHigh = high, oldLow = low, value = nums[i];
            high = max({value, oldHigh * value, oldLow * value});
            low = min({value, oldHigh * value, oldLow * value});
            answer = max(answer, high);
        }
        return answer;
    }
};
```

Time, Space, and Edge Cases

Each value performs constant work, so time is $O(n)$ and space is $O(1)$. Zero automatically starts a new product. Save both old extremes before either is overwritten. Test one value, all negatives, zeros between useful segments, and an odd or even count of negative values.

Walk Through the Example

Trace the example one update at a time

For `[-2,3,-4]`, start with $high = low = -2$. At value 3, the choices are $3, -6, -6$, so $high = 3$ and $low = -6$. At value -4, the choices are $-4, -12, 24$; the old minimum becomes the new maximum, producing the answer 24.

Why It Works

Every product ending at the current index either starts at that value or extends a product ending at the previous index. Multiplication by a negative value can exchange the maximum and minimum, which is why keeping both extremes is enough and needed.

Tests to try

Check `[-2]`, `[0,2]`, `[-2,0,-1]`, `[-2,3,-4]`, and a sequence containing two separated zeroes.

How to explain it in an interview

I track both the largest and smallest product ending at each position. The smallest is retained because a later negative number can turn it into the largest. Each value has three choices, giving linear time and constant extra space.

A Closer Look

Products need one more piece of state than sums. A very small negative product can become a very large positive product after another negative number. For that reason, the scan keeps both the largest and smallest product ending at the current position. Neither one can be discarded early.

At every value there are three choices: start a new subarray with this value, multiply it by the previous maximum, or multiply it by the previous minimum. The largest candidate becomes the new maximum and the smallest becomes the new minimum. With `[-2,3,-4]`, the states after 3 are maximum 3 and minimum -6. When -4 arrives, $(-6)(-4) = 24$, so the old minimum creates the new maximum. Zero also works naturally: it can end the old block and start a new one. Both old states must be saved before either is updated, otherwise the second calculation may use a value from the current step.

ARRAYS, STRINGS, AND GREEDY

Lesson 28: Longest Repeating Character Replacement

Problem at a glance

Difficulty	Approach	Main tool
Medium	Sliding window	Frequency array

The Problem

Given an uppercase string and an integer k, return the longest substring that can be made of one repeated character after replacing at most k characters.

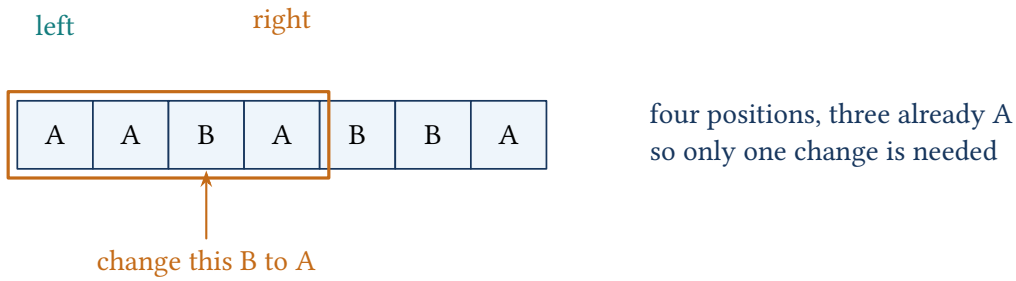
Window Condition

Inside a window, keep the most frequent character and replace every other position. A window is valid exactly when window length $-$ maximum frequency $\leq k$. When this cost becomes too large, move the left boundary.

Key idea: the active window needs at most k replacements

After shrinking, the window satisfies the replacement budget. The frequency array describes its characters, while `maxFrequency` is a safe nondecreasing upper bound used to decide when the left boundary must move.

Change one B to make four A's



Pseudocode and C++

```
left = 0; maxFrequency = 0; answer = 0
for right from 0 to n-1:
    increment count[s[right]]
    update maxFrequency
    while windowLength - maxFrequency > k:
        decrement count[s[left++]]
    update answer with windowLength
```

Code 28: Complete solution: Character Replacement

```cpp
#include <algorithm>
#include <array>
#include <string>

using namespace std;

class Solution {
public:
    int characterReplacement(string s, int k) {
        array<int, 26> count{};
        int left = 0, maxFrequency = 0, answer = 0;
        for (int right = 0; right < static_cast<int>(s.size()); ++right) {
            maxFrequency = max(maxFrequency, ++count[s[right] - 'A']);
            while (right - left + 1 - maxFrequency > k) {
                --count[s[left] - 'A'];
                ++left;
            }
            answer = max(answer, right - left + 1);
        }
        return answer;
    }
};
```

Why a Stale Maximum Is Safe

The stored maximum frequency need not decrease when the left side moves. A stale larger value may delay shrinking, but it cannot create a new answer longer than a window that was already achievable when that frequency was observed. Both boundaries move only forward: $O(n)$ time and $O(1)$ space.

Walk Through the Example

Trace the example one update at a time

For AABABBA with $k = 1$, the window AABA has length four and maximum frequency three, so it needs one replacement. Extending to AABAB needs two replacements; move the left boundary until the window again satisfies $length - maxFrequency \leq 1$. The best length remains four.

Why It Works

For a fixed window, changing every other character is best, so the required replacements equal its length minus its largest frequency. The algorithm records every maximal valid window reached by the right boundary.

Tests to try

Check "A", $k = 0$; all-identical text; alternating characters; k at least the string length; and a best window ending at the final character.

How to explain it in an interview

I use a variable sliding window. Character counts identify the dominant letter, and I shrink only when the replacement cost exceeds k. Since both pointers move forward, the running time is linear.

A Closer Look

The window does not need every character to match already. It only needs to be repairable with at most k replacements. Inside a window of length L, keep the character that appears most often and replace the other $L - f$ positions. The window is allowed exactly when $L - f \leq k$.

As the right edge moves, update the count for the new character and the largest frequency seen. If the replacement cost is too large, move the left edge and remove those characters from the active counts. The stored maximum frequency is allowed to be slightly old. It may delay a shrink, but it cannot produce a better answer that was never possible earlier. For AABABBA with $k = 1$, AABA is valid because three of its four positions are A. Extending farther breaks the budget, so the left edge moves. Each edge moves only forward, which is why the work is linear.

ARRAYS, STRINGS, AND GREEDY

Lesson 29: Longest Palindromic Substring

Problem at a glance

Difficulty	Approach	Main tool
Medium	Expand around centers	String boundaries

The Problem

Return the longest consecutive substring that reads the same from left to right and right to left.

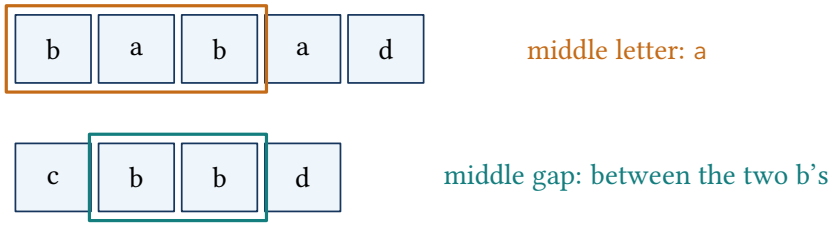
Every Palindrome Has a Center

There are n odd-length centers on characters and $n-1$ even-length centers between characters. Expand outward while the two characters match; every palindrome is discovered from its unique center.

Key idea: the characters strictly inside the pointers match

During an expansion, the substring between `left+1` and `right-1` is already a palindrome. Equal boundary characters extend it by two; the first mismatch proves that center cannot grow further.

A palindrome grows from a letter or a gap



Pseudocode

```
bestStart = 0; bestLength = 1
for center from 0 to n-1:
    expand(center, center)       // odd length
    expand(center, center + 1)   // even length
expand(left, right):
    while boundaries match: move both outward
    update the longest valid range just crossed
return the recorded substring
```

Code 29: Complete solution: Longest Palindromic Substring

```cpp
#include <string>

using namespace std;

class Solution {
public:
    string longestPalindrome(string s) {
        int bestStart = 0, bestLength = 1;
        auto expand = [&](int left, int right) {
            while (left >= 0 && right < static_cast<int>(s.size()) &&
                   s[left] == s[right]) {
                --left;
                ++right;
            }
            const int length = right - left - 1;
            if (length > bestLength) {
                bestLength = length;
                bestStart = left + 1;
            }
        };

        for (int center = 0; center < static_cast<int>(s.size()); ++center) {
            expand(center, center);
            expand(center, center + 1);
        }
        return s.substr(bestStart, bestLength);
    }
};
```

Dry Run and Complexity

For babad, expanding around index 1 finds bab; expanding around index 2 finds aba. Either maximum is valid. There are $2n - 1$ centers and each may expand $O(n)$, so time is $O(n^2)$ and extra space is $O(1)$.

Walk Through the Example

Trace the example one update at a time

For `cbbd`, the character centers produce only length-one palindromes. The gap between the two `b` characters expands once to form `bb`; the next outward comparison `c` versus `d` fails, so `bb` is recorded.

Why It Works

Every palindrome has one unique center: a character for odd length or a gap for even length. Expansion enumerates every palindrome belonging to that center, so testing all $2n - 1$ centers cannot miss the optimum.

Tests to try

Check empty handling if the interface permits it, one character, two equal characters, two unequal characters, an even optimum, and tied optima.

How to explain it in an interview

I expand from every character and every adjacent-character gap. Matching boundaries extend a valid palindrome; after the first mismatch I compare the last valid length with the best recorded range.

A Closer Look

A palindrome grows from its middle. An odd-length palindrome has one middle character, while an even-length palindrome has a gap between two characters. Those are the only two kinds of centers, so testing both at every position finds every possible palindrome.

For each center, place one pointer on the left and one on the right. While the indexes are valid and the characters match, record the current length and move both pointers outward. In `babad`, the center at the first `a` expands to `bab`; the next center can produce `aba`. Either length-three answer is valid. The important boundary detail is that the pointers move one step beyond the palindrome before the loop stops, so the found substring begins at `left+1` and has length `right-left-1`. Empty input and a two-character even palindrome are good checks for the index arithmetic.

ARRAYS, STRINGS, AND GREEDY

Lesson 30: Longest Substring Without Repeating Characters

Problem at a glance

Difficulty	Approach	Main tool
Medium	Sliding window	Last-position table

The Problem

Return the length of the longest substring containing no repeated character.

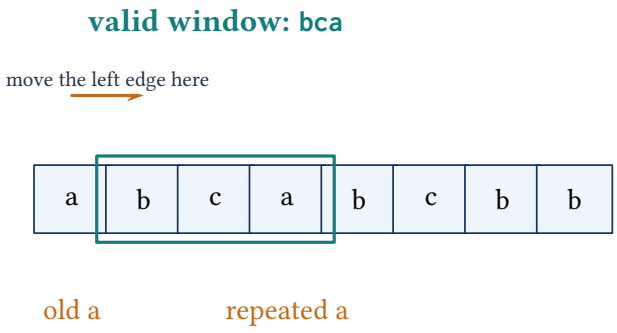
Jump the Left Boundary

When the right character was already seen inside the active window, the next valid window must begin after its previous occurrence. A last-position table lets the left boundary jump directly rather than removing characters one by one.

Key idea: the active window contains unique characters

Before recording the answer at each right endpoint, `left` is greater than the previous position of every duplicated character in the window. So `s[left..right]` contains no repeated character.

A repeated a moves the left edge past the old a



Pseudocode

```
last position of every character = -1
left = 0; answer = 0
for right from 0 to n-1:
    left = max(left, last[s[right]] + 1)
    last[s[right]] = right
    answer = max(answer, right - left + 1)
return answer
```

Code 30: Complete solution: Longest Substring Without Repeats

```cpp
#include <algorithm>
#include <array>
#include <string>

using namespace std;

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        array<int, 256> last;
        last.fill(-1);
        int left = 0, answer = 0;
        for (int right = 0; right < static_cast<int>(s.size()); ++right) {
            const unsigned char ch = static_cast<unsigned char>(s[right]);
            left = max(left, last[ch] + 1);
            last[ch] = right;
            answer = max(answer, right - left + 1);
        }
        return answer;
    }
};
```

Why It Works and Complexity

The left boundary never moves backward, even if the character's last occurrence lies before the current window. Each character is processed once, so time is $O(n)$ and the fixed table uses $O(1)$ space for byte-valued characters. Empty input correctly returns zero.

Walk Through the Example

Trace the example one update at a time

For `abba`, the second `b` moves `left` from 0 to 2, leaving `b`. At the final `a`, its previous position 0 lies outside the current window, so `left` stays 2 and the valid window becomes `ba`. The earlier window `ab` remains the best.

Why It Works

The left boundary is always one position beyond the most recent duplicate that lies inside the active window. So every recorded window is unique, and every longest unique suffix ending at each right boundary is considered.

Tests to try

Test an empty string, all identical characters, all unique characters, repeated characters outside the current window, and byte values with a negative plain-`char` representation.

How to explain it in an interview

I store the latest index of each character. When a repeat appears, the left boundary jumps forward past its previous copy but never moves backward. Each character is processed once, so the algorithm is linear.

A Closer Look

The active substring must contain no repeated character. A direct solution can restart from every index, but it repeatedly examines the same characters. The sliding window keeps the longest useful suffix instead. A table records the most recent position of each character.

When the right pointer sees a character that already appears inside the window, the left pointer jumps to one position after that old occurrence. It must never move backward, so the update uses the larger of the current left edge and the stored position plus one. In `abba`, seeing the second `b` moves left past the first `b`. When the final `a` is read, its old position lies outside the current window, so left stays where it is. This is the case that breaks solutions that assign `left=last[a]+1` without a maximum. The answer is updated after the window is valid.

ARRAYS, STRINGS, AND GREEDY

Lesson 31: Palindromic Substrings

Problem at a glance

Difficulty	Approach	Main tool
Medium	Interval dynamic programming	Two-dimensional DP table

The Problem

Count every palindromic substring. Equal text at different positions counts as different substrings.

Examples and positional counting

`"abc"` contains three palindromic substrings: `"a"`, `"b"`, and `"c"`. The string `"aaa"` contains six: three length-one intervals, two length-two intervals, and one length-three interval. Equal text at different positions counts separately.

From Repeated Checks to Interval DP

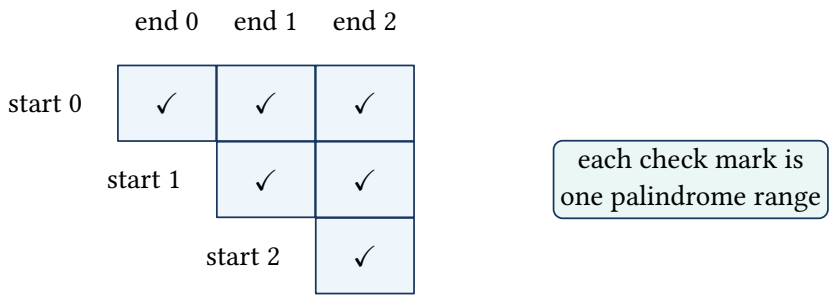
A direct method can generate every substring and scan inward to determine whether it is a palindrome. There are $O(n^2)$ intervals and each check can cost $O(n)$, producing $O(n^3)$ work.

Instead, let $dp[i][j]$ mean that the interval from index i through index j is a palindrome. The outer characters must match, and the interior must already be a palindrome. Intervals of length one and two form the base cases: $dp[i][j] = (s[i] = s[j]) \wedge \big(j - i \leq 1 \ \vee \ dp[i+1][j-1]\big)$.

Key idea: every interior interval is solved before its exterior

The table is filled with i moving from right to left and j moving from i to the right. So $dp[i+1][j-1]$ is known before $dp[i][j]$. Every true table cell corresponds to exactly one pair of substring endpoints and increases the answer once.

Six checked cells mean six palindromes in aaa



Pseudocode

```
count = 0
dp = n by n table filled with false
for left from n - 1 down to 0:
    for right from left to n - 1:
        shortInterior = (right - left <= 1)
        if s[left] == s[right] and
           (shortInterior or dp[left + 1][right - 1]):
            dp[left][right] = true
            count++
return count
```

Code 31: Complete solution: Palindromic Substrings

```cpp
#include <string>
#include <vector>

using namespace std;

class Solution {
public:
    int countSubstrings(string s) {
        const int n = static_cast<int>(s.size());
        vector<vector<bool>> dp(
            n, vector<bool>(n, false));
        int answer = 0;

        for (int left = n - 1; left >= 0; --left) {
            for (int right = left; right < n; ++right) {
                const bool shortInterior = right - left <= 1;
                if (s[left] == s[right] &&
                    (shortInterior || dp[left + 1][right - 1])) {
                    dp[left][right] = true;
                    ++answer;
                }
            }
        }
        return answer;
    }
};
```

Detailed Visual Dry Run

Fill order for aaa

i	j	Reason	Count
2	2	Single character a	1
1	1	Single character a	2
1	2	Equal endpoints and length two	3
0	0	Single character a	4
0	1	Equal endpoints and length two	5
0	2	Equal endpoints; $dp[1][1]$ is true	6

Why It Works, Complexity, and Edge Cases

The update rule is needed because a palindrome requires matching endpoints and a palindromic interior. It is enough for the same reason. Since every interval is represented by one table cell, every palindromic substring is counted exactly once. Time and table space are both $O(n^2)$.

Test one character, all equal characters, no multi-character palindrome, and both odd- and even-length palindromes. The loop order must not be reversed, because the interior state must already be available.

Walk Through the Example

Trace the example one update at a time

For abba, the single-character diagonal is true first. Length-two intervals then identify bb. When the cell for indices $(0, 3)$ is reached, the endpoints are both a and the interior cell for bb is already true, so abba is counted. Each true cell represents one distinct pair of positions.

Why It Works

The update rule is needed: unequal endpoints cannot form a palindrome, and a long interval with matching endpoints still fails if its interior is not palindromic. It is enough because matching endpoints placed around a palindromic interior produce a palindrome. The fill order guarantees that the required interior result is available.

Tests to try

Check an empty string if the surrounding interface permits it, one character, two equal characters, two unequal characters, all-equal input, and strings containing both even- and odd-length palindromes.

How to explain it in an interview

I assign one Boolean state to every substring interval. Matching endpoints form a palindrome when the interval is shorter than three or its interior was already proved palindromic. Filling from right to left resolves that dependency and counts each positional substring once.

A Closer Look

This problem asks for a count rather than one longest answer. Every single character is already a palindrome, and larger palindromes are formed by matching equal ends around a smaller palindrome. A two-dimensional table can store whether each substring $[i, j]$ is palindromic.

The ends match when `s[i]==s[j]`. The inside is automatically valid for substrings of length one or two; for longer substrings, the table entry $[i + 1, j - 1]$ must already be true. That dependency tells us to fill shorter lengths before longer ones. Each true entry adds one to the answer. For `aaa`, the three single letters, two length-two substrings, and the whole string give six. Expanding around centers is another valid approach and uses constant extra space, but the table makes the exact recurrence visible and connects naturally to later interval dynamic-programming problems.

INTERVAL SCHEDULING

Lesson 32: Meeting Rooms

Problem at a glance

Difficulty	Approach	Main tool
Easy	Sort and scan	Intervals

The Problem

Given meeting intervals, determine whether one person can attend all of them. Meetings conflict when their occupied times overlap.

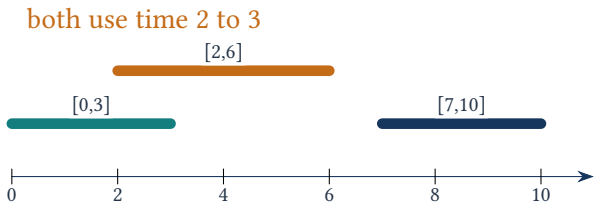
Sort by Start Time

After sorting, any overlap must occur between two adjacent meetings: if the current start is earlier than the previous end, both require the room at the same time.

Key idea: all meetings before the current one are compatible

During the scan, every already checked adjacent pair is non-overlapping. The previous interval has the latest relevant start among them, so comparing its end with the current start is enough.

The first two meetings overlap



Pseudocode

```
sort intervals by start time
for i from 1 to n-1:
    if intervals[i].start < intervals[i-1].end:
        return false
return true
```

Code 32: Complete solution: Meeting Rooms

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    bool canAttendMeetings(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        for (int i = 1; i < static_cast<int>(intervals.size()); ++i) {
            if (intervals[i][0] < intervals[i - 1][1]) return false;
        }
        return true;
    }
};
```

Complexity and Boundary Rule

Sorting costs $O(n \log n)$; the scan is $O(n)$. Adjacent meetings such as $[1, 2]$ and $[2, 3]$ do not overlap, so the comparison is strict <, not <=.

Walk Through the Example

Trace the example one update at a time

Sort `[[7,10],[2,4],[1,3]]` into `[[1,3],[2,4],[7,10]]`. The second meeting starts at 2 before the previous meeting ends at 3, so the schedule immediately fails.

Why It Works

After sorting by start time, if any overlap exists, one also exists between two adjacent intervals in that order. Checking each interval against the preceding end is enough.

Tests to try

Check no meetings, one meeting, touching endpoints, identical intervals, nested intervals, and input already sorted in reverse order.

How to explain it in an interview

I sort by start time so conflicts become adjacent. I then compare each start with the preceding end; a strict earlier start means overlap.

A Closer Look

Only one question matters after sorting: does the next meeting begin before the current one ends? If it does, one room cannot hold both meetings. If it does not, the current room becomes free in time.

Sorting by start time puts the meetings in the order in which they must be considered. For `[[0,30],[5,10],[15,20]]`, the meeting beginning at 5 overlaps the one ending at 30, so the answer is false. For `[[7,10],[2,4]]`, sorting gives $[2, 4]$ then $[7, 10]$, and there is no overlap. Touching endpoints such as $[1, 3]$ and $[3, 5]$ are allowed because one meeting ends exactly when the next begins. The sort costs $O(n \log n)$; after that, the scan only compares neighboring intervals.

GREEDY REACHABILITY

Lesson 33: Jump Game

Problem at a glance

Difficulty	Approach	Main tool
Medium	Greedy reachability	Array

The Problem

Each array value is the maximum jump length from that position. Return whether the final index is reachable from index zero.

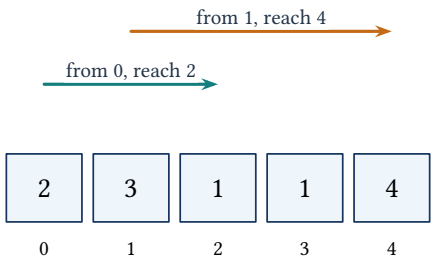
Keep the Reachable Frontier

If index i is within the farthest reachable position, then every landing position up to $i +$ `nums[i]` also becomes reachable. If the scan ever passes the frontier, no future position can be used.

Key idea: every index through farthest is reachable

Before examining index i, every position from zero through `farthest` can be reached. When $i \leq$ `farthest`, processing its jump can only extend this consecutive reachable prefix.

Reach index 2, then extend the reach to index 4



Pseudocode

```
farthest = 0
for i from 0 to n-1:
    if i > farthest: return false
    farthest = max(farthest, i + nums[i])
    if farthest >= n - 1: return true
return true
```

Code 33: Complete solution: Jump Game

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    bool canJump(vector<int>& nums) {
        int farthest = 0;
        for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
            if (i > farthest) return false;
            farthest = max(farthest, i + nums[i]);
            if (farthest >= static_cast<int>(nums.size()) - 1) return true;
        }
        return true;
    }
};
```

Complexity and Mistakes

The scan is $O(n)$ time and $O(1)$ space. The state is reachability, not the exact path or the minimum number of jumps. A zero is harmless if the frontier already extends beyond it, but fatal when it blocks the frontier.

Walk Through the Example

Trace the example one update at a time

For `[3,2,1,0,4]`, indices 0 through 3 are reachable, but none can extend the frontier beyond 3. When the scan reaches index 4, $4 > 3$, so the destination is unreachable. For `[2,3,1,1,4]`, index 1 extends the frontier directly to 4.

Why It Works

The reachable indices always form a prefix. Processing every index inside that prefix records the farthest next endpoint; seeing an index beyond it proves no earlier jump can reach that position or anything after it.

Tests to try

Test one element, a leading zero, zeros already covered by the frontier, an exact jump to the end, and a frontier that stops one position short.

How to explain it in an interview

I do not choose an exact path. I scan the reachable prefix and keep its farthest extension. If the scan ever moves beyond that frontier the answer is false; reaching the last index makes it true.

A Closer Look

Treat each position as a statement about future reach. From the current index, `i+nums[i]` is the farthest place reachable with one more jump. The algorithm keeps the farthest position reachable from any index seen so far.

Before using an index, first check that it is not beyond the current reach. If `i>farthest`, there is a gap that no earlier jump can cross, so the answer is false. Otherwise, enlarge `farthest` with the current jump. For `[2,3,1,1,4]`, index 0 reaches 2, and index 1 extends the reach to 4, which is the final index. For `[3,2,1,0,4]`, the reach stops at 3; index 4 cannot be processed. The algorithm does not need to construct the actual jump sequence because the question asks only whether a path exists.

BSTS AND TREE DEPTH

Lesson 34: Maximum Depth of Binary Tree

Problem at a glance

Difficulty	Approach	Main tool
Easy	Recursive DFS	Binary tree

The Problem

Return the number of nodes on the longest path from the root to a leaf. An empty tree has depth zero.

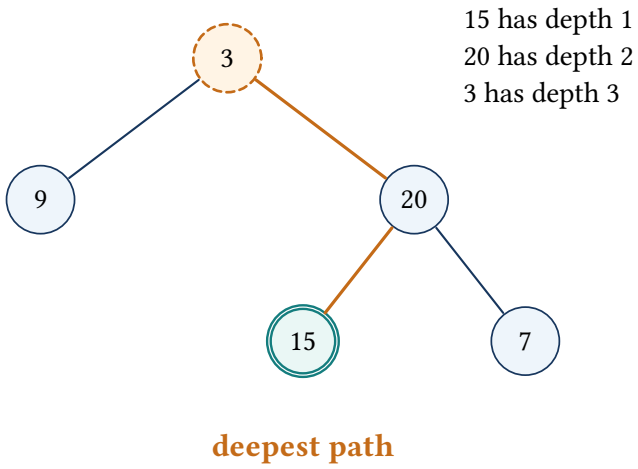
Recursive Definition

The maximum depth of a non-null node is one plus the larger depth of its two subtrees. This definition translates directly into postorder recursion.

Key idea: each call returns the exact depth of its subtree

After both child calls return, their depths are correct by the recursive hypothesis. Adding one for the current node produces the correct longest root-to-leaf node count for the current subtree.

The deepest root-to-leaf path contains three nodes



Pseudocode

```
depth(node):
    if node is null: return 0
    leftDepth = depth(node.left)
    rightDepth = depth(node.right)
    return 1 + max(leftDepth, rightDepth)
```

Code 34: Complete solution: Maximum Depth of Binary Tree

```cpp
#include <algorithm>

using namespace std;

class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == nullptr) return 0;
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

Time, Space, and Edge Cases

Every node is visited once: $O(n)$ time. The recursion stack is $O(h)$, which is $O(n)$ for a skewed tree and $O(\log n)$ for a balanced tree.

Walk Through the Example

Trace the example one update at a time

In the tree `[3,9,20,null,null,15,7]`, both children of 20 return depth 1, so node 20 returns 2. Node 9 returns 1, and the root returns $1 + \max(1, 2) = 3$.

Why It Works

The empty subtree has depth zero. Assuming both child calls return correct depths, adding one to their maximum gives exactly the longest root-to-leaf node count, which proves the update rule by working upward from the leaves.

Tests to try

Check a null root, one node, a left-only chain, a right-only chain, and a balanced tree whose deepest leaves occur on both sides.

How to explain it in an interview

I ask each child for its subtree depth and return one plus the larger result. Every node is visited once, while extra space equals the tree height because of recursion.

A Closer Look

The depth of a node is one plus the larger depth of its two children. This definition leads directly to a recursive solution. A null child contributes zero, and a leaf therefore returns one.

The calls first move down until they reach null pointers. As recursion returns, each node receives the completed depths of its left and right subtrees and adds one for itself. In a tree whose root has a shallow left child and a deeper right chain, the maximum follows the right side even though both sides are still evaluated. Every node is visited once, so the work is linear. The extra space comes from the call stack and equals the tree height: logarithmic for a balanced tree and linear for a completely skewed tree. A breadth-first solution can also count levels, but recursion matches the mathematical definition with less bookkeeping.

BSTS AND TREE DEPTH

Lesson 35: Kth Smallest Element in a BST

Problem at a glance

Difficulty	Approach	Main tool
Medium	Inorder traversal	BST and stack

The Problem

Given a binary search tree and k, return its k-th smallest value.

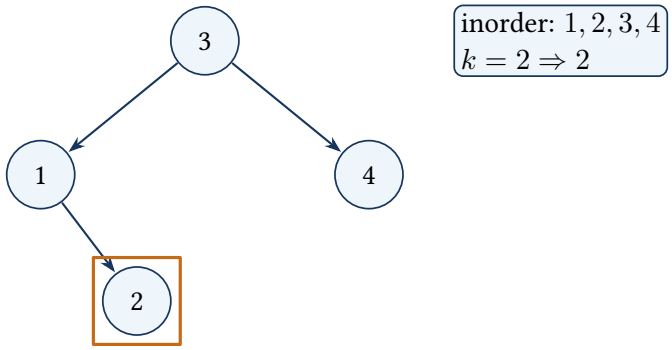
Use the BST Ordering

An inorder traversal of a BST visits values in ascending order. An explicit stack pauses the walk while descending left, then emits one node at a time. The k-th popped node is the answer.

Key idea: the next popped node is the smallest unvisited value

The stack stores the path to the next inorder node. All values already popped are smaller, and every unvisited value in a right subtree is processed only after its parent.

Inorder turns the BST into sorted output



Pseudocode

```
stack = empty; current = root
while current exists or stack is not empty:
    while current exists:
        push current; current = current.left
    current = pop stack
    decrement k; if k is zero: return current.value
    current = current.right
```

Code 35: Complete solution: Kth Smallest in a BST

```cpp
#include <stack>

using namespace std;

class Solution {
public:
    int kthSmallest(TreeNode* root, int k) {
        stack<TreeNode*> path;
        TreeNode* current = root;
        while (current != nullptr || !path.empty()) {
            while (current != nullptr) {
                path.push(current);
                current = current->left;
            }
            current = path.top();
            path.pop();
            if (--k == 0) return current->val;
            current = current->right;
        }
        return -1;
    }
};
```

Time and Space

The traversal stops after reaching the answer. Worst-case time is $O(n)$, and stack space is $O(h)$. The problem guarantees a valid k, so the fallback return is unreachable for valid input.

Walk Through the Example

Trace the example one update at a time

For the BST `[3,1,4,null,2]` with $k = 2$, push 3 and 1 while descending left. Pop 1 as rank one, move to its right child 2, then pop 2 as rank two and return it.

Why It Works

Inorder traversal emits BST values in sorted order. The stack stores the unprocessed ancestors needed to resume that order, so the k-th pop is exactly the k-th smallest value.

Tests to try

Check $k = 1$, $k = n$, a skewed tree, a balanced tree, and a target found after entering a right subtree.

How to explain it in an interview

I perform iterative inorder traversal and decrement k whenever a node is popped. Since inorder is sorted for a BST, the node that reduces k to zero is the answer.

A Closer Look

Inorder traversal of a binary search tree visits values in sorted order. The k-th value produced by that traversal is therefore the k-th smallest value in the tree. An explicit stack lets the method stop as soon as that value is reached.

Starting at the root, push every node on the path to the leftmost child. Pop one node, count it, and then repeat the same left-path process from its right child. The stack contains ancestors whose own visit or right subtree is still waiting. Decrement k on each pop, not on each push. When it reaches zero, return that node's value. In a balanced tree the stack holds at most the tree height. The worst case is a skewed tree, but early stopping often avoids visiting every node.

BSTS AND TREE DEPTH

Lesson 36: Lowest Common Ancestor of a BST

Problem at a glance

Difficulty	Approach	Main tool
Medium	Ordered tree search	BST

The Problem

Return the lowest node in a BST that is an ancestor of both given nodes. A node may be an ancestor of itself.

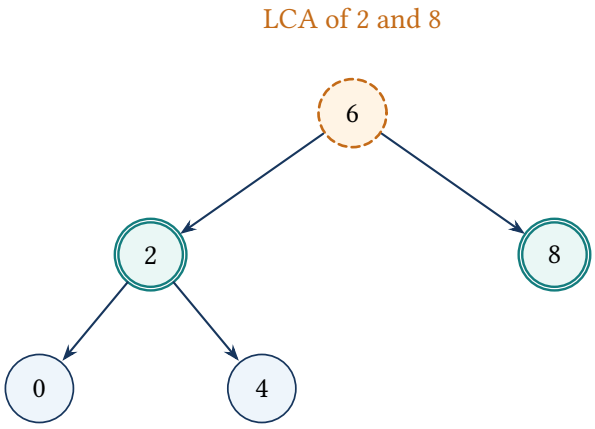
Follow the Shared Direction

If both target values are smaller than the current value, their ancestor lies left. If both are larger, it lies right. Otherwise the paths split at the current node, which is their lowest common ancestor.

Key idea: both targets remain inside the current subtree

Every move selects the only child subtree that can contain both values under the BST ordering. The first node where the targets no longer share a strict direction is the deepest node whose subtree contains them both.

The first path split is the answer



Pseudocode

```
current = root
while current exists:
    if p and q are both smaller: current = current.left
    else if p and q are both larger: current = current.right
    else: return current
return null
```

Code 36: Complete solution: LCA of a BST

```cpp
using namespace std;

class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root,
                                   TreeNode* p, TreeNode* q) {
        TreeNode* current = root;
        while (current != nullptr) {
            if (p->val < current->val && q->val < current->val) {
                current = current->left;
            } else if (p->val > current->val && q->val > current->val) {
                current = current->right;
            } else {
                return current;
            }
        }
        return nullptr;
    }
};
```

Time and Space

Only one root-to-node path is followed: $O(h)$ time and $O(1)$ space. Unlike the general binary-tree version, no parent map or full traversal is needed because ordering tells which subtree can contain each value.

Walk Through the Example

Trace the example one update at a time

In the BST rooted at 6, targets 2 and 4 are both smaller than 6, so move to node 2. At node 2, one target equals the current value while the other lies to the right; their paths split here, making node 2 the LCA.

Why It Works

BST ordering proves that when both targets are smaller or both are larger, only one child subtree can still contain both. The first node where neither shared direction applies is their deepest common ancestor.

Tests to try

Check targets on opposite sides of the root, one target equal to the LCA, adjacent nodes, and both targets deep in the same subtree.

How to explain it in an interview

I follow the one direction shared by both target values. When they split, or when the current node equals one target, the current node is the lowest common ancestor.

A Closer Look

The search can use the ordering of the BST rather than building parent links. At each node, compare both target values with the current value. If both are smaller, both targets must lie in the left subtree. If both are larger, both must lie in the right subtree.

The first time the targets do not share one strict direction, the paths have split. The current node is then their lowest common ancestor. Equality also stops the search because a node is allowed to be an ancestor of itself. For a tree rooted at 6 with targets 2 and 4, both values first send the search left. At node 2, one target equals the current value and the other lies to the right, so node 2 is the answer. Only one root-to-node path is followed, giving $O(h)$ time and constant extra space.

HEAPS AND SCHEDULING

Lesson 37: Meeting Rooms II

Problem at a glance

Difficulty	Approach	Main tool
Medium	Minimum end-time heap	Intervals and min heap

The Problem

Return the minimum number of rooms needed to hold all meetings without overlap.

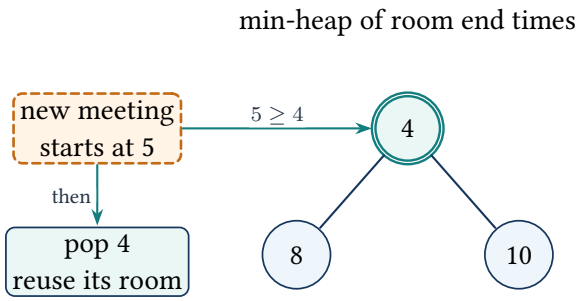
Track Rooms by Their End Times

Sort meetings by start. A min heap stores the ending time of every room currently in use. Before assigning a meeting, release every room whose meeting has ended; then push the new end. The maximum heap size is the answer.

Key idea: the heap contains exactly the occupied rooms

After expired end times are removed for a start time, each remaining heap entry belongs to a meeting overlapping the new one. Pushing the current end makes the heap size equal to the rooms simultaneously occupied.

The earliest ending room is released first



Pseudocode

```
sort meetings by start time
minHeap = empty; answer = 0
for each meeting:
    while heap top <= meeting.start: pop heap
    push meeting.end
    answer = max(answer, heap size)
return answer
```

Code 37: Complete solution: Meeting Rooms II

```cpp
#include <algorithm>
#include <functional>
#include <queue>
#include <vector>

using namespace std;

class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        priority_queue<int, vector<int>, greater<int>> ends;
        int answer = 0;
        for (const auto& meeting : intervals) {
            while (!ends.empty() && ends.top() <= meeting[0]) ends.pop();
            ends.push(meeting[1]);
            answer = max(answer, static_cast<int>(ends.size()));
        }
        return answer;
    }
};
```

Time and Space

Sorting and heap operations cost $O(n \log n)$ time. The heap uses $O(n)$ space in the worst case. Removing all expired rooms keeps the key rule exact; removing only one is enough for some variants but less directly models the current occupancy.

Walk Through the Example

Trace the example one update at a time

For `[[0,30],[5,10],[15,20]]`, push 30 for the first room. At start 5, room 30 is still occupied, so push 10 and the heap size becomes two. At start 15, pop 10 because it has ended, then push 20. The maximum heap size is two rooms.

Why It Works

After expired end times are removed, every heap entry corresponds to a meeting that overlaps the new start. So the heap size is exactly the current room demand, and its maximum is the minimum number of rooms that can schedule all meetings.

Tests to try

Check no meetings, one meeting, touching meetings, completely nested meetings, identical intervals, and many meetings ending at the same time.

How to explain it in an interview

I sort by start time and keep active end times in a min heap. The earliest ending rooms are released before each insertion. The maximum number of active end times is the required room count.

A Closer Look

Meeting Rooms II asks for the largest number of meetings active at the same time. Sort the meetings by start time and keep the end time of each occupied room in a min heap. The smallest end is the room that becomes available first.

Before placing a meeting, remove every end time that is less than or equal to its start. Those rooms are free. Push the new end, then record the largest heap size. With `[[0,30],[5,10],[15,20]]`, the heap becomes $[30]$, then contains 10 and 30, so two rooms are needed. At start 15, the end 10 is removed before 20 is inserted. Keeping only the earliest end at the top is what makes each reuse decision efficient. A max heap would expose the wrong room. Sorting and heap operations give $O(n \log n)$ time.

HEAPS AND SCHEDULING

Lesson 38: Non-overlapping Intervals

Problem at a glance

Difficulty	Approach	Main tool
Medium	Greedy by earliest finish	Intervals

The Problem

Return the minimum number of intervals to remove so the remaining intervals do not overlap.

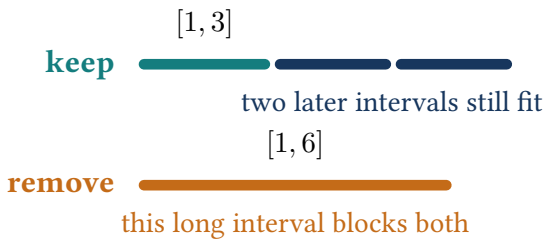
Keep the Interval That Finishes First

Sort by end time. Whenever the next interval starts before the last kept end, remove it. Choosing the earliest possible end leaves the greatest amount of space for all future intervals.

Key idea: the kept intervals finish as early as possible

After scanning a prefix, the algorithm has kept the maximum possible number of compatible intervals from that prefix, and the final kept interval has the smallest achievable end among choices of that size.

An early finish keeps more future choices



Pseudocode

```
sort intervals by end time
keptEnd = end of first interval; removals = 0
for each remaining interval:
    if interval.start < keptEnd: increment removals
    else: keptEnd = interval.end
return removals
```

Code 38: Complete solution: Non-overlapping Intervals

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end(),
             [](const vector<int>& a, const vector<int>& b) {
                 return a[1] < b[1];
             });
        int removals = 0;
        int lastEnd = intervals[0][1];
        for (int i = 1; i < static_cast<int>(intervals.size()); ++i) {
            if (intervals[i][0] < lastEnd) {
                ++removals;
            } else {
                lastEnd = intervals[i][1];
            }
        }
        return removals;
    }
};
```

Why It Works and Complexity

The standard exchange argument replaces any later-finishing kept interval with the greedy one without harming compatibility. Sorting dominates at $O(n \log n)$; the scan uses $O(1)$ extra state. Touching endpoints do not overlap.

Walk Through the Example

Trace the example one update at a time

For `[[1,2],[2,3],[3,4],[1,3]]`, keep $[1, 2]$, then $[2, 3]$, then $[3, 4]$. The interval $[1, 3]$ conflicts and is removed, producing the minimum removal count one.

Why It Works

Among overlapping choices, keeping the interval with the earliest finish leaves at least as much room for every future interval as keeping a later finish. Repeating this exchange argument proves the greedy optimum.

Tests to try

Check empty input, one interval, touching endpoints, identical intervals, nested intervals, and several conflicts sharing the same ending time.

How to explain it in an interview

I sort by end time and keep each interval whose start is at least the last kept end. Every rejected interval overlaps the most future-friendly choice, so counting those rejections gives the minimum removals.

A Closer Look

The greedy choice is to keep the interval that ends first. A shorter finishing time leaves at least as much room for every later interval. Sort by end, keep the first interval, and compare every next start with the last kept end.

If the next interval begins too early, count one removal and keep the earlier end already chosen. If it begins at or after that end, it is compatible and becomes the new last interval. For `[[1,2],[2,3],[3,4],[1,3]]`, the first three short intervals can all remain, while $[1, 3]$ must be removed. An exchange argument explains the choice: whenever another solution keeps a later-finishing interval, replacing it with the greedy interval cannot make a future compatible interval invalid. The scan therefore keeps as many intervals as possible, and removals equal total intervals minus kept ones.

HEAPS AND SCHEDULING

Lesson 39: Merge Intervals

Problem at a glance

Difficulty	Approach	Main tool
Medium	Sort and sweep	Intervals

The Problem

Merge every pair of overlapping intervals and return non-overlapping intervals.

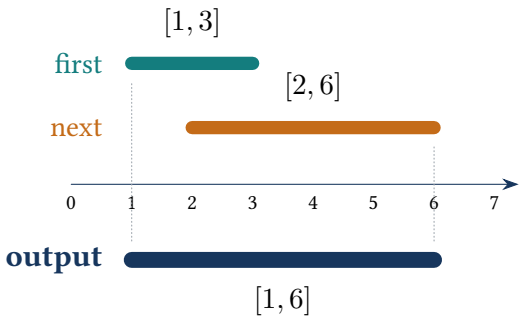
Build One Active Interval

After sorting by start, compare the next start with the end of the last merged interval. An overlap extends that end; otherwise the next interval begins a new non-overlapping group.

Key idea: the output is merged and ends with the active group

Before each comparison, all output intervals except possibly the last are final and cannot overlap any later interval. The last interval equals the union of the current overlapping group.

Overlapping ranges collapse into one range



Pseudocode

```
sort intervals by start time
merged = empty
for interval in sorted order:
    if merged is empty or interval starts after merged.back ends:
        append interval
    else: extend merged.back end
return merged
```

Code 39: Complete solution: Merge Intervals

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    vector<vector<int>> merge(
            vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        vector<vector<int>> merged;
        for (const auto& interval : intervals) {
            if (merged.empty() || merged.back()[1] < interval[0]) {
                merged.push_back(interval);
            } else {
                merged.back()[1] = max(merged.back()[1], interval[1]);
            }
        }
        return merged;
    }
};
```

Time and Space

Time is $O(n \log n)$ for sorting and $O(n)$ for the sweep. Excluding the returned vector, sorting may use $O(\log n)$ stack space. Equal endpoints merge because the intervals overlap at that endpoint.

Walk Through the Example

Trace the example one update at a time

Sort `[[1,3],[2,6],[8,10],[15,18]]`. Merge the first two into $[1, 6]$; the next starts after 6, so append $[8, 10]$, followed by $[15, 18]$.

Why It Works

Once intervals are sorted by start, only the final merged interval can overlap the next input. Extending that final end keeps the exact union; otherwise appending begins a new non-overlapping group.

Tests to try

Check one interval, contained intervals, equal endpoints, repeated intervals, one chain of overlaps, and input with no overlaps.

How to explain it in an interview

I sort by start time and compare each interval with the end of the current merged block. It either extends that block or begins a new one.

A Closer Look

Sorting by start time turns many separate overlap checks into one comparison with the active merged interval. The output is built from left to right. Its last interval is the only one that can still grow.

If the next start is greater than the last output end, there is a gap, so the next interval is appended. Otherwise the intervals overlap and the last end becomes the larger of the two ends. For `[[1,3],[2,6],[8,10],[15,18]]`, the first two ranges become $[1, 6]$. The next start, 8, lies beyond 6, so a new block begins. Contained intervals need no special case: taking the maximum end keeps the larger range. Equal endpoints merge because they share a point. After sorting, no interval earlier than the last output block can overlap a future interval.

HEAPS AND SCHEDULING

Lesson 40: Insert Interval

Problem at a glance

Difficulty	Approach	Main tool
Medium	Three-phase interval scan	Intervals

The Problem

Insert one interval into a sorted, non-overlapping interval list and merge any new overlaps.

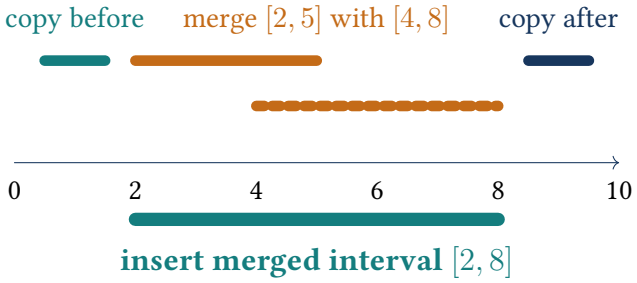
Before, Overlap, After

Append intervals ending before the new interval. Merge every interval whose start is at most the new end. Append the merged interval, then copy all remaining intervals.

Key idea: processed intervals are final and sorted

During the first phase, copied intervals lie strictly before the new range. During the second, the mutable new interval equals the union of every overlap seen so far. The remaining intervals begin after it once merging stops.

Insertion has three consecutive regions



Pseudocode

```
append intervals ending before newInterval starts
while intervals overlap newInterval:
    expand newInterval to their minimum start and maximum end
append the expanded newInterval
append every interval starting after it
return result
```

Code 40: Complete solution: Insert Interval

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    vector<vector<int>> insert(
            vector<vector<int>>& intervals,
            vector<int>& newInterval) {
        vector<vector<int>> result;
        int i = 0, n = static_cast<int>(intervals.size());
        while (i < n && intervals[i][1] < newInterval[0])
            result.push_back(intervals[i++]);
        while (i < n && intervals[i][0] <= newInterval[1]) {
            newInterval[0] = min(newInterval[0], intervals[i][0]);
            newInterval[1] = max(newInterval[1], intervals[i][1]);
            ++i;
        }
        result.push_back(newInterval);
        while (i < n) result.push_back(intervals[i++]);
        return result;
    }
};
```

Time, Space, and Edge Cases

Every interval is copied or merged once: $O(n)$ time and $O(n)$ output space. Test insertion before all intervals, after all intervals, inside one interval, and across several intervals.

Walk Through the Example

Trace the example one update at a time

Insert $[4, 8]$ into $[[1, 2], [3, 5], [6, 7], [9, 12]]$. Copy $[1, 2]$. Merge $[3, 5]$ to obtain $[3, 8]$, then merge $[6, 7]$ without changing the bounds. Append $[3, 8]$, followed by $[9, 12]$.

Why It Works

Because the original intervals are sorted and do not overlap, they form three consecutive regions: strictly before, overlapping, and strictly after the inserted interval. Processing these regions in order produces a sorted, fully merged result.

Tests to try

Test an empty list, insertion before or after everything, containment in either direction, touching endpoints, and one new interval spanning the complete list.

How to explain it in an interview

I exploit the existing sorted order. I first copy intervals that end before the new one, then enlarge the new boundaries across every overlap, and finally copy the remaining suffix. Each interval is visited once.

A Closer Look

Because the original intervals are already sorted and non-overlapping, they must fall into three consecutive regions around the inserted interval. First come intervals completely before it, then intervals touching or overlapping it, and finally intervals completely after it.

Copy the first region unchanged. In the overlap region, repeatedly lower the new start and raise the new end. When that loop stops, append the fully merged new interval once, then copy the remaining suffix. Inserting $[4, 8]$ into `[[1,2],[3,5],[6,7],[9,12]]` first copies $[1, 2]$, then expands the new range to $[3, 8]$, and finally copies $[9, 12]$. The comparisons are deliberately strict before the range and non-strict during merging so touching endpoints are handled as overlaps. Each original interval is examined once.

GRID AND GRAPH COMPONENTS

Lesson 41: Number of Islands

Problem at a glance

Difficulty	Approach	Main tool
Medium	Grid DFS	Matrix and recursion

The Problem

Count connected groups of land cells in a grid. Connections are horizontal or vertical, not diagonal.

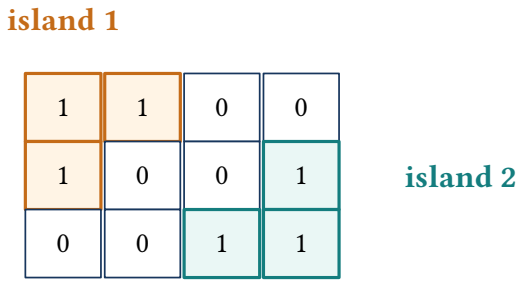
Flood One Island at a Time

When an unvisited land cell is found, increment the answer and flood-fill its entire component. Marking visited cells as water prevents recounting and makes the grid itself the visited structure.

Key idea: every discovered land component is erased exactly once

After the scan passes a cell, no remaining land cell in its connected component survives. So each future land discovery belongs to a new island and increments the count exactly once.

DFS floods all four-directionally connected land



Pseudocode

```
answer = 0
for every grid cell:
    if cell is land:
        increment answer
        DFS from it, changing all connected land to visited water
return answer
```

Code 41: Complete solution: Number of Islands

```cpp
#include <vector>

using namespace std;

class Solution {
    void flood(vector<vector<char>>& grid, int row, int col) {
        if (row < 0 || col < 0 || row >= static_cast<int>(grid.size()) ||
            col >= static_cast<int>(grid[0].size()) || grid[row][col] != '1')
            return;
        grid[row][col] = '0';
        flood(grid, row + 1, col); flood(grid, row - 1, col);
        flood(grid, row, col + 1); flood(grid, row, col - 1);
    }

public:
    int numIslands(vector<vector<char>>& grid) {
        int islands = 0;
        for (int r = 0; r < static_cast<int>(grid.size()); ++r) {
            for (int c = 0; c < static_cast<int>(grid[0].size()); ++c) {
                if (grid[r][c] == '1') {
                    ++islands;
                    flood(grid, r, c);
                }
            }
        }
        return islands;
    }
};
```

Time, Space, and Safety

Each cell is examined a constant number of times: $O(mn)$ time. Recursion can use $O(mn)$ stack space for one large island. The rules provide a non-empty grid; an iterative stack can avoid call-stack limits in production.

Walk Through the Example

Trace the example one update at a time

In the grid `[[1,1,0],[1,0,0],[0,0,1]]`, the scan first discovers the upper-left land cell. DFS erases the three connected cells, so none can be counted again. The final lower-right land cell starts a second DFS and produces the answer two.

Why It Works

A new island is counted only at an unvisited land cell. Flood fill reaches exactly its four-directional connected component and marks every member, so later scan positions cannot count that component again.

Tests to try

Check a single water cell, a single land cell, all land, all water, a thin row, a thin column, diagonal-only contact, and a large snake-shaped island.

How to explain it in an interview

I scan every cell and launch DFS only from unvisited land. That one DFS consumes the complete island. Since each cell changes from land to visited at most once, the total work is linear in the grid size.

A Closer Look

The outer grid scan finds the first cell of each island. Once land is found, DFS marks every land cell connected to it in the four allowed directions. Changing visited land to water lets the input grid serve as the visited set.

The DFS stops at a boundary, at water, or at a cell that was already erased. Only the outer scan increases the island count; recursive calls only finish the current component. This separation prevents one island from being counted many times. A diagonal land cell is not reached because the recursive calls use only up, down, left, and right. Every cell is checked a constant number of times, so time is $O(mn)$. The recursion stack can hold $O(mn)$ cells for one large winding island; an explicit stack can be used when recursion depth is a concern.

GRID AND GRAPH COMPONENTS

Lesson 42: Number of Connected Components in an Undirected Graph

Problem at a glance

Difficulty	Approach	Main tool
Medium	Graph DFS	Adjacency list

The Problem

Given n vertices and undirected edges, return the number of connected components.

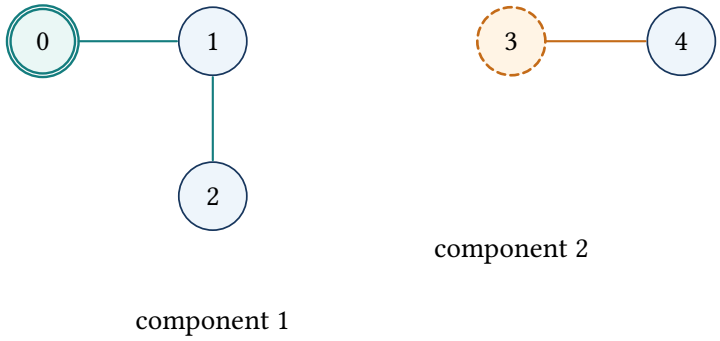
Start One Traversal per Component

Build an adjacency list. Each unvisited vertex starts a DFS that marks every vertex reachable from it; this single start represents one component.

Key idea: visited vertices belong to already counted components

After a DFS finishes, every vertex in that connected component is marked. So the next unvisited vertex cannot belong to any component already counted.

Two DFS starts reveal two components



Pseudocode

```
build an undirected adjacency list
visited = all false; components = 0
for each vertex:
    if vertex is unvisited:
        increment components
        DFS and mark its entire component
return components
```

Code 42: Complete solution: Connected Components

```cpp
#include <vector>

using namespace std;

class Solution {
    void visit(int node, const vector<vector<int>>& graph,
               vector<bool>& seen) {
        seen[node] = true;
        for (int neighbor : graph[node])
            if (!seen[neighbor]) visit(neighbor, graph, seen);
    }

public:
    int countComponents(int n, vector<vector<int>>& edges) {
        vector<vector<int>> graph(n);
        for (const auto& edge : edges) {
            graph[edge[0]].push_back(edge[1]);
            graph[edge[1]].push_back(edge[0]);
        }
        vector<bool> seen(n, false);
        int components = 0;
        for (int node = 0; node < n; ++node) {
            if (!seen[node]) {
                ++components;
                visit(node, graph, seen);
            }
        }
        return components;
    }
};
```

Time and Space

Building and traversing the graph costs $O(V + E)$ time and space. Isolated vertices are valid one-vertex components. Because the graph is undirected, each edge must be added in both directions.

Walk Through the Example

Trace the example one update at a time

With $n = 5$ and edges `[[0,1],[1,2],[3,4]]`, DFS from 0 marks $0, 1, 2$ as one component. The next unvisited vertex is 3, whose DFS also marks 4. Exactly two DFS launches occur, so the answer is two.

Why It Works

A DFS started at an unvisited vertex reaches exactly its connected component. Marking that entire set prevents recounting it, while every later unvisited vertex must belong to a different component.

Tests to try

Check no edges, one vertex, a fully connected graph, duplicate edges, self-loops, and several isolated vertices.

How to explain it in an interview

I build an undirected adjacency list and launch DFS from each still-unseen vertex. Each launch discovers one complete component, giving linear time in the vertices and edges.

A Closer Look

The graph may arrive as an edge list, but DFS needs to know the neighbors of one vertex quickly. First build an adjacency list by adding each undirected edge in both directions. Then scan all vertex numbers.

When an unvisited vertex appears, it starts one new connected component. Increase the answer and run DFS or BFS to mark every vertex reachable from it. Later vertices from that same component will already be marked and will not increase the count. Isolated vertices still count because they appear in the outer scan even though their neighbor list is empty. With n vertices and e edges, adjacency construction and traversal take $O(n + e)$ time. The visited array prevents cycles from causing repeated recursion.

GRID AND GRAPH COMPONENTS

Lesson 43: Longest Consecutive Sequence

Problem at a glance

Difficulty	Approach	Main tool
Medium	Hash-set starts	Hash set

The Problem

Return the length of the longest run of consecutive integer values in an unsorted array. The algorithm should run in linear average time.

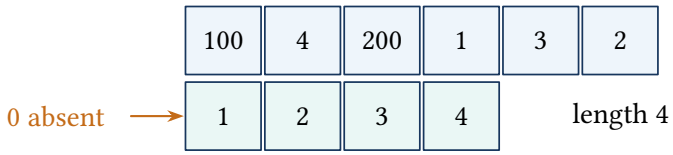
Count Only from Sequence Starts

Put every value in a set. A value begins a sequence only when its previous is absent. From such starts, count upward until the sequence ends. Interior values never launch redundant scans.

Key idea: each sequence is expanded exactly once

Only the smallest value of a consecutive run lacks a previous. So one scan accounts for the entire run, and every value participates in at most one forward expansion.

The missing previous identifies a start



Pseudocode

```
values = hash set of all numbers
answer = 0
for value in values:
    if value - 1 is absent:
        length = 1
        while value + length exists: increment length
        answer = max(answer, length)
return answer
```

Code 43: Complete solution: Longest Consecutive Sequence

```cpp
#include <algorithm>
#include <unordered_set>
#include <vector>

using namespace std;

class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> values(nums.begin(), nums.end());
        int answer = 0;
        for (int value : values) {
            if (!values.count(value - 1)) {
                int current = value, length = 1;
                while (values.count(current + 1)) {
                    ++current;
                    ++length;
                }
                answer = max(answer, length);
            }
        }
        return answer;
    }
};
```

Time and Space

Set operations are $O(1)$ average, and every distinct value is advanced through once, giving $O(n)$ average time and $O(n)$ space. Duplicates are removed automatically by the set.

Walk Through the Example

Trace the example one update at a time

For `[100,4,200,1,3,2]`, only 100, 200, and 1 lack predecessors. The scan starting at 1 advances through 2, 3, and 4, producing length four; values 2, 3, and 4 never start duplicate scans.

Why It Works

Every consecutive sequence has exactly one value with no previous. Starting only there counts every member once and prevents the quadratic repeated work caused by expanding from interior values.

Tests to try

Check empty input, duplicates, negative values, one long sequence, several tied sequences, and values near integer limits.

How to explain it in an interview

I place values in a hash set and expand only from values whose previous is absent. Those are exactly the starts of maximal sequences, so the total expected work is linear.

A Closer Look

The hash set gives fast membership tests, but starting a count from every value would still repeat work. A value begins a consecutive sequence only when its predecessor is absent. That one test identifies the left edge of each run.

For every start, test $x + 1, x + 2$, and so on until a number is missing. Record the longest length. In `[100,4,200,1,3,2]`, values 2, 3, and 4 do not start scans because their predecessors exist. Only 1 grows through the run $1, 2, 3, 4$. Although there is a loop inside another loop, each number belongs to only one forward scan, so the total expected work is linear. Duplicates disappear naturally in the set. Sorting would also work, but it costs $O(n \log n)$ and changes the recognition pattern.

LINKED-LIST MERGING

Lesson 44: Merge Two Sorted Lists

Problem at a glance

Difficulty	Approach	Main tool
Easy	Two-pointer merge	Linked lists

The Problem

Merge two sorted singly linked lists by reusing their nodes and return the head of the sorted result.

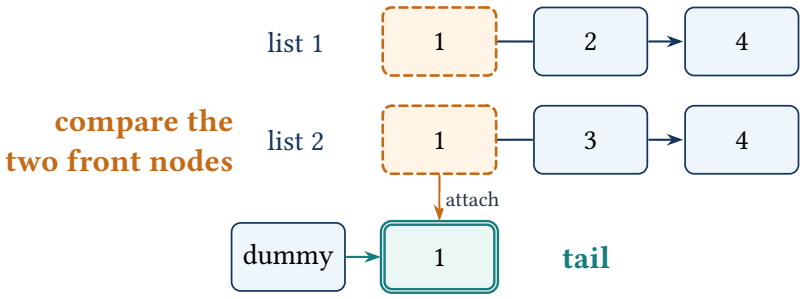
Attach the Smaller Front Node

A dummy node removes special handling for the first attachment. At each step, link the smaller current node after the tail and advance that list. Once one list ends, append the other remainder.

Key idea: the merged prefix is sorted and complete

The nodes before `tail` are exactly the smallest consumed nodes from both lists in sorted order. The two current pointers identify the smallest remaining choice from each input.

Two sorted fronts feed one output tail



Pseudocode

```
dummy tail starts an empty result
while both lists are non-empty:
    attach the smaller front node; advance that list
attach the remaining non-empty list
return dummy.next
```

Code 44: Complete solution: Merge Two Sorted Lists

```cpp
using namespace std;

class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        ListNode dummy(0);
        ListNode* tail = &dummy;
        while (list1 != nullptr && list2 != nullptr) {
            if (list1->val <= list2->val) {
                tail->next = list1;
                list1 = list1->next;
            } else {
                tail->next = list2;
                list2 = list2->next;
            }
            tail = tail->next;
        }
        tail->next = (list1 != nullptr) ? list1 : list2;
        return dummy.next;
    }
};
```

Time and Space

Each node is linked once: $O(m + n)$ time and $O(1)$ extra space. The dummy node is stack allocated and is not part of the returned list.

Walk Through the Example

Trace the example one update at a time

Merge `1->2->4` with `1->3->4`. Attach 1 from the first list, 1 from the second, then 2, 3, and 4. When one list becomes empty, attach the remaining suffix directly.

Why It Works

At every step the smaller front node is the smallest value not yet emitted. Attaching it keeps sorted order, and once one list is empty the other suffix is already sorted and no longer competes with another value.

Tests to try

Check two null lists, one null list, equal front values, one list entirely smaller, and lists containing negative or repeated values.

How to explain it in an interview

I use a dummy head and one tail pointer. The smaller current node is linked after the tail, its source advances, and the final remaining suffix is attached in constant time.

A Closer Look

The dummy node removes the special case for the first output node. A tail pointer always marks the final node in the merged list. Compare the two current values, connect the smaller node after the tail, and advance only the list that supplied it.

No new data nodes are required; the method reuses the existing links. When one list ends, its competitor is already sorted, so the complete remaining suffix can be attached in one step. For `1->2->4` and `1->3->4`, the tail follows values 1, 1, 2, 3, 4, 4. Saving the next choice before changing links is useful when drawing the process. Time is linear in the total number of nodes, and apart from the dummy node the algorithm uses constant extra space.

LINKED-LIST MERGING

Lesson 45: Merge K Sorted Lists

Problem at a glance

Difficulty	Approach	Main tool
Hard	Minimum-node heap	Linked lists and min heap

The Problem

Merge k sorted linked lists into one sorted list.

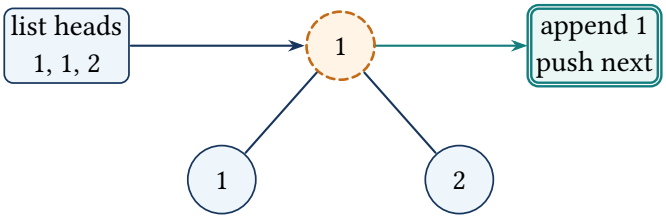
Expose One Choice per List

The smallest remaining node must be one of the current list heads. Store those heads in a min heap, remove the smallest, attach it, and push its next.

Key idea: the heap contains every possible next output node

For each non-empty unmerged list suffix, the heap contains its first node. Because each suffix is sorted, the minimum heap entry is globally the smallest remaining node and is safe to append.

One heap entry represents each list frontier



Pseudocode

```
push every non-null list head into a min heap
dummy tail starts the result
while heap is not empty:
    node = pop smallest
    attach node to result
    if node.next exists: push node.next
return dummy.next
```

Code 45: Complete solution: Merge K Sorted Lists

```cpp
#include <queue>
#include <vector>

using namespace std;

class Solution {
    struct Compare {
        bool operator()(ListNode* a, ListNode* b) const {
            return a->val > b->val;
        }
    };
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        priority_queue<ListNode*, vector<ListNode*>, Compare> heap;
        for (ListNode* head : lists) if (head != nullptr) heap.push(head);
        ListNode dummy(0);
        ListNode* tail = &dummy;
        while (!heap.empty()) {
            ListNode* node = heap.top();
            heap.pop();
            tail->next = node;
            tail = node;
            if (node->next != nullptr) heap.push(node->next);
        }
        return dummy.next;
    }
};
```

Time and Space

For N total nodes, each heap operation costs $O(\log k)$, producing $O(N \log k)$ time and $O(k)$ heap space. Null list heads are never pushed.

Walk Through the Example

Trace the example one update at a time

For lists `1->4->5`, `1->3->4`, and `2->6`, the heap begins with $1, 1, 2$. Pop the first 1 and push 4; pop the second 1 and push 3; then pop 2 and push 6. Continuing yields `1->1->2->3->4->4->5->6`.

Why It Works

Every remaining list is sorted, so its head is its smallest unmerged node. The smallest remaining node must be among the heap entries. Popping it is safe, and pushing its next restores one frontier per list.

Tests to try

Check zero lists, all-null lists, one list, duplicate values, lists of very different lengths, and negative values.

How to explain it in an interview

I place one current node from each list into a min heap. The heap exposes the next global output value, after which I insert only that node's next. This uses $O(k)$ extra space rather than storing all nodes.

A Closer Look

At any moment, the next output node must be the smallest current head among all lists. A min heap stores exactly those candidates. Push every non-null head, pop the smallest node, attach it to the result, and push that node's next pointer if it exists.

The heap never needs every node at once; it holds at most one candidate from each list. With k lists, each push and pop costs $O(\log k)$. Across N total nodes, the running time is $O(N \log k)$, and heap space is $O(k)$. The comparator must order node values in the min-heap direction. As in merging two lists, a dummy head simplifies construction and the original nodes can be relinked instead of copied. Empty lists are skipped when the heap is initialized.

STRING DP AND WINDOWS

Lesson 46: Longest Common Subsequence

Problem at a glance

Difficulty	Approach	Main tool
Medium	Two-dimensional DP	Strings and DP table

The Problem

Return the length of the longest sequence appearing in both strings in the same relative order. Characters need not be consecutive.

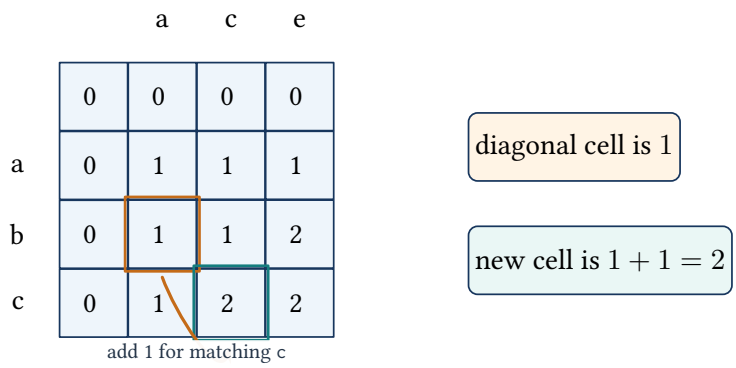
Prefix Update Rule

Let $dp[i][j]$ be the LCS length of the first i characters of the first string and the first j of the second. Equal final characters extend the diagonal state; otherwise skip one final character from either string.

Key idea: each cell solves one pair of prefixes

When $dp[i][j]$ is computed, the top, left, and diagonal previous cells already contain correct answers for smaller prefixes. The update rule covers whether the two final characters are paired or one is excluded.

A match reads the diagonal DP cell



Pseudocode

```
dp table starts at zero
for i from 1 to length(text1):
    for j from 1 to length(text2):
        if characters match: dp[i][j] = 1 + dp[i-1][j-1]
        else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
return dp[last row][last column]
```

Code 46: Complete solution: Longest Common Subsequence

```cpp
#include <algorithm>
#include <string>
#include <vector>

using namespace std;

class Solution {
public:
    int longestCommonSubsequence(string a, string b) {
        vector<vector<int>> dp(
            a.size() + 1, vector<int>(b.size() + 1, 0));
        for (int i = 1; i <= static_cast<int>(a.size()); ++i) {
            for (int j = 1; j <= static_cast<int>(b.size()); ++j) {
                if (a[i - 1] == b[j - 1]) dp[i][j] = 1 + dp[i - 1][j - 1];
                else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[a.size()][b.size()];
    }
};
```

Time and Space

For lengths m, n, time and table space are $O(mn)$. Space can be reduced to $O(n)$ with two rows, but the full table makes dependencies and sequence reconstruction clearer.

Walk Through the Example

Trace the example one update at a time

For abcde and ace, the first match writes one at the a/a cell. Mismatches copy the maximum from above or left. Matches at c and e read the diagonal and add one, so the bottom-right answer is three.

Why It Works

If the final characters match, a best subsequence can include that character and extend the solution for both shorter prefixes. If they differ, at least one final character is excluded, so the maximum of the top and left states is complete.

Tests to try

Check one empty string, identical strings, no common characters, repeated characters, reversed order, and one string contained as a subsequence.

How to explain it in an interview

I define each DP cell as the LCS length for two prefixes. Equal final characters use diagonal plus one; unequal characters choose the better result after skipping one side.

A Closer Look

The table state $dp[i][j]$ means the LCS length between suffixes beginning at positions i and j. If the two current characters match, that character can be used and the answer is one plus the diagonal state. If they do not match, one of the two current characters must be skipped.

This gives the recurrence $dp[i][j] = 1 + dp[i+1][j+1]$ for a match, otherwise the maximum of $dp[i+1][j]$ and $dp[i][j+1]$. Fill the table from the bottom right so all three needed states already exist. For abcde and ace, the matching choices a, c, and e produce length three without needing to be adjacent. Confusing a subsequence with a substring is the main conceptual mistake. The full table costs $O(mn)$ time and space, and rows can be rolled when only the length is required.

STRING DP AND WINDOWS

Lesson 47: Minimum Window Substring

Problem at a glance

Difficulty	Approach	Main tool
Hard	Variable sliding window	Character counts

The Problem

Return the shortest substring of `s` containing every character of `t`, including duplicate requirements. Return an empty string if none exists.

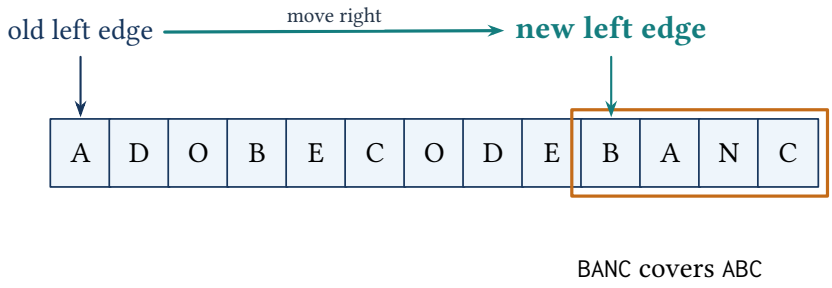
Expand Until Valid, Then Contract

A requirement table stores how many of each character are still needed. `missing` counts the total required occurrences not yet satisfied. Once it reaches zero, move the left boundary while the valid condition remains and record the shortest window.

Key idea: missing counts unsatisfied required occurrences

After adding or removing a boundary character, the requirement count and `missing` exactly describe whether the active window covers the target multiset. The contraction loop ends at the smallest valid window for that right endpoint.

The window contracts around the required multiset



Pseudocode

```
count required characters and number of satisfied kinds
for each right boundary:
    add its character to the window
    while all required kinds are satisfied:
        record the window if it is shorter
        remove the left character and advance left
return the best recorded substring
```

Code 47: Complete solution: Minimum Window Substring

```cpp
#include <array>
#include <limits>
#include <string>

using namespace std;

class Solution {
public:
    string minWindow(string s, string t) {
        array<int, 256> need{};
        for (unsigned char ch : t) ++need[ch];
        int missing = static_cast<int>(t.size());
        int left = 0, bestStart = 0, bestLength = numeric_limits<int>::max();
        for (int right = 0; right < static_cast<int>(s.size()); ++right) {
            unsigned char in = static_cast<unsigned char>(s[right]);
            if (need[in] > 0) --missing;
            --need[in];
            while (missing == 0) {
                if (right - left + 1 < bestLength) {
                    bestLength = right - left + 1;
                    bestStart = left;
                }
                unsigned char out = static_cast<unsigned char>(s[left++]);
                ++need[out];
                if (need[out] > 0) ++missing;
            }
        }
        return bestLength == numeric_limits<int>::max()
             ? "" : s.substr(bestStart, bestLength);
    }
};
```

Time and Space

Each boundary advances at most n times, so time is $O(n + |t|)$ and the fixed character table is $O(1)$. Duplicates in `t` are handled by counts rather than a set.

Walk Through the Example

Trace the example one update at a time

For `s=ADOBECODEBANC` and `t=ABC`, expand until the first valid window `ADOBEC`. Contracting loses the required A, so expansion resumes. Near the end, the valid condition is restored and repeated contraction produces `BANC`, the shortest length-four window.

Why It Works

The requirement array and `missing` exactly encode whether the current window covers the target multiset. For each right endpoint, the inner loop records every valid left contraction and stops immediately after removing a required occurrence. So the shortest valid window is tested.

Tests to try

Check an empty target if the interface permits it, a target longer than the source, repeated required characters such as `AABC`, exact equality, no solution, and a one-character answer.

How to explain it in an interview

I expand a counted window until all required occurrences are present, then contract it as far as the valid condition allows. Counts are essential because a set cannot represent duplicate requirements. Both boundaries move only forward.

A Closer Look

The window is valid when it contains every required occurrence from the target, including duplicates. A count table begins with the target requirements, and `missing` stores how many required character occurrences are still absent.

When the right edge adds a character whose remaining count is positive, `missing` decreases. Every added character then lowers its count; extra copies make the count negative. Once `missing` reaches zero, move the left edge while the window stays valid, recording shorter answers. Removing a character raises its count, and if it becomes positive, a required occurrence has been lost. For `ADOBECODEBANC` and `ABC`, this process first finds `ADOBEC` and later contracts to `BANC`. Both pointers move only forward, so the scan is linear.

BIT MANIPULATION

Lesson 48: Missing Number

Problem at a glance

Difficulty	Approach	Main tool
Easy	XOR cancellation	Array and bits

The Problem

An array contains n distinct values chosen from $[0, n]$. Return the one missing value.

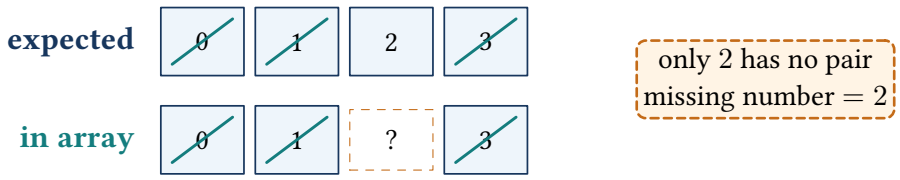
Cancel Equal Values with XOR

XOR every expected value 0 through n and every array value. Each present value occurs twice and cancels to zero; only the missing value remains.

Key idea: the running value is the XOR imbalance so far

After processing index i, the running value equals the XOR of the expected indexes processed and the array values processed, together with n. Values appearing in both groups cancel regardless of order.

Pairs cancel and expose the missing value



Pseudocode

```
answer = n
for i from 0 to n-1:
    answer = answer XOR i XOR nums[i]
return answer
```

Code 48: Complete solution: Missing Number

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    int missingNumber(vector<int>& nums) {
        int answer = static_cast<int>(nums.size());
```

```
        for (int i = 0; i < static_cast<int>(nums.size()); ++i)
            answer ^= i ^ nums[i];
        return answer;
    }
};
```

Time and Space

Time is $O(n)$, space is $O(1)$, and XOR avoids arithmetic overflow. The initial n is required because the loop indexes provide only 0 through $n - 1$.

Walk Through the Example

Trace the example one update at a time

For `[3,0,1]`, begin with $answer = 3$. XOR the pairs $(0, 3), (1, 0), (2, 1)$. Equal values cancel, leaving 2, the only number that appears in the expected range but not in the array.

Why It Works

XOR is associative and self-canceling. Combining every expected value $0 \ldots n$ with every actual value removes all matched pairs and leaves exactly the missing value.

Tests to try

Check a missing zero, a missing n, one-element input, shuffled input, and the largest permitted range.

How to explain it in an interview

I XOR all indexes, all array values, and the endpoint n. Every present number cancels with its expected copy, leaving the missing number with constant extra space.

A Closer Look

The expected values are $0, 1, \ldots, n$, while the array contains every one except the missing value. XOR combines the expected values with the actual values. Every number that appears in both groups cancels because $x \hat{} x = 0$, and zero does not change another value.

Start the answer at n, because loop indexes supply only 0 through $n - 1$. At each index, XOR both the index and the array value. For `[3,0,1]`, the combined expression contains 0, 1, and 3 twice, leaving only 2. XOR is associative and commutative, so the order does not matter. This avoids the overflow risk of a sum formula and uses constant extra space. The important tests are a missing zero and a missing n, since they expose incorrect initialization.

BIT MANIPULATION

Lesson 49: Number of 1 Bits

Problem at a glance

Difficulty	Approach	Main tool
Easy	Clear lowest set bit	Unsigned integer

The Problem

Return the number of set bits in the binary representation of a 32-bit unsigned integer.

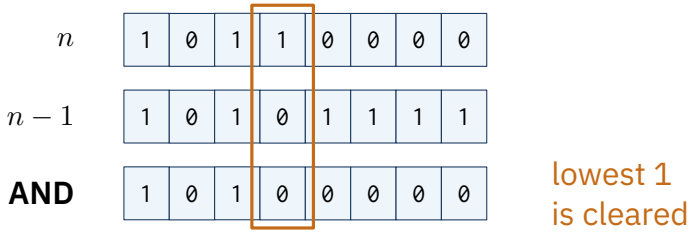
Remove One Set Bit per Iteration

The expression $n \,\&\, (n-1)$ clears the lowest set bit. Repeating it until zero makes the number of iterations equal to the number of ones.

Key idea: each loop removes exactly one one-bit

Subtracting one flips the lowest one to zero and every lower zero to one. ANDing with the original clears that one while preserving every higher bit.

Brian Kernighan's bit-clearing step



Pseudocode

```
count = 0
while n is not zero:
    n = n AND (n - 1)   // clear its lowest set bit
    increment count
return count
```

Code 49: Complete solution: Number of 1 Bits

```cpp
#include <cstdint>

using namespace std;

class Solution {
public:
    int hammingWeight(uint32_t n) {
        int count = 0;
        while (n != 0) {
            n &= n - 1;
            ++count;
        }
        return count;
    }
};
```

Time and Space

The loop runs once per set bit, at most 32 times: $O(1)$ for fixed-width input and $O(1)$ space. Using an unsigned type avoids sign-extension issues.

Walk Through the Example

Trace the example one update at a time

For binary `1011000`, subtracting one gives `1010111`; AND clears the lowest set bit to obtain `1010000`. Repeating produces `1000000` and then zero, so the count is three.

Why It Works

For any nonzero unsigned integer, $n \,\&\, (n-1)$ removes exactly its lowest set bit and changes no higher set bit. The number of iterations equals the number of set bits.

Tests to try

Check zero, one, all 32 bits set, only the highest bit set, and alternating bit patterns.

How to explain it in an interview

I repeatedly clear the lowest set bit with $n \,\&\, (n-1)$ and count how many times that operation is possible.

A Closer Look

Subtracting one from a nonzero binary number changes its lowest 1 bit to 0 and changes all lower 0 bits to 1. ANDing that result with the original number therefore clears exactly the lowest set bit and leaves every higher bit alone.

Repeat `n &= n-1` and count the iterations. Binary `1011000` becomes `1010000`, then `1000000`, then zero, so it contains three 1 bits. The loop runs once per set bit rather than once per bit position. For a fixed 32-bit value, both descriptions are constant time, but the bit-clearing form shows the useful operation directly. The input must be unsigned so shifts or arithmetic around the high bit do not introduce signed-number behavior. Zero correctly skips the loop and returns zero.

PREFIX AND SUFFIX PRODUCTS

Lesson 50: Product of Array Except Self

Problem at a glance

Difficulty	Approach	Main tool
Medium	Prefix and suffix products	Array

The Problem

Return an array where position i contains the product of every input value except `nums[i]`. Do not use division.

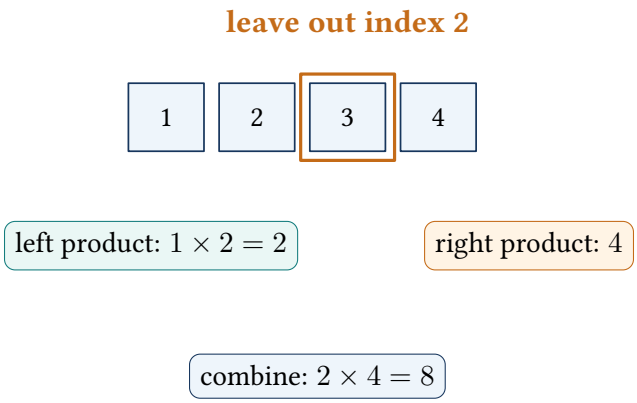
Combine Products from Both Sides

First store the product strictly to the left of each index. A reverse scan keeps the product strictly to the right and multiplies it into the answer.

Key idea: the current answer combines all known outside values

After the forward pass, `answer[i]` is the left prefix product. During the reverse pass, `suffix` is the product right of i, so their product excludes exactly the current value.

Left and right products meet at one index



Pseudocode

```
prefix = 1
for i from left to right:
    answer[i] = prefix
    prefix *= nums[i]
suffix = 1
for i from right to left:
    answer[i] *= suffix
    suffix *= nums[i]
return answer
```

Code 50: Complete solution: Product of Array Except Self

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        vector<int> answer(nums.size(), 1);
        int prefix = 1;
        for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
            answer[i] = prefix; prefix *= nums[i];
        }
        int suffix = 1;
        for (int i = static_cast<int>(nums.size()) - 1; i >= 0; --i) {
            answer[i] *= suffix; suffix *= nums[i];
        }
        return answer;
    }
};
```

Time, Space, and Edge Cases

Time is $O(n)$; excluding output, space is $O(1)$. Zero values require no special case: the products naturally become zero in the appropriate positions.

Walk Through the Example

Trace the example one update at a time

For `[1,2,3,4]`, the left pass writes $[1, 1, 2, 6]$. The right pass multiplies by suffixes $24, 12, 4, 1$, producing $[24, 12, 8, 6]$. Neither pass includes the value at its own index.

Why It Works

Before the right pass, `answer[i]` equals the product strictly left of i. Multiplying by the running product strictly right of i gives exactly the required product of every other position.

Tests to try

Check one zero, two zeroes, negative values, two elements, and products equal to one. Use wider arithmetic if adapting beyond the stated bounds.

How to explain it in an interview

I first store every left-side product in the output. A reverse pass multiplies each entry by its right-side product, achieving linear time without division or extra arrays.

A Closer Look

The restriction against division matters most when zeros appear. Prefix and suffix products avoid that problem completely. First move left to right. At index i, store the product of every value strictly before i, then multiply the running prefix by `nums[i]`.

Next move right to left with a running suffix. Multiply the stored prefix by the product of every value strictly after the index, then update the suffix. For `[1,2,3,4]`, the first pass stores `[1,1,2,6]`. The second pass multiplies by suffixes 24, 12, 4, and 1 to produce `[24,12,8,6]`. One zero makes every answer except its own position zero; two zeros make every answer zero, all without a special branch. The returned array is not counted as auxiliary space, so the algorithm uses constant extra state.

LINKED-LIST POINTERS

Lesson 51: Remove Nth Node From End of List

Problem at a glance

Difficulty	Approach	Main tool
Medium	Fixed pointer gap	Linked list

The Problem

Remove the n-th node from the end of a singly linked list and return the head.

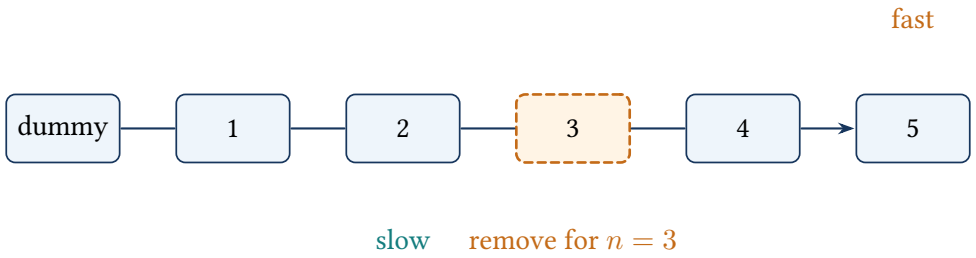
Create an n-Node Gap

A dummy node handles deletion of the original head. Advance `fast` $n + 1$ steps from the dummy, then move both pointers until `fast` reaches null. `slow` then precedes the node to delete.

Key idea: fast stays n+1 links ahead of slow

The fixed gap makes the remaining distance after `fast` reaches null equal to the required offset from the end. So `slow->next` is the target node.

The pointer gap locates the previous



Pseudocode

```
dummy.next = head; fast = slow = dummy
move fast forward n + 1 steps
while fast exists:
    move fast and slow one step
slow.next = slow.next.next
return dummy.next
```

Code 51: Complete solution: Remove Nth Node From End

```cpp
using namespace std;

class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode dummy(0); dummy.next = head;
        ListNode* fast = &dummy; ListNode* slow = &dummy;
        for (int i = 0; i <= n; ++i) fast = fast->next;
        while (fast != nullptr) { fast = fast->next; slow = slow->next; }
        slow->next = slow->next->next;
        return dummy.next;
    }
};
```

Time and Space

One pass uses $O(L)$ time and $O(1)$ space. The dummy node makes removing the first real node identical to every other deletion.

Walk Through the Example

Trace the example one update at a time

For `1->2->3->4->5` with $n = 2$, advance `fast` three links beyond the dummy. Move both pointers until `fast` becomes null; `slow` then precedes node 4, which is bypassed.

Why It Works

The initial gap is $n + 1$ links. Moving both pointers keeps it, so when `fast` reaches null, `slow.next` is exactly the n-th node from the end.

Tests to try

Check a one-node list, removing the head, removing the tail, $n = 1$, and n equal to the list length.

How to explain it in an interview

A dummy node eliminates the head-deletion special case. I create a fixed pointer gap, advance both pointers together, and bypass the node after the slow pointer.

A Closer Look

The main difficulty is finding the node to remove without first measuring the list length. Put both pointers at a dummy node before the head. Move the fast pointer $n + 1$ steps so there are exactly n nodes between fast and slow.

Then move both pointers together until fast becomes null. Slow now points to the node immediately before the target, so one link update removes it. The dummy node is important when the first real node

must be deleted. For `1->2->3->4->5` with $n = 2$, slow stops at 3 and changes its next link from 4 to 5. The method assumes n is valid for the list. It uses one pass after creating the gap, runs in $O(L)$ time, and uses constant extra space.

LINKED-LIST POINTERS

Lesson 52: Reorder List

Problem at a glance

Difficulty	Approach	Main tool
Medium	Split, reverse, and weave	Linked list

The Problem

Transform $L_0 \to L_1 \to \cdots \to L_n$ into $L_0 \to L_n \to L_1 \to L_{n-1} \to \cdots$ in place.

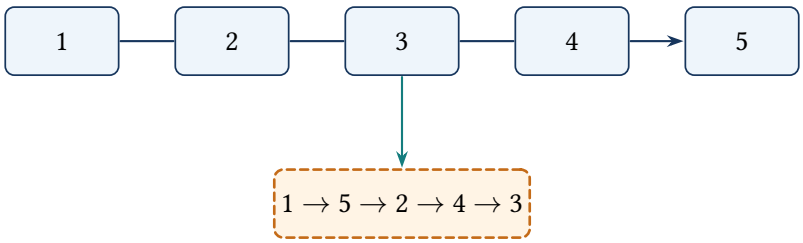
Three Pointer Phases

Use slow and fast pointers to find the middle, reverse the second half, then alternate nodes from the first half and the reversed second half.

Key idea: the woven prefix already has final order

During merging, every node before the two current pointers is fixed in the required outside-in order. Saved next pointers keep both unmerged suffixes.

Reverse the back half, then alternate



Pseudocode

```
find the middle with slow and fast pointers
reverse the second half
first = head; second = reversed half
while second exists:
    save both next pointers
    link first -> second -> next first
    advance both halves
```

Code 52: Complete solution: Reorder List

```cpp
using namespace std;

class Solution {
public:
    void reorderList(ListNode* head) {
        if (head == nullptr || head->next == nullptr) return;
        ListNode *slow = head, *fast = head;
        while (fast->next != nullptr && fast->next->next != nullptr) {
            slow = slow->next; fast = fast->next->next;
        }
        ListNode* second = slow->next; slow->next = nullptr;
        ListNode* previous = nullptr;
        while (second != nullptr) {
            ListNode* next = second->next; second->next = previous;
            previous = second; second = next;
        }
        ListNode* first = head; second = previous;
        while (second != nullptr) {
            ListNode* nextFirst = first->next;
            ListNode* nextSecond = second->next;
            first->next = second; second->next = nextFirst;
            first = nextFirst; second = nextSecond;
        }
    }
};
```

Time and Space

Each phase is linear, so total time is $O(n)$ and space is $O(1)$. Cutting the first half before reversing prevents an accidental cycle.

Walk Through the Example

Trace the example one update at a time

For `1->2->3->4->5`, slow stops at 3. Cut after 3 and reverse `4->5` into `5->4`. Weave 1 with 5, then 2 with 4; the first half's remaining node 3 stays last, producing `1->5->2->4->3`.

Why It Works

The middle split creates two independent lists. Reversal places the original tail at the front of the second half. Each weaving step appends the next unused front node followed by the next unused back node, exactly matching the required order.

Tests to try

Check two nodes, three nodes, even and odd lengths, repeated values, and a one-node list. Verify that the final node points to null and no cycle is introduced.

How to explain it in an interview

I solve the problem in three linear phases: find the middle, reverse the second half, and alternate the two halves. Saved next pointers prevent the unmerged suffixes from being lost.

A Closer Look

Reordering is easier when split into three familiar linked-list operations. First find the middle with slow and fast pointers. Cut the list into two halves. Reverse the second half, then weave one node from the first half with one node from the reversed half.

For `1->2->3->4->5`, the halves become `1->2->3` and `4->5`. Reversing the second gives `5->4`. Weaving produces `1->5->2->4->3`. Before changing either next pointer, save both next nodes so the unprocessed suffixes are not lost. Odd and even lengths differ only in whether the first half has one extra node. No values need to be copied and no array is required. Each phase is linear, so the complete operation is $O(n)$ time and $O(1)$ extra space.

LINKED-LIST POINTERS

Lesson 53: Reverse Linked List

Problem at a glance

Difficulty	Approach	Main tool
Easy	Iterative pointer reversal	Linked list

The Problem

Reverse every link in a singly linked list and return the new head.

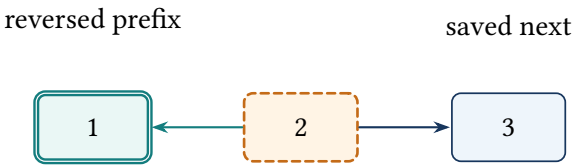
Keep Next Before Rewiring

Keep a reversed prefix and an untouched suffix. Save the current node's next pointer, redirect it toward the prefix, and advance both boundaries.

Key idea: previous heads the fully reversed prefix

Before each iteration, `previous` is a correctly reversed list of all processed nodes, while `current` heads the untouched remainder.

One link changes direction per step



Pseudocode

```
previous = null; current = head
while current exists:
    nextNode = current.next
    current.next = previous
    previous = current
    current = nextNode
return previous
```

Code 53: Complete solution: Reverse Linked List

```cpp
using namespace std;

class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* previous = nullptr;
        while (head != nullptr) {
            ListNode* next = head->next;
            head->next = previous;
            previous = head;
            head = next;
        }
        return previous;
    }
};
```

Time and Space

Time is $O(n)$, space is $O(1)$. Rewiring before saving the next pointer loses the unprocessed suffix.

Walk Through the Example

Trace the example one update at a time

For `1->2->3`, save node 2 before changing node 1 to point to null. Then make 2 point to 1 and 3 point to 2. The final `previous` pointer is the new head 3.

Why It Works

Before each iteration, `previous` heads the reversed prefix and `head` heads the untouched suffix. Saving the suffix link before rewiring keeps all nodes and grows the reversed prefix by one.

Tests to try

Check a null list, one node, two nodes, repeated values, and a long list where pointer loss would be visible.

How to explain it in an interview

I keep a reversed prefix and an unprocessed suffix. Each iteration saves the next node, reverses one pointer, and advances both boundaries.

A Closer Look

Reversing a list means redirecting each next pointer toward the node that was previously before it. Three pointers are enough: `previous`, `current`, and a saved `next` node.

At each step, save `current->next` before overwriting it. Point current backward to previous, then advance previous and current. Starting with `1->2->3`, the first update makes `1->null`; the second makes `2->1`; the third makes `3->2->1`. When current becomes null, previous is the new head. Forgetting to save the forward link loses the rest of the list after the first update. The empty list and a one-node list pass through the same loop without special restructuring. Every node is changed once, giving linear time and constant extra space.

BIT MANIPULATION

Lesson 54: Reverse Bits

Problem at a glance

Difficulty	Approach	Main tool
Easy	Fixed-width bit scan	Unsigned integer

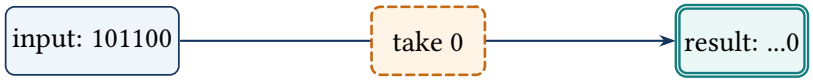
The Problem

Reverse the order of all 32 bits in an unsigned integer.

Key idea: the result contains the reversed processed suffix

After i iterations, the result's lowest i construction steps contain the first i input bits in reverse order. Shifting before insertion opens the next output position.

Bits leave the right and enter the left-to-right result



Pseudocode

```
answer = 0
repeat 32 times:
    answer = (answer << 1) OR (n AND 1)
    n = n >> 1
return answer
```

Code 54: Complete solution: Reverse Bits

```cpp
#include <cstdint>

using namespace std;

class Solution {
public:
    uint32_t reverseBits(uint32_t n) {
        uint32_t result = 0;
        for (int i = 0; i < 32; ++i) {
            result = (result << 1) | (n & 1U);
            n >>= 1;
        }
        return result;
    }
};
```

Time and Space

Exactly 32 iterations give $O(1)$ time and space. Processing all positions, including leading zeros, is essential.

Walk Through the Example

Trace the example one update at a time

Using an 8-bit illustration, `00010110` is consumed from the right. Appending its bits to the result produces `01101000`. The real implementation repeats the same operation exactly 32 times.

Why It Works

At iteration i, the result contains the lowest i input bits in reversed order. Shifting before appending makes room for the next bit; exactly 32 rounds also place original leading zeroes correctly.

Tests to try

Check zero, one, only the highest bit, all bits set, and alternating bit patterns.

How to explain it in an interview

I read one low bit at a time, shift the result left, append that bit, and shift the input right. A fixed 32 iterations keep leading zeroes.

A Closer Look

The result has exactly 32 bits, so the loop should always run 32 times, including leading zeros. On each iteration, shift the result left to make room for one new bit. Copy the lowest bit of the input with `n & 1`, then shift the input right.

This reads the input from right to left while building the answer from left to right. For a small pattern such as `00000101`, the read bits are 1, 0, 1, followed by zeros, and they appear at the opposite end of the result. Use an unsigned 32-bit type so the right shift inserts zeros instead of extending a sign bit. A loop that stops when n becomes zero would fail to place the remaining leading zeros in their correct reversed positions unless the result position were tracked separately.

MATRIX TRANSFORMATIONS

Lesson 55: Rotate Image

Problem at a glance

Difficulty	Approach	Main tool
Medium	Transpose and reverse rows	Square matrix

The Problem

Rotate an $n \times n$ matrix 90 degrees clockwise in place.

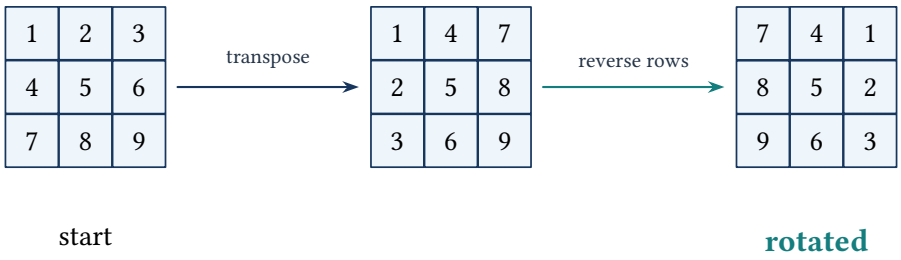
Two In-place Reflections

Transposing swaps (r, c) with (c, r). Reversing every row then moves the transposed columns into their clockwise positions.

Key idea: each swap fixes a symmetric pair

The transpose loop visits only cells above the diagonal, so every off-diagonal pair is exchanged exactly once. Row reversal then completes the mapping $(r, c) \mapsto (c, n - 1 - r)$.

Transpose, then reverse each row



Pseudocode

```
for every pair above the main diagonal:
    swap matrix[row][col] with matrix[col][row]
for every row:
    reverse the row
```

Code 55: Complete solution: Rotate Image

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int n = static_cast<int>(matrix.size());
        for (int r = 0; r < n; ++r)
            for (int c = r + 1; c < n; ++c)
                swap(matrix[r][c], matrix[c][r]);
        for (auto& row : matrix) reverse(row.begin(), row.end());
    }
};
```

Time and Space

Every cell is touched a constant number of times: $O(n^2)$ time and $O(1)$ extra space.

Walk Through the Example

Trace the example one update at a time

For $\begin{smallmatrix}1&2&3\\4&5&6\\7&8&9\end{smallmatrix}$, transposition gives $\begin{smallmatrix}1&4&7\\2&5&8\\3&6&9\end{smallmatrix}$. Reversing each row then yields the clockwise rotation $\begin{smallmatrix}7&4&1\\8&5&2\\9&6&3\end{smallmatrix}$.

Why It Works

Transposition maps (r, c) to (c, r). Reversing each row then maps it to $(c, n-1-r)$, which is exactly the coordinate transformation for a 90-degree clockwise rotation.

Tests to try

Check 1×1, 2×2, odd and even dimensions, negative entries, and avoid swapping diagonal pairs twice.

How to explain it in an interview

I decompose the rotation into two in-place symmetries: transpose across the main diagonal, then reverse every row.

A Closer Look

A square matrix can be rotated without another matrix by separating the movement into two simple operations. Transpose across the main diagonal, then reverse every row. The transpose changes rows into columns; the row reversal puts those columns in clockwise order.

For $\begin{smallmatrix}1&2&3\\4&5&6\\7&8&9\end{smallmatrix}$, transposing gives $\begin{smallmatrix}1&4&7\\2&5&8\\3&6&9\end{smallmatrix}$. Reversing each row gives $\begin{smallmatrix}7&4&1\\8&5&2\\9&6&3\end{smallmatrix}$. During the transpose, swap only cells above the diagonal with their reflected partners; visiting both halves would swap every pair twice and undo the work. The matrix must be square for this direct in-place mapping. Time is $O(n^2)$, with constant auxiliary space.

TREE COMPARISON AND SERIALIZATION

Lesson 56: Same Tree

Problem at a glance

Difficulty	Approach	Main tool
Easy	Paired recursion	Two binary trees

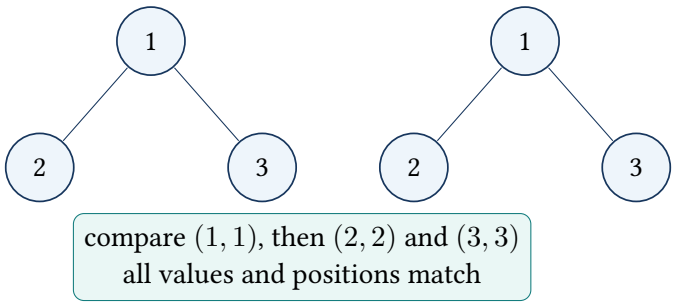
The Problem

Return whether two binary trees have identical structure and equal values at matching nodes.

Key idea: each call compares matching subtrees

Two null nodes match; exactly one null node fails; otherwise equal roots and matching left and right subtree pairs are needed and enough.

Matching nodes are compared in pairs



Pseudocode

```
same(p, q):
    if both are null: return true
    if exactly one is null or values differ: return false
    return same(p.left, q.left) AND same(p.right, q.right)
```

Code 56: Complete solution: Same Tree

```cpp
using namespace std;

class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (p == nullptr || q == nullptr) return p == q;
        return p->val == q->val && isSameTree(p->left, q->left) &&
               isSameTree(p->right, q->right);
    }
};
```

Time and Space

At most all nodes are compared: $O(n)$ time and $O(h)$ recursion space.

Walk Through the Example

Trace the example one update at a time

Compare roots first. If both contain 1, recursively compare their left children and then their right children. A missing child on only one side fails immediately even if every remaining value matches.

Why It Works

Two rooted trees are equal exactly when their roots match and their matching left and right subtrees are equal. The base cases handle matching nulls and structural mismatches, proving the recursive test.

Tests to try

Check two null trees, one null tree, equal values with different shapes, one differing leaf, and identical skewed trees.

How to explain it in an interview

I compare matching nodes. Two nulls match; one null or unequal values fail; otherwise both matching child pairs must also match.

A Closer Look

Two trees are the same only when their structure and values match at every corresponding position. The recursive cases follow that statement directly. Two null pointers match. One null pointer and one real node do not. Two real nodes match only if their values match and both pairs of children match.

The recursion should compare left with left and right with right. It cannot ignore missing children, because trees with the same values in different shapes are not the same. The method stops as soon as a mismatch is found. When the trees match, every corresponding node is visited once, so time is $O(n)$. The call stack uses $O(h)$ space, where h is the larger tree height. Testing two empty trees, one empty tree, unequal roots, and equal values with different shapes covers the important base cases.

TREE COMPARISON AND SERIALIZATION

Lesson 57: Serialize and Deserialize Binary Tree

Problem at a glance

Difficulty	Approach	Main tool
Hard	Preorder with null markers	Tree and string stream

The Problem

Convert a binary tree into a string and reconstruct the identical tree from that string.

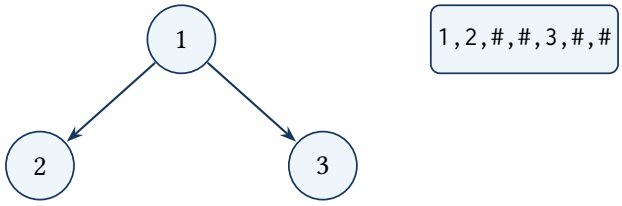
Keep Shape with Null Tokens

Preorder values alone are ambiguous. Emit a marker for every null child. The decoder consumes tokens in the same recursive order, so each value receives the exact left and right structure it originally had.

Key idea: each recursive decode consumes one complete subtree

The current token is the root description. A null marker consumes one token; a value consumes itself followed by exactly the tokens for its left and right subtrees.

Null markers make tree shape explicit



Pseudocode

```
serialize(node):
    if null: append null marker
    else append value, serialize left, serialize right
deserialize(tokens):
    read next token
    if null marker: return null
    node = new node(token)
    node.left = deserialize(tokens)
    node.right = deserialize(tokens)
    return node
```

Code 57: Complete solution: Serialize and Deserialize Tree

```cpp
#include <sstream>
#include <string>

using namespace std;

class Codec {
    void write(TreeNode* node, ostringstream& out) {
        if (node == nullptr) { out << "# "; return; }
        out << node->val << ' '; write(node->left, out); write(node->right, out);
    }
    TreeNode* read(istringstream& in) {
        string token; in >> token;
        if (token == "#") return nullptr;
        TreeNode* node = new TreeNode(stoi(token));
        node->left = read(in); node->right = read(in); return node;
    }
public:
    string serialize(TreeNode* root) { ostringstream out; write(root, out); return out.str(); }
    TreeNode* deserialize(string data) { istringstream in(data); return read(in); }
};
```

Time and Space

Both operations visit every node and null child: $O(n)$ time, $O(n)$ output/tree space, and $O(h)$ recursion space.

Walk Through the Example

Trace the example one update at a time

For a root 1 with left child 2 and right child 3, preorder serialization emits `1 2 # # 3 # #`. During decoding, token 1 creates the root; the next complete token group constructs its left subtree, and the remaining group constructs its right subtree.

Why It Works

Preorder plus null markers is clear. A value token determines one node and recursively consumes exactly two subtree encodings; a null marker ends one child. Working upward from the leaves shows that decoding recreates the original shape and values.

Tests to try

Check an empty tree, one node, negative values, a left-only chain, a right-only chain, repeated values, and a tree whose shape would be ambiguous without null markers.

How to explain it in an interview

I serialize in preorder and write an explicit token for every null pointer. The decoder reads the same grammar recursively, so each call consumes one complete subtree. Both operations are linear.

A Closer Look

Serialization needs to preserve structure, not only values. Preorder traversal records a node before its children, and a special null marker records every missing child. Those markers make the representation unambiguous.

For a node, write its value, then serialize the left and right subtrees. For a null pointer, write a marker such as `N`. Deserialization reads tokens in the same order. A null marker returns a null pointer; a number creates a node whose left and right children are built recursively. A tree containing only values cannot distinguish a left child from a right child, which is why the markers are essential. Negative and multi-digit values need a delimiter, not character-by-character parsing. Both directions process one token per node or null link, so their time is linear in the representation size.

MATRIX STATE

Lesson 58: Set Matrix Zeroes

Problem at a glance

Difficulty	Approach	Main tool
Medium	First-row and first-column markers	Matrix

The Problem

If a matrix cell is zero, set its entire row and column to zero in place.

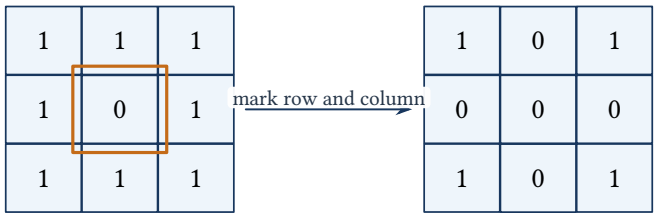
Store Markers Inside the Matrix

Use the first cell of each row and column as a zero marker, plus one Boolean for whether the original first column contained zero. Apply markers after the discovery pass so newly written zeros do not spread incorrectly.

Key idea: markers describe original zero rows and columns

After discovery, `matrix[r][0]` and `matrix[0][c]` are zero exactly when row r or column c must be cleared.

A zero writes one row marker and one column marker



Pseudocode

```
record whether first row or first column originally contains zero
use first row and first column to mark zeroed rows and columns
zero all marked interior cells
zero the first row and first column when originally required
```

Code 58: Complete solution: Set Matrix Zeroes

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    void setZeroes(vector<vector<int>>& a) {
        int rows = a.size(), cols = a[0].size(); bool firstCol = false;
        for (int r = 0; r < rows; ++r) {
            if (a[r][0] == 0) firstCol = true;
            for (int c = 1; c < cols; ++c)
                if (a[r][c] == 0) a[r][0] = a[0][c] = 0;
        }
        for (int r = rows - 1; r >= 0; --r) {
            for (int c = cols - 1; c >= 1; --c)
                if (a[r][0] == 0 || a[0][c] == 0) a[r][c] = 0;
            if (firstCol) a[r][0] = 0;
        }
    }
};
```

Time and Space

Time is $O(mn)$, extra space $O(1)$. The separate first-column flag resolves the shared marker at `matrix[0][0]`.

Walk Through the Example

Trace the example one update at a time

For `[[1,1,1],[1,0,1],[1,1,1]]`, discovery writes zero into the first cell of row 1 and column 1. The second pass clears every interior cell whose row or column marker is zero, producing `[[1,0,1],[0,0,0],[1,0,1]]`.

Why It Works

Markers are written only while reading the original matrix. Once discovery is complete, a cell must become zero exactly when its row marker or column marker is zero. The separate first-column flag resolves the dual meaning of the top-left marker.

Tests to try

Check one cell, one row, one column, a zero at $(0, 0)$, zeros only in the first row or first column, multiple zeroes, and an all-zero matrix.

How to explain it in an interview

I reuse the first row and first column as marker storage, which avoids allocating separate Boolean arrays. I delay mutation of the remaining cells until all original zero locations have been recorded.

A Closer Look

Using the first row and first column as marker storage avoids a second matrix. When a zero is found at (r, c), set the first cell of row r and the first cell of column c to zero. Later passes use those markers to clear interior cells.

The first row and first column need separate Boolean flags because their cells are also being used as storage. Scan them before writing markers. Then mark zeros from the interior, clear interior rows and columns from the markers, and finally clear the first row or column if its original flag was true. Changing cells to zero during the first scan without this plan can create new zeros that incorrectly spread farther. Every cell is visited a constant number of times, giving $O(mn)$ time and constant auxiliary space.

MATRIX STATE

Lesson 59: Spiral Matrix

Problem at a glance

Difficulty	Approach	Main tool
Medium	Shrinking boundaries	Matrix

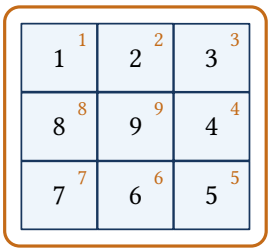
The Problem

Return matrix values in clockwise spiral order.

Key idea: the boundaries enclose every unvisited cell

At the start of each round, all cells outside `top`, `bottom`, `left`, and `right` have been emitted exactly once. Traversing the perimeter and shrinking its sides keeps this condition.

Four boundaries peel one rectangular layer



read the outside: $1, 2, 3, 4, 5, 6, 7, 8$
then read the center: 9

Pseudocode

```
set top, bottom, left, right boundaries
while boundaries are valid:
    read top left-to-right; increment top
    read right top-to-bottom; decrement right
    if rows remain: read bottom right-to-left; decrement bottom
    if columns remain: read left bottom-to-top; increment left
return output
```

Code 59: Complete solution: Spiral Matrix

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& a) {
        vector<int> out; int top=0, bottom=a.size()-1, left=0, right=a[0].size()-1;
        while (top <= bottom && left <= right) {
```

```
            for (int c=left;c<=right;++c) out.push_back(a[top][c]); ++top;
            for (int r=top;r<=bottom;++r) out.push_back(a[r][right]); --right;
            if (top <= bottom) { for (int c=right;c>=left;--c) out.push_back(a[bottom][c]); --bottom; }
            if (left <= right) { for (int r=bottom;r>=top;--r) out.push_back(a[r][left]); ++left; }
        }
        return out;
    }
};
```

Time and Space

Every cell is emitted once: $O(mn)$ time and $O(1)$ extra space. The checks before the bottom and left traversals prevent duplicates in a single remaining row or column.

Walk Through the Example

Trace the example one update at a time

For a 3×4 matrix, read the top row, right column, bottom row in reverse, and left column upward. The boundaries then describe only the inner row, which is read left-to-right without revisiting a cell.

Why It Works

After each side is emitted, moving its boundary inward removes exactly those cells from the remaining rectangle. Conditional bottom and left passes prevent duplication when that rectangle collapses to one dimension.

Tests to try

Check one cell, one row, one column, $2 \times n$, $m \times 2$, and non-square matrices.

How to explain it in an interview

I keep four boundaries around the unvisited rectangle and consume its sides clockwise. Each cell leaves the rectangle exactly once.

A Closer Look

Four boundaries describe the unvisited rectangle: top, bottom, left, and right. Read the top row from left to right and move top down. Read the right column from top to bottom and move right left. Then, if rows remain, read the bottom row backward; if columns remain, read the left column upward.

The two safety checks are necessary for a single remaining row or column. Without them, those cells are added twice. For a 3×4 matrix, the first round consumes the complete outer ring, and the boundaries then describe only the inner row. Each cell enters the output exactly once, so time is $O(mn)$. Apart from the returned vector, only four indexes are stored.

TREE CONTAINMENT

Lesson 60: Subtree of Another Tree

Problem at a glance

Difficulty	Approach	Main tool
Easy	Tree matching at every root	Binary trees

The Problem

Return whether `subRoot` occurs as an identical subtree within `root`.

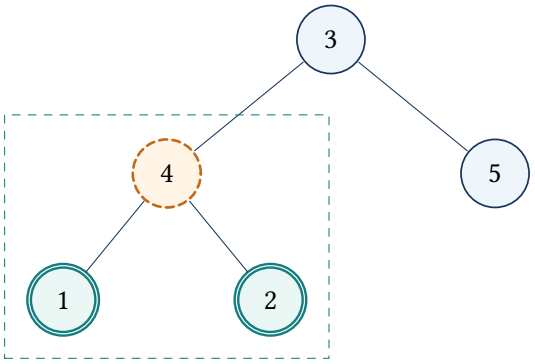
Separate Search from Equality

At every main-tree node, test full tree equality. If it fails, recursively search the left and right subtrees.

Key idea: every possible matching root is examined

The search call returns true exactly when a matching root occurs in its current subtree. The equality helper verifies both values and null structure.

Choose a possible root, then compare its full shape



Pseudocode

```
isSubtree(root, subRoot):
    if subRoot is null: return true
    if root is null: return false
    if sameTree(root, subRoot): return true
    return isSubtree(root.left, subRoot)
           OR isSubtree(root.right, subRoot)
```

Code 60: Complete solution: Subtree of Another Tree

```cpp
using namespace std;

class Solution {
    bool same(TreeNode* a, TreeNode* b) {
        if (a == nullptr || b == nullptr) return a == b;
        return a->val == b->val && same(a->left,b->left) && same(a->right,b->right);
    }
public:
    bool isSubtree(TreeNode* root, TreeNode* subRoot) {
        if (root == nullptr) return false;
        return same(root, subRoot) || isSubtree(root->left, subRoot) ||
               isSubtree(root->right, subRoot);
    }
};
```

Time and Space

Worst-case time is $O(mn)$ when equality is retried at many similar nodes; recursion uses $O(h)$ combined depth. Hashing subtrees can improve repeated comparison but is more complex.

Walk Through the Example

Trace the example one update at a time

If `subRoot` begins with value 4, visit every value-4 node in the larger tree. At each choice, compare the complete left and right shapes; one extra descendant is enough to reject that choice.

Why It Works

Any valid subtree must begin at some node of the larger tree. The outer recursion examines every possible root, and the equality helper accepts exactly when the entire rooted structure and all values match.

Tests to try

Check identical trees, a one-node subtree, repeated choice values, matching values with different shape, and a possible subtree larger than the host.

How to explain it in an interview

I treat every node as a possible subtree root and run an exact tree-equality check there. If it fails, I continue recursively on both child subtrees.

A Closer Look

At each node of the main tree, ask whether a tree starting there is exactly the target subtree. This uses two recursive functions with different jobs. The outer function searches possible starting nodes; the inner function checks whether two trees match completely.

If the inner comparison succeeds, return true. Otherwise search the current node's left and right subtrees. A null target is a subtree of any tree, while a non-null target cannot be found inside a null main tree. The equality helper must check both values and structure, just like Same Tree. In the worst case, many candidate roots have the target's root value and the comparison repeats, leading to $O(mn)$ time. The direct method is still clear and reliable for interview constraints; subtree hashing is a more advanced optimization.

BITWISE ARITHMETIC

Lesson 61: Sum of Two Integers

Problem at a glance

Difficulty	Approach	Main tool
Medium	XOR and carry	Bits

The Problem

Add two integers without using + or −.

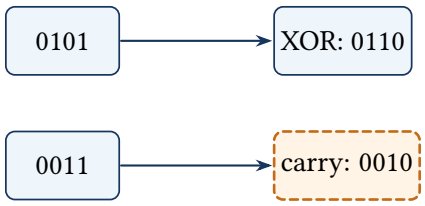
Separate Partial Sum and Carry

XOR adds bits without carry. AND finds positions generating carry, shifted one place left. Repeat until no carry remains.

Key idea: partial sum plus carry equals the original total

At every iteration, `a` contains the carry-free partial sum and `b` contains all pending carries. Redistributing them with XOR and AND keeps their mathematical sum.

XOR writes sum bits; AND creates carries



Pseudocode

```
while b is not zero:
    carry = (a AND b) << 1
    a = a XOR b
    b = carry
return a
```

Code 61: Complete solution: Sum of Two Integers

```cpp
#include <cstdint>

using namespace std;

class Solution {
public:
    int getSum(int a, int b) {
```

```
        uint32_t x = static_cast<uint32_t>(a);
        uint32_t y = static_cast<uint32_t>(b);
        while (y != 0) { uint32_t carry = (x & y) << 1; x ^= y; y = carry; }
        return static_cast<int>(x);
    }
};
```

Time and Space

For 32-bit integers, at most 32 carry movements give $O(1)$ time and space. Unsigned intermediates make left shifting well-defined.

Walk Through the Example

Trace the example one update at a time

For 5 and 3, XOR gives 6, the sum without carries. AND identifies the shared low bit and shifting produces carry 2. Repeating with 6 and 2 gives partial sum 4 and carry 4, then final result 8.

Why It Works

XOR adds each bit without carry, while shifted AND contains exactly the carries required at the next position. Repeating until no carry remains is is equivalent to ordinary binary addition.

Tests to try

Check zero, two positive values, opposite signs, two negative values, and results near the 32-bit limits.

How to explain it in an interview

I separate binary addition into a carry-free sum using XOR and a carry mask using shifted AND. I repeat until the carry mask becomes zero.

A Closer Look

Binary addition can be expressed without the plus operator. XOR adds two bits without carrying: equal bits produce 0 and different bits produce 1. AND finds positions where both bits are 1, and shifting that result left places the carry in the next position.

Repeat with `sum=a^b` and `carry=(a&b)«1`. Replace `a` with sum and `b` with carry until no carry remains. For 5 and 3, `0101 XOR 0011` gives `0110`, while the carry is `0010`. Further rounds propagate that carry to produce 8. Unsigned intermediate values avoid undefined behavior when a carry enters the sign bit; the final bit pattern can then be converted back to `int`. Fixed-width carry propagation takes a bounded number of rounds.

FREQUENCIES AND HEAPS

Lesson 62: Top K Frequent Elements

Problem at a glance

Difficulty	Approach	Main tool
Medium	Frequency heap	Hash map and min heap

The Problem

Return the k values occurring most often in an array.

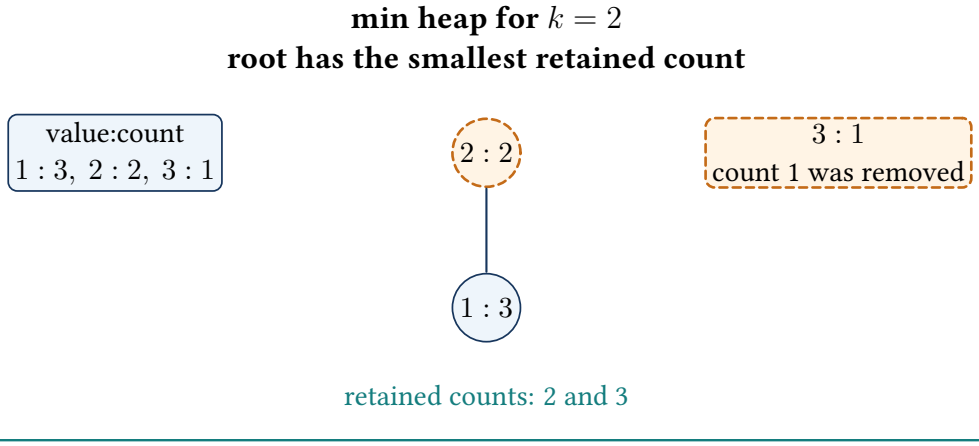
Count, Then Keep Only k Choices

Build a frequency map. A min heap ordered by frequency stores at most k entries; when it grows larger, remove the least frequent choice.

Key idea: the heap contains the best k frequencies seen so far

After each map entry is processed, any discarded element has frequency no greater than every retained element. So the final heap contains exactly a valid top-k set.

The smallest retained frequency is easiest to evict



Pseudocode

```
count every value in a hash map
minHeap = empty
for each (value, frequency):
    push pair into heap
    if heap size exceeds k: pop smallest frequency
return the values remaining in the heap
```

Code 62: Complete solution: Top K Frequent Elements

```cpp
#include <queue>
#include <unordered_map>
#include <utility>
#include <vector>

using namespace std;

class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int,int> count; for (int x:nums) ++count[x];
        using Entry=pair<int,int>;
        priority_queue<Entry,vector<Entry>,greater<Entry>> heap;
        for (const auto& entry : count) {
            heap.push({entry.second, entry.first});
            if ((int)heap.size() > k) heap.pop();
        }
        vector<int> answer; while(!heap.empty()){answer.push_back(heap.top().second);heap.pop();}
        return answer;
    }
};
```

Time and Space

For n values and u distinct values, time is $O(n + u \log k)$, with $O(u + k)$ space. Any output order is accepted.

Walk Through the Example

Trace the example one update at a time

For `[1,1,1,2,2,3]` with $k = 2$, the map stores $1 \mapsto 3$, $2 \mapsto 2$, and $3 \mapsto 1$. A size-two min heap eventually retains frequencies three and two, so the returned values are 1 and 2.

Why It Works

After processing any number of map entries, evicting the smallest frequency whenever the heap exceeds k leaves the best k entries seen so far. Induction over the entries proves the final heap is a valid top-k set.

Tests to try

Check $k = 1$, k equal to the number of distinct values, negative values, tied frequencies, one input value, and heavily duplicated input.

How to explain it in an interview

I count first, then keep only k choices in a min heap. Keeping the weakest retained choice at the top makes replacement efficient and avoids sorting all distinct values.

A Closer Look

First count how many times each value appears. The answer needs only the k largest frequencies, so a min heap of size at most k is enough. Push each value-frequency pair. Whenever the heap grows beyond k, remove the smallest frequency.

After all pairs are processed, every item left in the heap belongs to the top k. The returned order usually does not matter. If there are u unique values, counting takes expected $O(n)$ time and heap work costs $O(u \log k)$. A heap containing all unique values would also work but would use more space and perform more expensive operations. Bucket sorting by frequency is a linear alternative, but the bounded heap is a useful pattern whenever only a small number of best items are requested.

GRID DP

Lesson 63: Unique Paths

Problem at a glance

Difficulty	Approach	Main tool
Medium	Grid dynamic programming	One-dimensional DP

The Problem

Count paths from the top-left to bottom-right of an $m \times n$ grid when each move goes only right or down.

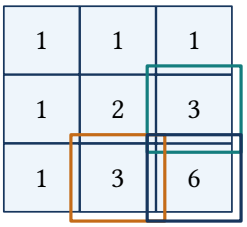
Each Cell Receives Paths from Two Directions

The number of ways into a cell is the sum of ways into the cell above and the cell to the left. A one-dimensional row stores both: before updating `dp[c]` is the value from above, while `dp[c-1]` is from left.

Key idea: dp[c] is the path count for the current processed row

After column c is updated, `dp[c]` contains all paths to the current cell. Entries to its right still represent the previous row.

Paths flow from above and left



from above: 3
from left: 3
new value: $3 + 3 = 6$

Pseudocode

```
dp[0...n-1] = 1
repeat for each remaining row:
    for column from 1 to n-1:
        dp[column] += dp[column - 1]
return dp[n - 1]
```

Code 63: Complete solution: Unique Paths

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> dp(n, 1);
        for (int r = 1; r < m; ++r)
            for (int c = 1; c < n; ++c) dp[c] += dp[c - 1];
        return dp[n - 1];
    }
};
```

Time and Space

Time is $O(mn)$, space is $O(n)$. The first row and first column contain one path each, which explains the initialization to one.

Walk Through the Example

Trace the example one update at a time

For a 3×3 grid, start $[1, 1, 1]$. The second row updates to $[1, 2, 3]$, and the third row becomes $[1, 3, 6]$. The final entry shows six ways to reach the lower-right corner.

Why It Works

Every cell can be entered only from above or from the left. During a left-to-right update, `dp[c]` still stores the count from above and `dp[c-1]` stores the updated count from the left, so their sum is exactly the new state.

Tests to try

Check 1×1, one row, one column, a square grid, and highly rectangular grids.

How to explain it in an interview

I compress the two-dimensional update rule into one row. Each entry adds the ways from above and left, giving $O(mn)$ time and $O(n)$ space.

A Closer Look

Each grid position can be reached only from the cell above or the cell to the left. Therefore the number of paths to a cell is the sum of those two counts. The first row and first column each contain only one path because movement is restricted to right and down.

A one-dimensional array can represent the current row. Initialize every entry to one. For each later row, update from left to right with `dp[col] += dp[col-1]`. Before the update, `dp[col]` is the count from above; `dp[col-1]` is the newly computed count from the left. For a 3×7 grid the final value is 28. This reduces space from $O(mn)$ to $O(n)$ while keeping the same $O(mn)$ time.

STRING SCANNING AND STACKS

Lesson 64: Valid Anagram

Problem at a glance

Difficulty	Approach	Main tool
Easy	Character-frequency equality	Count array

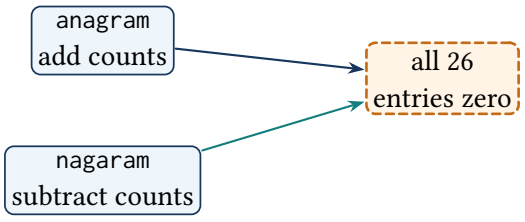
The Problem

Return whether two strings contain exactly the same characters with the same multiplicities.

Key idea: the count array stores the remaining character imbalance

Incrementing for the first string and decrementing for the second leaves all zeros exactly when their multisets are identical.

Matching letters cancel in the frequency table



Pseudocode

```
if lengths differ: return false
counts = 26 zeroes
for i from 0 to n-1:
    increment counts[s[i]]
    decrement counts[t[i]]
return every count is zero
```

Code 64: Complete solution: Valid Anagram

```cpp
#include <array>
#include <string>

using namespace std;

class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) return false;
        array<int,26> count{};
        for (int i=0;i<(int)s.size();++i){++count[s[i]-'a'];--count[t[i]-'a'];}
        for(int value:count) if(value!=0) return false;
        return true;
    }
};
```

Time and Space

Time is $O(n)$, alphabet space is $O(1)$. Length mismatch can be rejected immediately.

Walk Through the Example

Trace the example one update at a time

For `anagram` and `nagaram`, incrementing counts from the first word and decrementing them with the second returns every letter count to zero. For `rat` and `car`, the `t` and `c` entries remain nonzero.

Why It Works

Equal-length lowercase strings are anagrams exactly when every character occurs the same number of times. The combined increment/decrement array measures those frequency differences directly.

Tests to try

Check empty strings, one character, repeated letters, a length mismatch, and strings differing by only one occurrence.

How to explain it in an interview

I compare character multisets with one fixed-size count array. Matching additions and removals cancel; all zeroes at the end prove an anagram.

A Closer Look

Two strings are anagrams when every character appears the same number of times. With lowercase English letters, a fixed array of 26 counts is simpler than sorting. First reject different lengths, because no count adjustment can repair that mismatch.

For each position, increase the count for the character in the first string and decrease the count for the character in the second. At the end, every entry must be zero. Combining the two updates in one loop is safe because the strings have equal length. For `anagram` and `nagaram`, the same letters cancel even though their positions differ. Sorting would take $O(n \log n)$; counting takes $O(n)$ time and constant alphabet space. A larger or Unicode alphabet would use a hash map instead of a 26-cell array.

STRING SCANNING AND STACKS

Lesson 65: Valid Palindrome

Problem at a glance

Difficulty	Approach	Main tool
Easy	Filtered two pointers	String

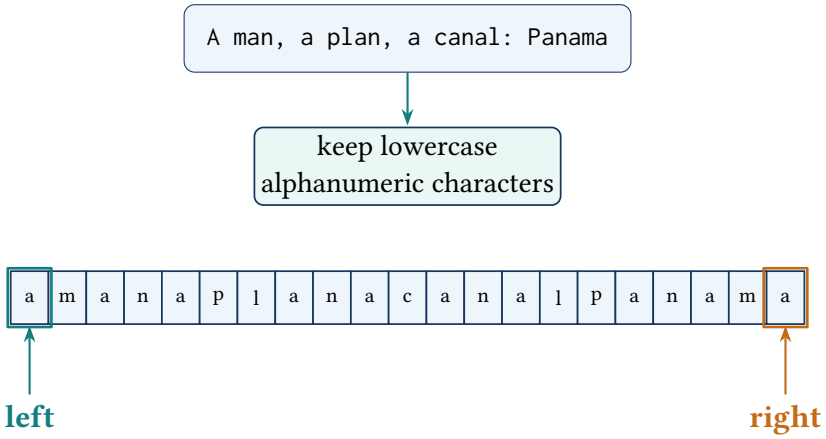
The Problem

Ignoring non-alphanumeric characters and letter case, determine whether a string is a palindrome.

Key idea: all characters outside the pointers already match

Each pointer skips irrelevant characters. When both point to alphanumeric characters, their lowercase forms must match before the search moves inward.

Pointers ignore punctuation and compare useful characters



the end characters match: a = a; move both pointers inward

Pseudocode

```
left = 0; right = n - 1
while left < right:
    skip non-alphanumeric characters on both sides
    if lowercase characters differ: return false
    move both boundaries inward
return true
```

Code 65: Complete solution: Valid Palindrome

```cpp
#include <cctype>
#include <string>

using namespace std;

class Solution {
public:
    bool isPalindrome(string s) {
        int left=0,right=(int)s.size()-1;
        while(left<right){
            while(left<right && !isalnum((unsigned char)s[left])) ++left;
            while(left<right && !isalnum((unsigned char)s[right])) --right;
            if(tolower((unsigned char)s[left])!=tolower((unsigned char)s[right])) return false;
            ++left;--right;
        }
        return true;
    }
};
```

Time and Space

Each pointer moves inward once: $O(n)$ time and $O(1)$ space. Casting to unsigned char makes the classification functions safe.

Walk Through the Example

Trace the example one update at a time

For `"A man, a plan, a canal: Panama"`, the pointers skip spaces and punctuation, compare `a` with `a`, then continue inward case-insensitively until they cross.

Why It Works

Skipping non-alphanumeric characters makes the two pointers identify the next pair in the filtered string. If every mirrored pair matches after case normalization, that filtered string equals its reverse.

Tests to try

Check empty text, punctuation only, mixed case, digits, an early mismatch, and a mismatch near the center.

How to explain it in an interview

I scan inward from both ends, ignoring characters outside the problem's alphabet and comparing the remaining characters without case.

A Closer Look

Move one pointer from each end. Skip any character that is not a letter or digit, then compare lowercase forms of the remaining characters. A mismatch proves the string is not a palindrome; matching characters let both pointers move inward.

For `A man, a plan, a canal: Panama`, spaces and punctuation are ignored, leaving matching pairs from the outside inward. Character functions should receive an `unsigned char` value so non-ASCII negative `char` values do not cause undefined behavior. The pointers must be checked after each skip so an input containing only punctuation does not read outside the string. Every position is passed at most once, giving linear time and constant extra space without constructing a filtered copy.

STRING SCANNING AND STACKS

Lesson 66: Valid Parentheses

Problem at a glance

Difficulty	Approach	Main tool
Easy	Delimiter matching	Stack

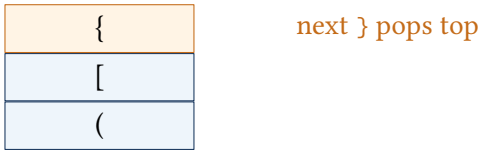
The Problem

Return whether every opening bracket is closed by the correct type in the correct order.

Key idea: the stack contains exactly the unmatched openings

An opening bracket is pushed. A closing bracket must match the stack top, because that is the most recent still-unclosed delimiter.

Nested delimiters close in LIFO order



Pseudocode

```
stack = empty
for character in string:
    if opening delimiter: push it
    else if stack empty or top is not its partner: return false
    else pop stack
return stack is empty
```

Code 66: Complete solution: Valid Parentheses

```cpp
#include <stack>
#include <string>

using namespace std;

class Solution {
public:
    bool isValid(string s) {
        stack<char> openings;
        for(char ch:s){
            if(ch=='('||ch=='['||ch=='{') openings.push(ch);
            else {
                if(openings.empty()) return false;
```

```
                char open=openings.top(); openings.pop();
                if((ch==')'&&open!='(')||(ch==']'&&open!='[')||(ch=='}'&&open!='{')) return ↩
    ↪ false;
            }
        }
        return openings.empty();
    }
};
```

Time and Space

Time is $O(n)$, stack space $O(n)$. An empty stack before a closing bracket and leftover openings after the scan are both invalid.

Walk Through the Example

Trace the example one update at a time

For `([{}])`, push each opening delimiter. The closing brace pops `{`, the closing bracket pops `[`, and the closing parenthesis pops `(`. For `([)]`, the first closing parenthesis conflicts with the stack top `[`.

Why It Works

Nested delimiters must close in reverse opening order, exactly the order supplied by a stack. Every closing delimiter is validated against the only opening delimiter it could legally match.

Tests to try

Check an empty string, one unmatched delimiter, adjacent pairs, deep nesting, crossed types, and leftover opening delimiters.

How to explain it in an interview

I push opening delimiters and require every closing delimiter to match the stack top. The stack must also be empty after the complete scan.

A Closer Look

An opening delimiter creates work that must be completed later, so store it on a stack. A closing delimiter must match the most recent unmatched opening delimiter. This last-in, first-out rule is exactly the stack order.

Push `(`, `[`, or `{` when it appears. For a closing character, first ensure the stack is not empty, then compare it with the top. Pop only after a correct match. At the end, the stack must be empty; otherwise some opening delimiter was never closed. The string `([{}])` works because closures occur in reverse opening order, while `([)]` fails at the first closing parenthesis. Each character causes at most one push or pop, so the solution is linear.

TREES AND WORD SEGMENTATION

Lesson 67: Validate Binary Search Tree

Problem at a glance

Difficulty	Approach	Main tool
Medium	Recursive value bounds	Binary tree

The Problem

Return whether every node satisfies the global binary-search-tree ordering.

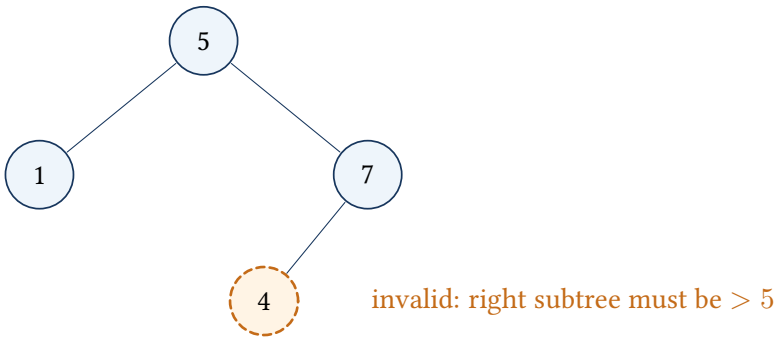
Carry Ancestor Bounds

Local child comparisons are inenough. Each recursive call receives the strict lower and upper bounds imposed by all ancestors.

Key idea: every node in the current subtree must lie inside its bounds

The left call tightens the upper bound to the current value; the right call tightens the lower bound. Strict inequalities reject duplicates.

Ancestor bounds reach deeper than the parent



Pseudocode

```
valid(node, lower, upper):
    if node is null: return true
    if node.value <= lower or node.value >= upper: return false
    return valid(node.left, lower, node.value)
           AND valid(node.right, node.value, upper)
```

Code 67: Complete solution: Validate BST

```cpp
#include <limits>

using namespace std;

class Solution {
    bool valid(TreeNode* node,long long low,long long high){
        if(node==nullptr) return true;
        if(node->val<=low||node->val>=high) return false;
        return valid(node->left,low,node->val)&&valid(node->right,node->val,high);
    }
public:
    bool isValidBST(TreeNode* root){return valid(root,numeric_limits<long long>::lowest(),↩
   ↪ numeric_limits<long long>::max());}
};
```

Time and Space

Every node is checked once: $O(n)$ time, $O(h)$ stack space. Wide bounds avoid collisions with valid `int` endpoints.

Walk Through the Example

Trace the example one update at a time

In `[5,1,4,null,null,3,6]`, node 3 is locally smaller than parent 4 but lies in the root's right subtree, where every value must exceed 5. Passing the inherited lower bound 5 exposes the violation.

Why It Works

Each recursive call carries the complete range allowed by all ancestors. Restricting the upper bound on a left edge and the lower bound on a right edge makes every accepted node satisfy the global BST definition.

Tests to try

Check valid integer endpoints, duplicate values, a deep ancestor violation, a skewed valid BST, and an empty tree.

How to explain it in an interview

I validate each node against inherited lower and upper bounds rather than only its parent. Child calls tighten exactly one side of that range.

A Closer Look

Every node in a BST must satisfy restrictions created by all its ancestors, not only by its parent. Pass a lower and upper bound into the recursive call. The current value must lie strictly between them.

The left child inherits the lower bound and receives the current value as its new upper bound. The right child inherits the upper bound and receives the current value as its new lower bound. Wide integer bounds avoid trouble when a node contains `INT_MIN` or `INT_MAX`. Strict comparisons also reject duplicates when the problem defines a strict BST. A local parent check can miss a value deep in the left subtree that is larger than the root; the carried interval correctly rejects it. Every node is checked once.

TREES AND WORD SEGMENTATION

Lesson 68: Word Break

Problem at a glance

Difficulty	Approach	Main tool
Medium	Prefix segmentation DP	String and hash set

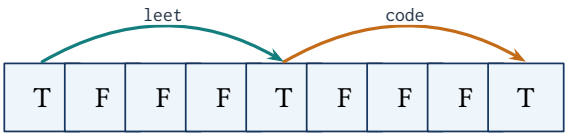
The Problem

Return whether a string can be split into one or more dictionary words. Words may be reused.

Key idea: dp[i] means the prefix ending before i is segmentable

For every true `dp[i]`, trying dictionary words tests whether one legal word can extend that already valid prefix. So every true state corresponds to a complete segmentation.

True prefix states jump across dictionary words



Pseudocode

```
dp[0] = true
for end from 1 to n:
    for start from 0 to end-1:
        if dp[start] and s[start...end) is in dictionary:
            dp[end] = true; break
return dp[n]
```

Code 68: Complete solution: Word Break

```cpp
#include <string>
#include <unordered_set>
#include <vector>

using namespace std;

class Solution {
public:
    bool wordBreak(string s,vector<string>& words){
        unordered_set<string> dict(words.begin(),words.end());
        vector<bool> dp(s.size()+1,false); dp[0]=true;
        for(int end=1;end<=(int)s.size();++end)
            for(int start=0;start<end;++start)
```

```
                if(dp[start]&&dict.count(s.substr(start,end-start))){dp[end]=true;break;}
            return dp[s.size()];
        }
};
```

Time and Space

The direct prefix DP uses $O(n^2)$ transitions plus substring construction cost; space is $O(n+D)$. Limiting starts by maximum word length improves the constant factors.

Walk Through the Example

Trace the example one update at a time

For `leetcode` with dictionary `[leet,code]`, begin with $dp[0] = true$. The word `leet` makes $dp[4]$ true. From that reachable boundary, `code` makes $dp[8]$ true, proving the full string can be segmented.

Why It Works

A true $dp[end]$ is created only by a true earlier boundary followed by a dictionary word, so every true state has a valid segmentation. The reverse is also true:, the final word of any valid segmentation begins at some true earlier boundary, so the transition discovers every possible solution.

Tests to try

Check an empty prefix, one dictionary word, repeated word use, overlapping choices such as `cars`, a long almost-valid suffix, and an input with no valid first word.

How to explain it in an interview

I use prefix dynamic programming because a locally valid word choice may block the suffix. Each true index records that the entire prefix is valid, allowing all later dictionary endings to build on it.

A Closer Look

Let `dp[i]` mean that the prefix ending just before index i can be split into dictionary words. Start with `dp[0]=true`, representing the empty prefix. From each reachable position, test dictionary words or later end positions and mark a new prefix when the intervening substring is a word.

For `leetcode` with `leet` and `code`, the reachable positions become 0, 4, and 8. A word match from an unreachable position must not be used, because the text before it has no valid split. A hash set gives fast dictionary lookup, though substring construction still has a cost. The basic index-based solution is often described as $O(n^2)$ plus substring work. The key idea is that each Boolean summarizes all possible segmentations of one prefix.

TRIE-GUIDED GRID SEARCH

Lesson 69: Word Search II

Problem at a glance

Difficulty	Approach	Main tool
Hard	Trie-pruned backtracking	Trie and grid

The Problem

Find all dictionary words that can be formed by adjacent grid cells without reusing a cell within one word.

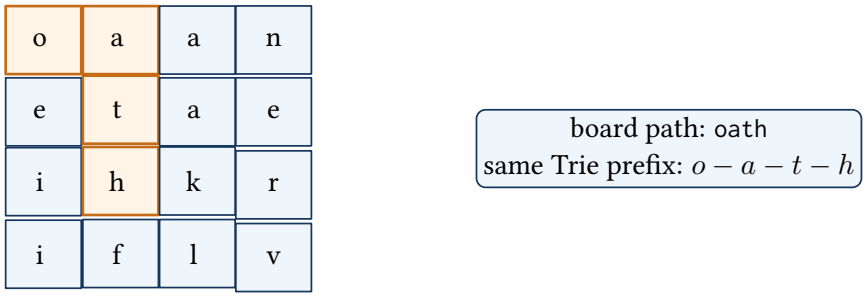
Share Prefix Work with a Trie

Insert all words into a Trie. DFS from each cell follows only Trie edges that exist; impossible prefixes stop immediately. Store a complete word at terminal nodes and clear it after reporting to avoid duplicates.

Key idea: the DFS path equals the Trie prefix at its node

Every active board path uses distinct marked cells and spells exactly the characters from the Trie root to the current Trie node. Missing edges safely prune every dictionary word sharing that impossible prefix.

Board paths and Trie prefixes advance together



Pseudocode

```
insert every word into a trie
for every board cell:
    DFS(cell, trie root)
DFS(cell, trie node):
    stop if outside, visited, or character edge is absent
    move to child; report and clear any completed word
    mark cell, explore four neighbors, then restore cell
return reported words
```

Code 69: Complete solution: Word Search II

```cpp
#include <array>
#include <string>
#include <vector>

using namespace std;

class Solution {
    struct Trie { array<Trie*,26> next{}; string word; };
    void search(vector<vector<char>>& b,int r,int c,Trie* node,vector<string>& out){
        if(r<0||c<0||r==(int)b.size()||c==(int)b[0].size()||b[r][c]=='#') return;
        char ch=b[r][c]; Trie* child=node->next[ch-'a']; if(child==nullptr) return;
        if(!child->word.empty()){out.push_back(child->word);child->word.clear();}
        b[r][c]='#'; search(b,r+1,c,child,out);search(b,r-1,c,child,out);
        search(b,r,c+1,child,out);search(b,r,c-1,child,out); b[r][c]=ch;
    }
public:
    vector<string> findWords(vector<vector<char>>& board,vector<string>& words){
        Trie* root=new Trie();
        for(const auto& word:words){Trie* p=root;for(char ch:word){if(!p->next[ch-'a'])p->next↩
   ↪ [ch-'a']=new Trie();p=p->next[ch-'a'];}p->word=word;}
        vector<string> out; for(int r=0;r<(int)board.size();++r)for(int c=0;c<(int)board[0].↩
   ↪ size();++c)search(board,r,c,root,out); return out;
    }
};
```

Time, Space, and Safety

Trie construction is $O(W)$ for total dictionary characters. Search is bounded by board starts and valid prefix branches; the loose worst case is $O(mn\,4^L)$. Backtracking restores each cell. Production code should own Trie nodes with smart pointers or a node pool.

Walk Through the Example

Trace the example one update at a time

On the standard board, DFS beginning at `o` follows Trie edges `o->a->t->h` and reports `oath`. A neighboring character with no Trie edge stops immediately. Clearing the terminal word prevents another board path from returning the same dictionary word twice.

Why It Works

The active board path and Trie path always spell the same prefix. So a missing Trie child proves that no dictionary word can extend that board path. Every reported terminal node corresponds to a valid non-reusing path, and every valid dictionary path is explored from its first cell.

Tests to try

Check duplicate dictionary entries, words sharing long prefixes, a one-cell board, a word longer than the number of cells, repeated board letters, and several board paths spelling the same word.

How to explain it in an interview

I build one Trie for all words so their shared prefixes are explored once. DFS advances through both the board and Trie, marks the current cell, and restores it during backtracking. Missing Trie edges provide the crucial pruning.

A Closer Look

Searching separately from every grid cell repeats prefix work for every word. A Trie shares those prefixes. Insert all words, then start backtracking from each cell whose character is a child of the Trie root.

During DFS, the current Trie node tells which next letters can still lead to a word. A missing child stops the branch immediately. When a Trie node stores a complete word, add it to the answer and clear or mark that stored word so the same result is not returned twice. Temporarily mark the board cell as visited, explore four neighbors, then restore it for other paths. Removing exhausted Trie branches is an optional pruning improvement. The important distinction is that board state prevents cell reuse in one path, while Trie state prevents searching prefixes that belong to no requested word.

TRIE-GUIDED GRID SEARCH

Lesson 70: Word Search

Problem at a glance

Difficulty	Approach	Main tool
Medium	Grid backtracking	Matrix and recursion

The Problem

Return whether one word can be formed from adjacent cells without reusing a cell in the same path.

Key idea: the marked path spells the matched word prefix

At recursion index i, all earlier characters have been matched by distinct adjacent cells. The current cell must equal character i, after which it is temporarily marked during all four choices.

Backtracking explores and restores a board path

A	B	C	E
S	F	C	S
A	D	E	E

matched path: `ABCCED`
moves: right, right, down, down, left
restore every cell when returning

Pseudocode

```
for every board cell:
    if DFS(cell, word index 0): return true
DFS(cell, index):
    if index equals word length: return true
    reject invalid, visited, or mismatching cells
    mark cell; search four neighbors for index + 1
    restore cell and return whether any branch succeeded
return false
```

Code 70: Complete solution: Word Search

```cpp
#include <string>
#include <vector>

using namespace std;

class Solution {
    bool dfs(vector<vector<char>>& b,const string& w,int i,int r,int c){
        if(i==(int)w.size()) return true;
```

```
        if(r<0||c<0||r==(int)b.size()||c==(int)b[0].size()||b[r][c]!=w[i]) return false;
        char ch=b[r][c]; b[r][c]='#';
        bool ok=dfs(b,w,i+1,r+1,c)||dfs(b,w,i+1,r-1,c)||dfs(b,w,i+1,r,c+1)||dfs(b,w,i+1,r,c-1)
    ↪ ;
        b[r][c]=ch; return ok;
    }
public:
    bool exist(vector<vector<char>>& board,string word){
        for(int r=0;r<(int)board.size();++r)for(int c=0;c<(int)board[0].size();++c)if(dfs(
    ↪ board,word,0,r,c))return true; return false;
    }
};
```

Time and Space

Worst-case time is $O(mn\,4^L)$, with $O(L)$ recursion space. Restoring the character is required even when a branch fails.

Walk Through the Example

Trace the example one update at a time

To find `ABCCED`, start at the matching `A`, mark it, and recursively follow adjacent `B`, `C`, `C`, `E`, and `D`. A failed branch restores every marked cell so another route may use it.

Why It Works

The DFS state records a board position and the next word index. It explores every legal non-reusing path with that prefix; marking prevents cycles and restoration keeps the board for sibling search branches.

Tests to try

Check a one-letter word, a word longer than the board, repeated letters, paths requiring turns, a tempting path that reuses a cell, and no match.

How to explain it in an interview

I start backtracking from every possible first character. Each recursive step matches one letter, temporarily marks the cell, explores four neighbors, and restores the cell afterward.

A Closer Look

Try to match the word one character at a time. A recursive state contains the grid position and the index of the next required character. If the index reaches the word length, the complete word has been matched.

Reject a state when it is outside the grid, the cell is already used in this path, or its character differs. Otherwise temporarily mark the cell, explore the four neighbors for the next character, and restore the cell before returning. Restoration is essential because a cell used by one attempted path may be needed

by another. The outer loops try every cell as a possible start. Simple checks such as a word longer than the number of cells can fail early. The worst case is exponential because many matching prefixes may branch, but pruning happens immediately on every mismatch.

BINARY SEARCH AND SUBSEQUENCES

Lesson 71: Search in Rotated Sorted Array

Problem at a glance

Difficulty	Approach	Main tool
Medium	Modified binary search	Array

The Problem

Find a target in a rotated sorted array of distinct values and return its index, or -1.

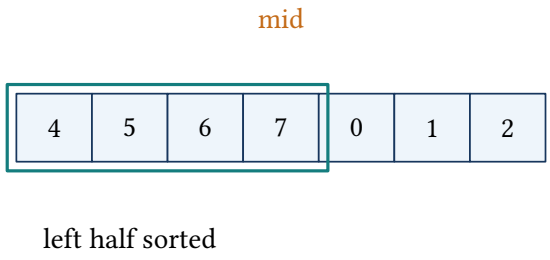
One Half Is Always Sorted

At each midpoint, at least one side is normally ordered. Check whether the target lies inside that sorted value range; keep that half if it does, otherwise discard it.

Key idea: the target, if present, remains inside the boundaries

The sorted-half range test identifies a half that cannot contain the target. Discarding only that half keeps every possible target index.

Use the sorted half to choose safely



Pseudocode

```
left = 0; right = n - 1
while left <= right:
    middle = midpoint
    if nums[middle] is target: return middle
    identify the sorted half
    if target lies inside that half: keep it
    else: keep the other half
return -1
```

Code 71: Complete solution: Search in Rotated Sorted Array

```cpp
#include <vector>

using namespace std;

class Solution {
public:
    int search(vector<int>& nums,int target){
        int left=0,right=(int)nums.size()-1;
        while(left<=right){int mid=left+(right-left)/2;if(nums[mid]==target)return mid;
            if(nums[left]<=nums[mid]){
                if(nums[left]<=target&&target<nums[mid])right=mid-1;else left=mid+1;
            }else{
                if(nums[mid]<target&&target<=nums[right])left=mid+1;else right=mid-1;
            }
        } return -1;
    }
};
```

Time and Space

Each comparison removes half the range: $O(\log n)$ time and $O(1)$ space. The distinct-value condition avoids ambiguous equal boundaries.

Walk Through the Example

Trace the example one update at a time

For `[4,5,6,7,0,1,2]` and target 0, the first midpoint is value 7. The left half is sorted, but 0 is not in $[4, 7)$, so discard it. In the remaining range, binary search reaches value 0 at index 4.

Why It Works

At least one half around the midpoint is sorted. If the target lies inside that half's inclusive/exclusive value bounds, the opposite half cannot contain it; otherwise the sorted half can be discarded. So the target is keepd whenever it exists.

Tests to try

Check no rotation, rotation by one position, target at either boundary, target at the pivot, two elements, one element, and a missing target.

How to explain it in an interview

I first identify the sorted half, then use ordinary value-range reasoning to decide whether the target belongs there. This retains binary search's logarithmic elimination despite the rotation.

A Closer Look

At least one half of the current interval is normally sorted, even though the whole array is rotated. Compare the middle value with the boundaries to decide which half has normal order. Then ask whether the target lies inside that sorted value range.

If the left half is sorted and the target is between the left value and the middle value, discard the right half; otherwise discard the left. Apply the mirror rule when the right half is sorted. For `[4,5,6,7,0,1,2]` and target 0, the first middle is 7. The left half is sorted but does not contain 0, so the search moves right and finds the target. Careful inclusive and exclusive comparisons prevent losing a boundary target. Each decision halves the search range, so time is logarithmic when values are distinct.

BINARY SEARCH AND SUBSEQUENCES

Lesson 72: Longest Increasing Subsequence

Problem at a glance

Difficulty	Approach	Main tool
Medium	Subsequence DP	Array

The Problem

Return the length of the longest strictly increasing subsequence. Chosen positions retain order but need not be adjacent.

Key idea: dp[i] is the best increasing subsequence ending at i

Every previous $j < i$ with $nums[j] < nums[i]$ can extend its best subsequence by the current value. Taking the maximum considers every possible penultimate element.

A current value extends compatible earlier endings

read highlighted positions left to right: $2 < 3 < 7 < 18$

Pseudocode

```
dp[i] = 1 for every position
for i from 0 to n-1:
    for j from 0 to i-1:
        if nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j] + 1)
return maximum dp value
```

Code 72: Complete solution: Longest Increasing Subsequence

```cpp
#include <algorithm>
#include <vector>

using namespace std;

class Solution {
public:
    int lengthOfLIS(vector<int>& nums){
        vector<int> dp(nums.size(),1); int answer=1;
        for(int i=0;i<(int)nums.size();++i){
            for(int j=0;j<i;++j) if(nums[j]<nums[i]) dp[i]=max(dp[i],dp[j]+1);
```

```
            answer=max(answer,dp[i]);
        } return answer;
    }
};
```

Time and Space

The direct update rule uses $O(n^2)$ time and $O(n)$ space. A patience sorting method improves time to $O(n \log n)$, but this table most directly shows the subsequence state.

Walk Through the Example

Trace the example one update at a time

For `[10,9,2,5,3,7,101,18]`, every state begins at one. Value 5 extends the subsequence ending at 2 to length two; value 7 extends either $2, 5$ or $2, 3$ to length three; value 18 extends a length-three state to the final answer four.

Why It Works

Every increasing subsequence ending at index i has some penultimate index $j < i$ with a smaller value, unless its length is one. Testing all such j includes the previous position in a best subsequence, making $dp[i]$ correct.

Tests to try

Check a decreasing array, an increasing array, all equal values, one value, repeated values mixed with growth, negative values, and a best subsequence that does not include the final element.

How to explain it in an interview

I define $dp[i]$ as the best increasing subsequence ending exactly at i. Every smaller earlier value is a legal previous. The maximum over all ending positions is the answer.

A Closer Look

The direct dynamic-programming state is `dp[i]`, the length of the longest increasing subsequence that ends exactly at index i. Every single value can form a length-one subsequence. Look at earlier positions $j < i$; when `nums[j] < nums[i]`, value i can extend the subsequence ending at j.

Take the largest `dp[j]+1` among those valid predecessors, then keep the largest state anywhere as the final answer. In `[10,9,2,5,3,7,101,18]`, the value 7 can extend subsequences ending at 2, 5, or 3; the best choice gives length three at that point. The values need not be adjacent, which separates a subsequence from a subarray. This clear recurrence uses $O(n^2)$ time and $O(n)$ space; the tails/binary-search method is a later optimization.

DP, TREES, AND REACHABILITY

Lesson 73: Climbing Stairs

Problem at a glance

Difficulty	Approach	Main tool
Easy	Fibonacci-style DP	Rolling states

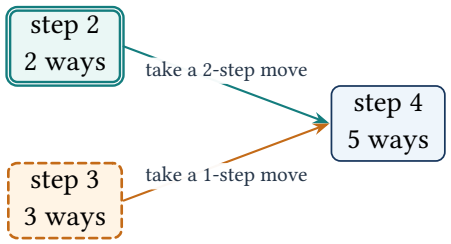
The Problem

Count distinct ways to reach step n using moves of one or two steps.

Key idea: the two rolling values solve the previous two steps

Every route to step i arrives from $i-1$ or $i-2$, and these cases are separate, so their counts add.

Each step receives ways from the previous two steps



$$2 + 3 = 5$$

Pseudocode

```
if n <= 2: return n
oneBack = 2; twoBack = 1
for step from 3 to n:
    current = oneBack + twoBack
    twoBack = oneBack; oneBack = current
return oneBack
```

Code 73: Complete solution: Climbing Stairs

```cpp
using namespace std;

class Solution {
public:
    int climbStairs(int n){
        int twoBack=1,oneBack=1;
        for(int step=2;step<=n;++step){int current=oneBack+twoBack;twoBack=oneBack;oneBack=current;}
        return oneBack;
    }
};
```

Time and Space

Time is $O(n)$, space is $O(1)$. The conceptual count for step zero is one empty way, which makes the update rule work at the boundary.

Walk Through the Example

Trace the example one update at a time

The counts for steps 0 through 5 are $1, 1, 2, 3, 5, 8$. For step 5, every valid climb ends with either a one-step move from step 4 or a two-step move from step 3, so the two preceding counts add to eight.

Why It Works

The final move splits all valid climbs into two separate groups: those arriving from $n - 1$ and those arriving from $n - 2$. Their counts sum, and only the two previous states are required.

Tests to try

Check $n = 1$, $n = 2$, the smallest allowed input, and larger values near the return-type limit.

How to explain it in an interview

This is Fibonacci-style dynamic programming. I keep only the previous two counts because each new state depends exclusively on them.

A Closer Look

To reach step i, the final move must come from step $i - 1$ or step $i - 2$. Therefore the number of ways is the sum of the previous two counts. This is the Fibonacci pattern with problem-specific starting values.

For one step there is one way. For two steps there are two ways: $1 + 1$ or 2. Keep only those two previous values because no older state is needed. Each new step computes their sum, then shifts the pair forward. For five steps the counts are 1, 2, 3, 5, and 8. Defining the base cases clearly prevents the common off-by-one error. The loop uses $O(n)$ time and $O(1)$ space, whereas plain recursion repeats the same smaller stair counts many times.

DP, TREES, AND REACHABILITY

Lesson 74: Invert Binary Tree

Problem at a glance

Difficulty	Approach	Main tool
Easy	Recursive child swap	Binary tree

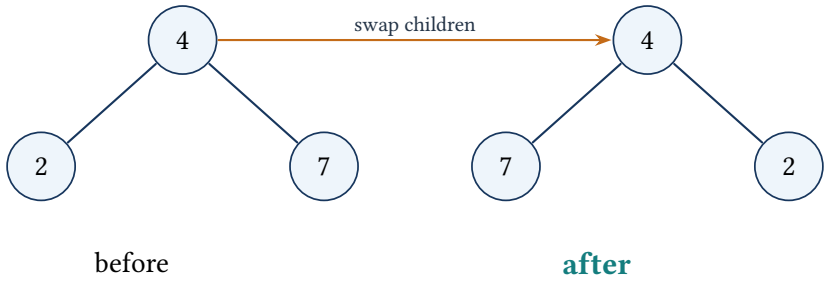
The Problem

Mirror a binary tree by exchanging the left and right child of every node.

Key idea: a completed call returns a fully mirrored subtree

The current node swaps its child pointers, then recursively mirrors both subtrees. Every edge below the node is reversed horizontally once.

Every left and right edge changes sides



Pseudocode

```
invert(node):
    if node is null: return null
    swap node.left and node.right
    invert(node.left)
    invert(node.right)
    return node
```

Code 74: Complete solution: Invert Binary Tree

```cpp
#include <utility>

using namespace std;

class Solution {
public:
    TreeNode* invertTree(TreeNode* root){
        if(root==nullptr) return nullptr;
        swap(root->left,root->right);
```

```
        invertTree(root->left); invertTree(root->right); return root;
    }
};
```

Time and Space

Every node is visited once: $O(n)$ time and $O(h)$ recursion space.

Walk Through the Example

Trace the example one update at a time

For root 4 with children 2 and 7, swap them first. Recursively swap the children below 7 and below 2 as well. Applying the same operation a second time restores the original tree.

Why It Works

A tree is mirrored exactly when every node exchanges its left and right subtrees and both exchanged subtrees are themselves mirrored. The recursive procedure performs this definition at every node.

Tests to try

Check a null tree, one node, a skewed tree, an asymmetric tree, and verify that double inversion returns the original structure.

How to explain it in an interview

I swap each node's child pointers and recursively invert both resulting subtrees. Every node is processed once, with stack depth equal to height.

A Closer Look

Inverting a tree is a local operation repeated at every node. Swap the left and right child pointers, then recursively invert the two child subtrees. A null pointer is the base case.

The order can be swap-then-recurse or recurse-then-swap as long as the calls follow the pointers consistently. For a root 4 with children 2 and 7, the first swap places 7 on the left and 2 on the right; recursion then mirrors the children below them. No new nodes are required because only links change. Every node is visited once, so time is $O(n)$. Recursive stack space is $O(h)$; a queue can perform the same transformation breadth first. A one-node tree and a completely skewed tree are useful tests.

DP, TREES, AND REACHABILITY

Lesson 75: Pacific Atlantic Water Flow

Problem at a glance

Difficulty	Approach	Main tool
Medium	Reverse multi-source DFS	Grid and two reachability sets

The Problem

Return cells from which water can flow to both oceans. Water moves to adjacent cells of equal or lower height; the Pacific touches top and left edges, and the Atlantic touches bottom and right edges.

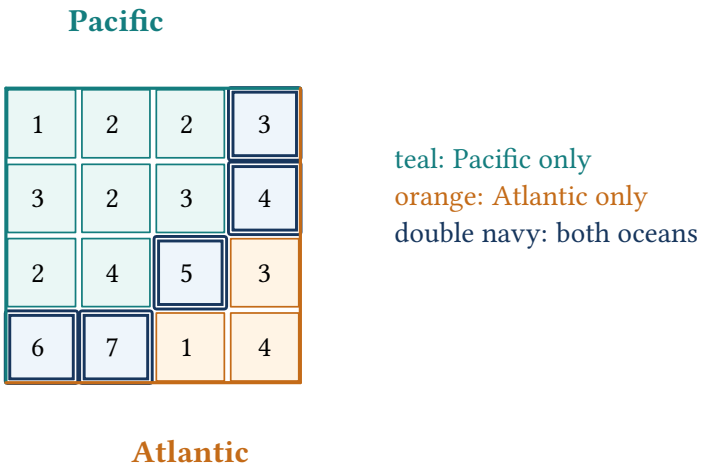
Search Backward from the Oceans

Forward search from every cell repeats work. Reverse the edges: start from each ocean boundary and move to neighbors of equal or greater height. Cells reached by both reverse searches can flow to both oceans in the original direction.

Key idea: reverse-reached cells can flow to the starting ocean

Every reverse step climbs from a cell to a neighbor at least as high. Reversing that step gives a legal downhill or level flow back to the ocean.

Two ocean floods climb inward and overlap



Pseudocode

```
run reverse DFS from all Pacific border cells
run reverse DFS from all Atlantic border cells
reverse DFS may move to an equal-or-higher neighbor
answer = every cell reached by both traversals
return answer
```

Code 75: Complete solution: Pacific Atlantic Water Flow

```cpp
#include <vector>

using namespace std;

class Solution {
    void flow(const vector<vector<int>>& h,int r,int c,vector<vector<bool>>& seen){
        if(seen[r][c]) return; seen[r][c]=true;
        static const int d[5]={1,0,-1,0,1};
        for(int k=0;k<4;++k){int nr=r+d[k],nc=c+d[k+1];
            if(nr>=0&&nc>=0&&nr<(int)h.size()&&nc<(int)h[0].size()&&!seen[nr][nc]&&h[nr][nc]>=h[r][c]) flow(h,nr,nc,seen);
        }
    }
public:
    vector<vector<int>> pacificAtlantic(vector<vector<int>>& heights){
        int m=heights.size(),n=heights[0].size();
        vector<vector<bool>> pac(m,vector<bool>(n)),atl=pac;
        for(int r=0;r<m;++r){flow(heights,r,0,pac);flow(heights,r,n-1,atl);}
        for(int c=0;c<n;++c){flow(heights,0,c,pac);flow(heights,m-1,c,atl);}
        vector<vector<int>> out;
        for(int r=0;r<m;++r)for(int c=0;c<n;++c)if(pac[r][c]&&atl[r][c])out.push_back({r,c});
        return out;
    }
};
```

Why It Works and Complexity

Each ocean traversal visits each cell at most once, so time and reachability space are $O(mn)$; recursion can also reach $O(mn)$. Intersecting the two visited grids is needed: union would include cells reaching only one ocean.

Walk Through the Example

Trace the example one update at a time

Seed the Pacific search from the top and left borders and the Atlantic search from the bottom and right. From an ocean cell, move only to neighbors of equal or greater height. A high interior cell reached by both searches appears in both Boolean grids and is added to the result.

Why It Works

A reverse edge from height h to a neighbor of height at least h corresponds to a legal forward water step back toward the ocean. So each visited grid contains exactly the cells that can reach its ocean. Their intersection is exactly the requested answer.

Tests to try

Check one cell, one row, one column, all equal heights, strictly increasing terrain, strictly decreasing terrain, plateaus, and cells lying directly on both ocean boundaries.

How to explain it in an interview

Instead of starting a repeated downhill search from every cell, I reverse the graph and launch two multi-source traversals from the ocean borders. Cells visited by both traversals can flow to both oceans.

A Closer Look

Water normally flows from a cell to an equal or lower neighbor. Starting from every cell and simulating that direction repeats a great deal of work. Reverse the question: begin at each ocean and move from a cell to neighbors with equal or greater height. Every reached cell can flow back down to that ocean.

Run one multi-source DFS or BFS from the Pacific borders and another from the Atlantic borders. Store the reached cells in two Boolean grids. A cell belongs in the answer when both grids mark it. The reverse height comparison is the most important detail; using the original downhill comparison from the ocean would move in the wrong direction. Each ocean traversal processes a cell at most once, so total time and storage are $O(mn)$. Corners may begin in both searches, and equal-height plateaus are connected in both directions.

C++ Reference and Guided Practice

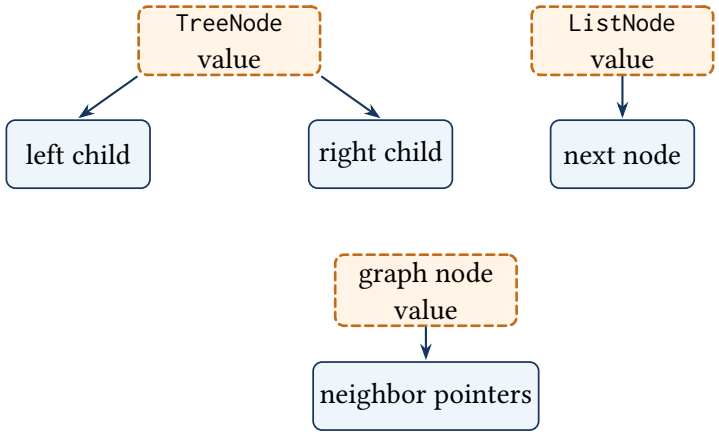
The solutions use the signatures expected by an online judge. This reference collects the node definitions that are normally supplied by the judge and clarifies the difference between borrowed links and owned objects. It also provides a small set of exercises for turning the completed lessons into active interview practice.

Implementation validation

All 75 complete solutions are checked independently with a C++ compiler. The audit includes judge-provided tree, list, and graph types, so a solution cannot pass merely because another chapter supplied a missing type or header. Representative examples and boundary cases are also exercised for each algorithm family.

Judge-provided pointer structures

Values and links in the three common node types



The pointer fields describe structure. In judge code, the judge normally owns the input nodes and the solution temporarily follows or rearranges those links.

Common judge definitions

Code 76: Typical linked-list and binary-tree definitions

```cpp
using namespace std;

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

Code 77: Typical graph-node definition

```cpp
#include <utility>
#include <vector>

using namespace std;

class Node {
public:
    int val;
    vector<Node*> neighbors;

    Node() : val(0) {}
    Node(int value) : val(value) {}
    Node(int value, vector<Node*> links)
        : val(value), neighbors(move(links)) {}
};
```

Raw pointers and ownership

The examples use raw pointers because the judge owns the supplied structures and requires standard method signatures. Production C++ should make ownership explicit with RAII containers and smart pointers where one object genuinely owns another. A map used while cloning a graph stores lookup pointers; it does not become a second owner of the original or cloned nodes.

A reliable code-review pass

Before submitting a solution, read it once without thinking about the example. Check the following properties directly in the code.

Five checks before submission

1. Every input permitted by the rules reaches a valid base case or loop termination condition.
2. Array and string indexes are checked before access, especially after a pointer moves or a window shrinks.
3. Values needed by an update are copied before the original variables are overwritten.
4. Recursive and graph solutions mark, unmark, or memoize state at the exact point required by their key rule.
5. The stated complexity counts sorting, heap operations, recursion depth, and output storage consistently.

Guided practice with hints

For each exercise, first draw the changing state, then write the key rule and only afterward modify the implementation.

Practice prompts

Area	Exercise	Hint after drawing
Arrays and greedy	Return the buy and sell days, not only the profit.	Save the day when the running minimum changes and snapshot both days when the best profit improves.
Two pointers	Generalize 3Sum to an arbitrary target.	Compare the total to the requested target; the safe pointer directions do not change.
Trie recursion	Count every word matching a wildcard pattern.	Sum successful branches instead of returning after the first match.
Tree queues	Produce zigzag level order.	Keep the BFS queue unchanged and alter only the destination index inside each row.
Graphs	Return one valid course order.	Record vertices when they leave a zero-indegree queue; a short result still detects a cycle.
Dynamic programming	Reconstruct a minimum coin selection.	Store the final coin responsible for each improving transition.
Heaps	Support deletion from a median stream.	Combine the heaps with delayed deletion counts in hash maps.
Strings	Decode payloads containing digits and #.	The stored length, not a delimiter search, controls how many payload characters to read.

About the Author

Om Tripathi

Om Tripathi wrote *Cracking The LeetCode Interview* to make data structures and algorithms easier to see, trace, and rebuild. Each lesson connects an interview pattern to a concrete example, a visual state change, and a complete C++ solution.

The 75 problems are grouped by ideas that appear again and again. Once you can see what the algorithm keeps track of, the code becomes much easier to write again during an interview.

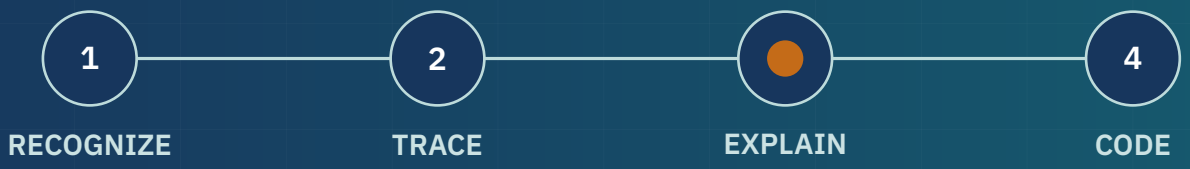
Author's note

Do not only read the diagrams. Cover the answer, draw the example yourself, and then write the code without looking. The aim is to understand the idea, not to memorize 75 separate programs.

CRACKING THE LEETCODE INTERVIEW

See the state.
Follow the change.
Rebuild the solution.

This book organizes 75 interview problems around the small set of ideas that appear repeatedly: what to keep, how it changes, and why each move is safe. Every lesson connects a visual dry run to complete C++ code, complexity analysis, common mistakes, and a practice checkpoint.



ARRAYS **HASHING** **TWO POINTERS** **WINDOWS**
STACKS **TREES** **HEAPS** **GRAPHS**
GREEDY **BINARY SEARCH** **DYNAMIC PROGRAMMING**

Om Tripathi

GITHUB EDITION
READ TRACE PRACTICE IMPROVE